队伍编号	MCB260XXXX
赛道	A

基于集成学习与条件生成模型的啤酒风味预测研究

摘 要

啤酒风味由数百种化合物共同决定。本文以 250 款啤酒的 231 项化学指标与 50 个感官属性为对象，先处理数据，再建立评分、关键化合物筛选与风格生成迁移三个模型，创新在于把检出限机制、属性相关图和化合物共表达模块分别嵌入插补、评分与约束迁移。以附件 8 的校准浓度为参照，查明附件 1 中 37 列为常用对数、33 列为原始浓度；可核验的 60 个缺失单元全为未检出，属左删失型缺失，建立**删失插补模型**，机制判定 60/60 正确，避免了常规插补约 3 个标准差的偏差。

针对问题一，题目要求只用化学数据给出品评表各维度的 0 至 6 分评分。首先以**岭回归、支持向量回归、极端随机树、梯度提升树**等权集成，再在附件 5 的属性相关图上做逐属性自适应权重的多目标堆叠，建立**堆叠集成评分模型**；然后由品评员标准化机理推出 0 分锚点与公共档宽 1.073，在训练折内估计锚点与方差校准后换算为分级，乙醛项用 **Beidler 饱和方程**给分。5 折 3 次重复交叉验证中平均 R^2为 0.260，点估计在 9 种方法中最高，与等权集成同属第一梯队，显著高于最好的单模型；中位 R^2为 0.198。严格折外评分的二次加权 Kappa 为 0.303，±1 档一致率 98.9%；额外缺失 40%时轻量集成的 R^2仅降 2.9%。Beer#1 至 Beer#12 的评分见附录 B 与支撑材料。

针对问题二，题目要求给出每个感官维度的 Top-3 关键化合物并论证稳定性与合理性。首先按无标度准则取软阈值β=4 构建共表达图，用 **Louvain 算法**划分出 6 个模块，分别对应酵母代谢、酒花萜类、野生菌等来源；然后融合 Spearman 相关、**TreeSHAP** 与 **Lasso 稳定性选择**三种重要性，在图上随机游走平滑并惩罚同模块冗余，建立**图去冗余筛选模型**。60 次 bootstrap 中模块层面入选频率为 0.808，与最稳定的基线相当、显著高于 Lasso；Top-3 内冗余仅为单一相关排序的 1/5，成对置信区间不含 0；71.4%的维度命中来源类别或文献标志物，随机选择仅 35.8%。

针对问题三，题目要求按风格生成化合物谱并把 Pils/Lager 迁移为 Tripel。首先经**高斯 copula** 变换后在 30 维**概率主成分**隐空间中为各风格建立向全体收缩的均值与协方差，建立**收缩条件生成模型**，生成谱的未检出率误差为 0.029，C2ST AUC 为 0.472，接近随机水平，22 个风格的生成画像与品评画像平均相关 0.82；然后以**高斯最优传输**为方向、按赛题以超阈值项界定缺陷并作线性约束，建立**约束最优传输模型**并用**二次规划**求解。迁移后 11 款中 6 款被判为 Tripel 且判别概率均超过 0.5，平均判别概率由 0 升至 0.502，按赛题口径的缺陷恶化由 20 项次降为 0，没有新增越阈项，判别概率也高于缺陷浓度一律不升的严格方案（0.463）。

关键词：删失插补　多目标堆叠　共表达网络　条件生成　最优传输

目录

一、问题重述.....1
1.1 问题背景.....1
1.2 问题要求.....1
二、问题分析.....1
2.1 数据分析.....1
2.2 问题一的分析.....1
2.3 问题二的分析.....2
2.4 问题三的分析.....2
2.5 问题总体思路.....2
三、模型假设.....3
四、符号说明.....3
五、数据预处理.....4
5.1 尺度核验.....4
5.2 离群值缩尾.....4
5.3 缺失值插补.....4
六、问题一：堆叠集成评分模型.....6
6.1 思路框架.....6
6.2 机理分析.....6
6.3 模型的建立.....6
6.4 模型的求解与检验.....8
七、问题二：图去冗余筛选模型.....11
7.1 思路框架.....11
7.2 机理分析.....11
7.3 模型的建立.....12
7.4 模型的求解与检验.....13
八、问题三：收缩生成与约束迁移模型.....16
8.1 思路框架.....16
8.2 机理分析.....16
8.3 生成模型的建立.....17
8.4 约束迁移模型的建立.....18
8.5 生成模型的求解与检验.....18
8.6 迁移模型的求解与检验.....21
九、模型的灵敏度分析.....23
十、模型的评价、改进与推广.....24
10.1 模型的优点.....24
10.2 模型的缺点.....24
10.3 模型的改进.....24
10.4 模型的推广.....24
参考文献.....25
AI 工具使用声明.....25
附录.....26

一、 问题重述

1.1 问题背景

啤酒以麦芽和水为主要原料，加入酒花后经酵母发酵酿成。它的香气、口味与口感由数百种风味化合物共同决定：酒花带来萜类化合物，形成草本与果香；酵母代谢产生酯类和高级醇，带来水果与溶剂气息；麦芽、野生菌群、香料和陈化过程又各自贡献一批物质。化合物之间相互掩蔽、协同，浓度与感知强度之间多为非线性关系，因此从化学成分出发预测风味，一直是食品感官科学中的难题[1,2]。

赛题给出一个商用啤酒风味代谢数据集，由附件 1 至附件 8 组成：附件 1 为 250 款啤酒的 231 项化学指标，其中一部分已取对数；附件 2 为化学指标之间的相关矩阵；附件 3 为训练有素的品评小组使用的 0 至 6 分品评表；附件 4 为 250 款啤酒在 50 个感官属性上经标准化处理的品评得分；附件 5、6 分别为感官属性之间、化学指标与感官属性之间的相关矩阵；附件 7 为消费者论坛评论提取的 187 个描述词得分；附件 8 收录了阈值、名称对照、校准浓度和价格等参考数据。数据中存在真实的缺失值与异常值。

1.2 问题要求

基于上述数据完成以下三个任务。

问题一：数据缺失条件下的感官评分。以附件 1 的化合物数据为基础，按附件 3 定义的 Aroma 与 Flavour 两套、共约 40 个感官维度，给出 0 至 6 分制的分级评分结果。

问题二：化合物与风味的关系分析。利用附件 2 构建化合物共表达图，结合附件 1 的特征，给出每个感官维度的 Top-3 关键化合物，并论证其稳定性与化学合理性。稳定性可以用 bootstrap 重采样下的入选频率，或不同建模方法之间 Top-3 的 Jaccard 系数来量化。

问题三：风格条件下的风味指纹生成与迁移。以 tasting_category_fine 为风格标签，学习风格条件生成模型，在给定风格（如 Tripel、Kriek、Lambic）下生成合理的化合物谱并给出感官画像；同时完成跨风格迁移，例如把 Pils/Lager 的化合物谱迁移为 Tripel 风格，并保证 DMS、Diacetyl、4VG、trans-2-nonenal、Stale hops 等超阈值缺陷项不恶化。

二、 问题分析

2.1 数据分析

三个问题共用附件 1 的化学数据，数据质量决定后面所有结论的可信度，所以先做统一的预处理。首先，附件 1 的 231 列并不处在同一尺度上，赛题只说部分数值取过对数，需要逐列核实底数与正负值的含义；然后，数据中有 73 个缺失单元和一批离群值，需要查清缺失的成因，再选择插补方法；最后，附件 4 的感官得分是标准化后的连续值，要回到 0 至 6 分制，需要先明确标准化的口径。这几项工作放在第五章完成，结果供三问共用。

2.2 问题一的分析

问题一要求只用化学数据给出品评表上约 40 个维度的 0 至 6 分评分。首先，附件 3 的品评表与附件 4 的 50 个属性一一对应，唯独品评表里的乙醛一项在附件 4 中没有标签，需要单独处理；然后，以附件 4 为监督信号建立从化学谱到 50 个标准化得分的回归模型，属性之间存在很强的相关（附件 5），可以用多目标堆叠利用属性间的相关信息，建立堆叠集成评分模型；最后，根据品评员标准化的机理，用 0 分锚点（或中位数锚点）与公共档宽把连续得分换算为 0 至 6 分，用交叉验证的袋外预测给出 Beer#1 至 Beer#12 的评分，并检验在缺失率继续上升时模型是否稳健。

2.3 问题二的分析

问题二要求在共表达图的基础上，为每个感官维度找出 Top-3 关键化合物。首先，把附件 2 的相关矩阵转换为加权网络，按无标度拓扑准则选取软阈值，计算拓扑重叠并划分化合物模块，得到共表达图的结构；然后，在问题一所用特征的基础上，从单变量相关、梯度提升树 SHAP 值、Lasso 稳定性选择三个角度度量化合物的重要性，三者融合后在共表达图上做随机游走平滑，再用最大边际相关准则惩罚同一模块内的冗余化合物，建立图去冗余筛选模型；最后，用 bootstrap 入选频率和方法间 Jaccard 系数检验稳定性，用化合物来源类别与文献标志物检验化学合理性。

2.4 问题三的分析

问题三要求学习风格条件生成模型并完成跨风格迁移。首先，22 个风格中最少的只有 3 款，直接估计每个风格 231 维的分布不可行，需要先把化学谱压缩到概率主成分隐空间；然后，在隐空间中为每个风格建立均值与协方差，并按样本量向全体收缩，解码时加入残差噪声与检出下限截断，建立收缩条件生成模型，生成谱再送入问题一的模型得到感官画像；最后，把 Pils/Lager 的谱沿两个风格分布之间的高斯最优传输映射推向 Tripel，保留每款啤酒自身的残差，并按赛题注释以超阈值项界定缺陷，约束缺陷化合物不新增越阈（已超阈的不再升高），求解二次规划，用问题一的模型复核缺陷感官得分。

2.5 问题总体思路

三问的关系如图 1 所示：第五章的预处理为三问提供统一的数据；问题一的感官预测模型在问题三中被用来给生成谱打分，并复核迁移后的缺陷得分；问题二的共表达模块结构又解释了问题三中缺陷约束为何会牵动一批相关化合物。

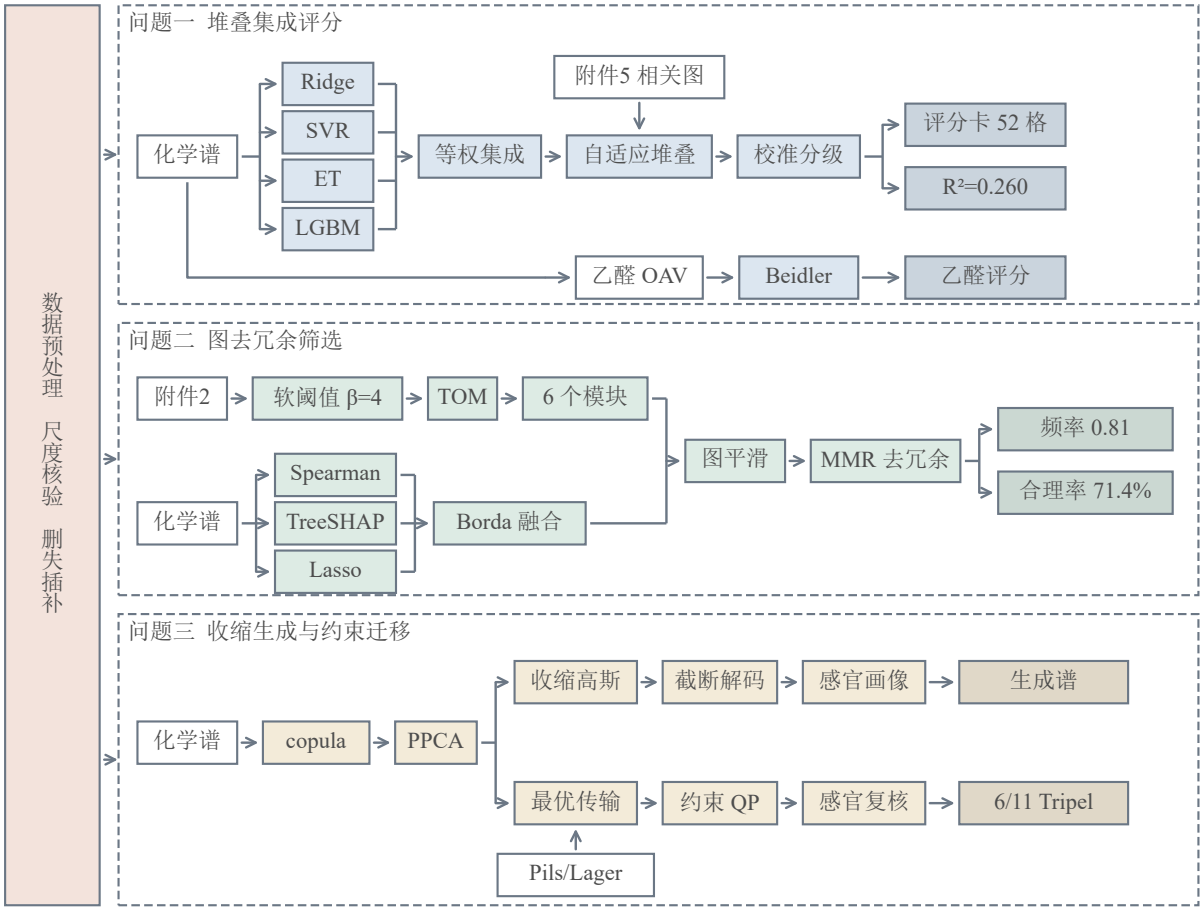


图 1 总体思路

三、 模型假设

（1） 附件 4 的感官得分由品评员原始 0 至 6 分按品评员标准化后取平均得到[1]，标准化只改变尺度，不改变同一属性内各款啤酒的强弱次序。

（2） 附件 1 的缺失不是人为随意删除造成的：经附件 8 核实的 60 个缺失单元为仪器未检出（左删失）；aroma_hops_sum 的 13 个缺失没有真值可核，按其与酒花化合物的关联视为高值一端的非随机缺失。

（3） 化学谱经 copula 变换并压缩到概率主成分隐空间后，同一风格的啤酒近似服从多元正态分布，不同风格共享一部分协方差结构，小样本风格的参数可以借助全体样本进行估计。

（4） 化合物浓度对感知强度的作用可以通过对数浓度刻画；DMS、双乙酰、4VG、反-2-壬烯醛等指定缺陷化合物超过感官阈值才记为缺陷，其余化合物超过阈值只形成相应的风味。

（5） 品评与化学测量针对同一批次样品完成，样品在两类测量之间没有发生陈化。

四、 符号说明

表 1 符号说明

符号	说明	单位
$\boldsymbol{x}_i \in \mathbb{R}^p$	第 i 款啤酒统一到 $\log_{10}$ 尺度后的化学指标向量，$p = 231$	–
ℓ_j	第 j 项化学指标的检出下限（取对数后）	–
y_{ik}	第 i 款啤酒在第 k 个感官属性上的标准化得分（附件 4）	–
g_{ik}	0 至 6 分制下的分级评分	分
Δ	一个评分档位对应的标准化得分宽度	–
ρ_{jl}	化学指标 j 与 l 的 Spearman 相关系数（附件 2）	–
β	共表达网络的软阈值幂次	–
T_{jl}	拓扑重叠（TOM）相似度	–
s_{kj}	化合物 j 对属性 k 的融合重要性得分	–
η, ω	图平滑系数与冗余惩罚系数	–
f_{kj}	bootstrap 重采样中化合物 j 进入属性 k 的 Top-3 的频率	–
$\boldsymbol{z}$	化合物谱在概率主成分隐空间中的坐标	–
$\boldsymbol{\mu}_c$, $\boldsymbol{\Sigma}_c$	风格 c 在隐空间中的均值与协方差	–
κ, ν	风格均值与协方差向全体收缩的强度	–
$\boldsymbol{W}$, σ^2	概率主成分的载荷矩阵与噪声方差	–
$\boldsymbol{S}_c$, $\boldsymbol{S}_w$	风格 c 的隐空间样本协方差、合并组内协方差	–
$\boldsymbol{A}$	两个高斯分布之间最优传输映射的线性部分	–
D, τ_d	缺陷化合物集合及其感官阈值	同浓度
α	风格迁移强度，$\alpha = 1$ 为完全迁移	–
OAV_j	气味活度值，浓度与感官阈值之比	–

其他符号在文中出现时具体说明。

五、 数据预处理

三问共用附件 1 的化学数据，本章先核实各列的尺度，再处理离群值与缺失值，得到统一的化学矩阵 $\boldsymbol{X} \in \mathbb{R}^{250\times 231}$。

5.1 尺度核验

附件 8 的 Calibrated chemical dataset 给出了其中 70 项指标的校准浓度 c_{ij}。对每一列拟合 $x_{ij} = a_j \log_{10} c_{ij} + b_j$ 与 $x_{ij} = a'_j c_{ij} + b'_j$，比较两种关系的线性程度，结论如下。

（1）37 列满足 $a_j = 1.000$、$b_j = 0.000$，相关系数为 1，说明这些列是浓度的常用对数，底数为 10 而不是 e。若底数为 e，斜率应为 $\ln 10 \approx 2.303$。负值对应浓度小于 1 的情形，例如 pH 的中位数 0.616 对应 $10^{0.616} = 4.13$。赛题说明中称乙醇、色度、苦味值、pH、CO_2 保留原始量纲，核验发现它们同样取过对数。

（2）其余 33 列与校准浓度的比值中位数为 1.00，是未取对数的原始浓度，未检出处以 10^{-9} 至 10^{-6} 量级的极小正数占位。

（3）没有校准值的 161 列是 GC-MS 峰面积的对数，其中 101 列存在大量取值恰为 -1.818 的样本，这是未检出时的占位值，即该列的检出下限。

于是全部列可以统一到 $\log_{10}$ 尺度。对原始浓度列取 $\ell_j = \log_{10}(c_j^{\min}/2)$，其中 $c_j^{\min}$ 为该列最小的有效检出值，令

$$x'_{ij} = \log_{10} \max\left(x_{ij},\, 10^{\ell_j}\right), \tag{1}$$

已取对数的列保持不变，各列的最小值即为其检出下限 ℓ_j。另外，用附件 1 计算的 Spearman 相关矩阵与附件 2 的平均绝对差只有 0.001，说明附件 2 是 Spearman 相关。赛题正文称其为 Pearson 相关，附件中的数据说明称其为 Spearman 相关，核验结果支持后者。另外，赛题称共有 226 种啤酒，附件 1 实际为 250 款，226 是参考文献 [1] 中化学指标的个数，本文以附件的实际数据为准。

后文只使用 pH、色度、苦味值等对数值的相对大小，不把它们当作物理浓度来解释。

5.2 离群值缩尾

化学指标中有大量未检出的占位值，若按全列计算标准差，结果会受其影响而产生偏差。因此只在检出值上计算中位数 m_j 与中位数绝对偏差 MAD_j，把偏离超过 $5 \times 1.4826\,\mathrm{MAD}_j$ 的值缩尾到边界：

$$\tilde{x}_{ij} = \min\Big(\max\left(x'_{ij},\, m_j - 5\sigma_j\right),\, m_j + 5\sigma_j\Big), \qquad \sigma_j = 1.4826\,\mathrm{MAD}_j. \tag{2}$$

共有 254 个单元被缩尾，分布在 58 列中，偏高与偏低的分别为 136 个和 118 个，集中在乳酸（27 个）、丁香酚（19 个）、β-大马酮（17 个）等少数在极个别啤酒中含量极高的化合物上。与校准浓度逐格比对还发现，原始浓度列中有 1 至 6 个单元与校准值相差超过 12%，属于赛题所说的合理噪声，缩尾后对后续分析的影响很小。

5.3 缺失值插补

附件 1 共有 73 个缺失单元，分布在 9 列。缺失值怎么补，取决于它为什么缺。附件 8 恰好给出了其中 8 列、共 60 个缺失单元的校准浓度，可以直接检验缺失机制，结果如下。

（1）这 60 个单元的真实浓度全部为 0，即仪器未检出。缺失只发生在浓度最低的一端，属于左删失型的非随机缺失[3]。

（2）芳香酒花总量 aroma_hops_sum 的 13 个缺失没有参照值，但缺失样本中有 6 款是 Hoppy 风格，其芳樟醇、香叶醇、香茅醇的对数均值分别比其余样本高 0.91、0.68、0.91，即缺失集中在酒花香气最浓的一端，属于右侧的非随机缺失。

两种机制如图 2 所示。图 2(a) 为 GC-MS 色谱：浓度 c_{ij} 越高峰越高，峰高低于检出限 ℓ_j 的化合物记为未检出，在附件 1 中或者记为占位的最低值，或者直接留空。图 2(b) 为化学矩阵，橙框的两列中，斜线格是留空的未检出单元，它们的真值与同列中记为 ℓ 的单元一样都在检出限以下，应补到 ℓ_j；绿框一列的缺失出现在酒花香气最浓的啤酒上，只能借共表达图上的邻居列来推断。

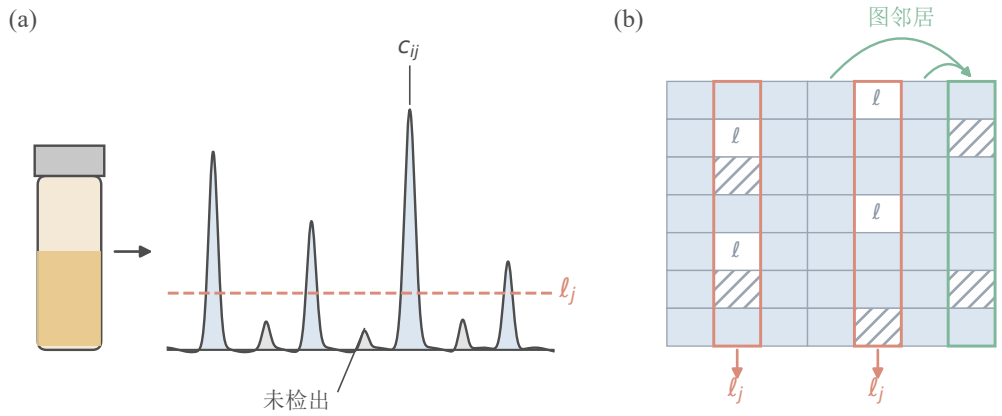


图 2　缺失机制：(a) 检出限；(b) 插补方向

两种机制下，均值、中位数、KNN、多重插补都会把缺失值估计为接近样本中心的值，与真实的偏离方向相反。为此建立删失插补模型：

$$\hat{x}_{ij}=\begin{cases}\ell_j, & j\in\mathcal{L}\text{（左删失列，未检出）},\\ \hat{\beta}_{j0}+\displaystyle\sum_{l\in\mathcal{N}_{15}(j)}\hat{\beta}_{jl}\tilde{x}_{il}+\sum_{c}\hat{\gamma}_{jc}\mathbb{1}[\text{风格}_i=c], & \text{其他列},\end{cases}\tag{3}$$

其中 $\mathcal{L}$ 为经附件 8 核实的 8 个左删失列，$\mathcal{N}_{15}(j)$ 为附件 2 中与第 j 列相关最强的 15 个化合物，即共表达图上的邻居，系数由岭回归估计，并迭代两轮。图上的邻居回归可以随邻居一起外推到高值区，不会像均值插补那样将酒花含量最高的啤酒估计为平均水平。

表 2　插补方法比较

插补方法	真实缺失 RMSE	真实缺失 MAE	随机遮蔽 RMSE
均值插补	3.796	3.735	0.988±0.057
中位数插补	3.975	3.913	1.025±0.059
KNN（$k=7$）	3.022	2.918	0.779±0.051
多重插补 MICE	3.184	3.058	0.771±0.050
共表达图邻居回归	3.161	3.036	0.733±0.045
删失插补（本文）	0.000	0.000	0.733±0.045

插补方法的比较如表 2 所示。真实缺失单元上，四种常规方法的误差都在 3 个标准差左右，说明它们把未检出的化合物估计为中等浓度，与实际情况不符。删失插补将这 60 个单元赋值为检出下限，与附件 8 的未检出记录一致。由于检出下限本身就是未检出值在本文尺度下的记法，这一项的误差为 0 是机制判断正确的结果，不应看作数值精度。随机遮蔽实验每次随机选 30 列、每列遮蔽 25 个已观测值，重复 10 次，用来检验随机缺失部分的效果，共表达图邻居回归的误差为 0.733，比 KNN 和 MICE 低 5% 左右。删失插补在非删失列上就是共表达图邻居回归，所以两者在随机遮蔽下的误差相同。综合两类检验：删失机制的判定在 60 个单元上全部正确（60/60），随机缺失部分与图邻居回归并列最好，后文统一使用插补后的矩阵 $\boldsymbol{X}$。

六、 问题一：堆叠集成评分模型

6.1 思路框架

问题一的求解路线如图 3 所示：先由化学谱回归 50 个标准化感官得分，再用属性之间的相关性做自适应堆叠，最后按品评员标准化的机理换算为 0 至 6 分。

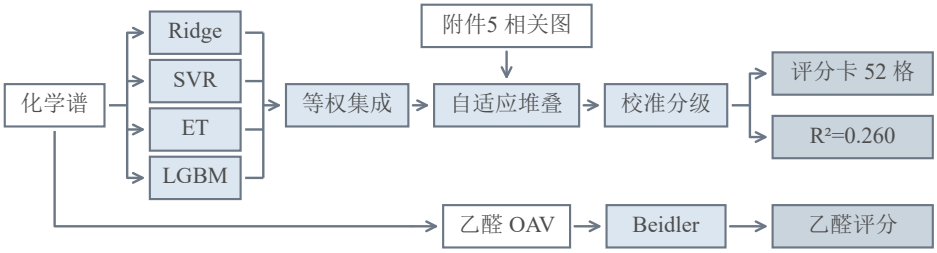


图 3 问题一框架

6.2 机理分析

附件 3 共有 12 张空白品评表，分别对应 Beer#1 至 Beer#12（赛题说明写 10 张，以实际文件为准），另有一张汇总表。每张品评表左侧为 Aroma（香气），列有麦芽、酒花、酯类三组各 5 项，以及乙醛、酒体、苦味、杀口感、余味、DMS、金属味、4VG、橙皮、芫荽、丁香、总体评价，共 27 项；右侧为 Flavour（口味），列有三组各 5 项，以及乙醛、酒精感、甜味、酸度、反-2-壬烯醛、双乙酰、陈酒花、乳酸、乙酸、畜棚味，共 25 项。两侧合计 52 个评分格，赛题称约 40 个维度，本文按表中实际的 52 格全部给出评分。汇总表的 50 列与附件 4 的 50 个属性一一对应；乙醛在 Aroma 与 Flavour 两侧各占一格，附件 4 中没有它的标签，由机理公式单独给分，两格共用同一个分数。因此 52 个评分格对应 51 个不同的维度，即 50 个附件 4 属性加乙醛。

附件 4 的得分不是原始分。参考文献 [1] 的方法部分说明，品评员先用 0 至 6 分打分，再按品评员做标准化和中心化，最后对出席的品评员取平均。由此可以推出两条性质，它们是换算回 0 至 6 分的依据。

（1）同一个属性内，所有品评员都打 0 分的啤酒会集中在最低值附近。数据中橙皮、芫荽、丁香等属性分别有 158、196、184 款啤酒落在最低值附近 0.1 以内，这一组即对应 0 分，记该属性的 1% 分位数 $y_{k,(1\%)}$ 为 0 分锚点。

（2）标准化以品评员为单位进行，每个品评员只有一个尺度，所以 1 分在不同属性上对应的标准化宽度相同，记为公共档宽 Δ。在出现 0 分锚点的属性中，极差最大的丁香属性跨度为 6.441，它最强的一款啤酒对应满分 6 分，于是 $\Delta = 6.441/6 = 1.073$。

麦芽总体、酒花总体、酯类总体、苦味、甜味、酒精感、酒体、杀口感、余味、总体评价这 13 个属性在每款啤酒上都能被感知，分布近似对称，没有聚集的最低值，因此改用中位数 $\tilde{y}_k$ 对应中间档 3 分作锚点。两类锚点如图 4 所示，图中圆点表示各款啤酒的得分，上方色带为 0 至 6 档，每档宽 Δ。

6.3 模型的建立

（1）基学习器。记标准化后的化学特征为 $\boldsymbol{x}_i$，属性 k 的得分为 y_{ik}。化学指标有 231 项而样本只有 250 个，既有线性关系也有阈值效应，因此选用四类机理互补的学习器：岭回归 Ridge 刻画线性主效应，径向基支持向量回归 SVR 刻画平滑非线性，极端随机树 ET 通过随机切分刻画阈值与交互，梯度提升树 LGBM 刻画分段的强非线性。四者等权平均，得到集成预测

$$\hat{y}_{ik}^{\mathrm{ENS}} = \frac{1}{4} \sum_{m \in \{\mathrm{Ridge,SVR,ET,LGBM}\}} \hat{f}_{mk}(\boldsymbol{x}_i). \tag{4}$$

（2）属性相关图上的自适应堆叠。附件 5 表明感官属性之间高度相关，例如酒精感

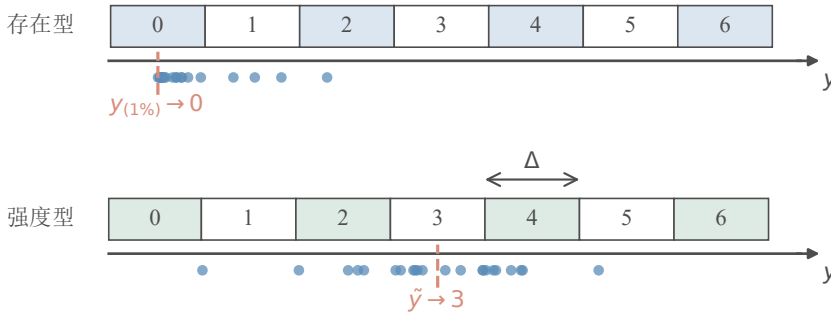


图 4　分级锚点

与酒体的相关系数为 0.79，酸度、乳酸、乙酸三者相互相关。以 $|r_{kl}| > 0.3$ 为边建立属性相关图，记 $\mathcal{A}(k)$ 为属性 k 与其邻居的集合，第二层用岭回归从邻居的集成预测中再预测属性 k，即多目标堆叠[5]（Multi-Target Stacking，记为 MTS）：

$$\hat{y}_{ik}^{\mathrm{MTS}} = \gamma_{k0} + \sum_{l \in \mathcal{A}(k)} \gamma_{kl}\, \hat{y}_{il}^{\mathrm{ENS}}. \tag{5}$$

第二层的训练数据取训练集上五折内层交叉验证的袋外预测，避免把第一层的过拟合带进第二层。堆叠对相关属性多、信号强的属性有利，对孤立的弱信号属性反而可能引入噪声。为此对每个属性单独选择堆叠权重 λ_k，在内层袋外预测上做网格搜索，称为自适应多目标堆叠（记为 AMTS）：

$$\lambda_k^{*} = \arg\min_{\lambda \in \{0, 0.1, \ldots, 1\}} \sum_{i} \left(y_{ik} - (1-\lambda)\hat{y}_{ik}^{\mathrm{ENS}} - \lambda\, \hat{y}_{ik}^{\mathrm{MTS}} \right)^2. \tag{6}$$

（3）方差再校准与分级。集成预测存在向均值收缩的倾向，直接分级会使高分档几乎消失。在每个训练折内，用内层袋外预测对每个属性拟合一条校准直线 $\tilde{y}_{ik} = a_k + b_k \hat{y}_{ik}$，并用该训练折的标签重新估计 0 分锚点、中位数锚点与档宽（各折档宽在 0.85 至 1.07 之间，下文的 $\Delta = 1.073$ 为全样本估计值，用于把品评得分换算为参照评分），测试折只做变换，不参与任何拟合，再按两类锚点换算：

$$g_{ik} = \mathrm{clip}\Big(\mathrm{round}\big(\phi_k(\tilde{y}_{ik})\big),\, 0,\, 6\Big), \qquad \phi_k(y) = \begin{cases} \dfrac{y - y_{k,(1\%)}}{\Delta}, & k \text{ 为存在型属性}, \\[2ex] 3 + \dfrac{y - \tilde{y}_k}{\Delta}, & k \text{ 为强度型属性}. \end{cases} \tag{7}$$

（4）乙醛项。附件 4 没有乙醛的标签，改用气味活度值 $\mathrm{OAV} = c/T$ 与嗅觉受体结合的 Beidler 饱和方程[4] 给分：

$$g_i^{\mathrm{ac}} = \mathrm{round}\left(6 \cdot \frac{\mathrm{OAV}_i}{\mathrm{OAV}_i + K} \right), \qquad K = 5, \tag{8}$$

$K = 5$ 使浓度恰好达到阈值时得 1 分。附件 8 给出的乙醛阈值为 50 mg/L，高于全部 250 款啤酒的实测值（最高 18.6 mg/L），按此阈值所有啤酒都是 0 分，因此改用文献中啤酒乙醛的风味阈值 10 mg/L[2]。需要说明的是，附件 4 没有乙醛的感官标签，附件 7 论坛描述词中的乙醛香气与乙醛浓度的 Spearman 相关只有 -0.06，无法用来检验，因此乙醛一项是依据阈值的机理推断，没有经过数据检验。

综上所述，得到**堆叠集成评分模型**：

$$
\begin{cases}
\hat{y}_{ik}^{\mathrm{ENS}} = \dfrac{1}{4}\sum_{m} \hat{f}_{mk}(\boldsymbol{x}_i), & k = 1, \dots, 50, \\
\hat{y}_{ik}^{\mathrm{MTS}} = \gamma_{k0} + \sum_{l \in \mathcal{A}(k)} \gamma_{kl}\, \hat{y}_{il}^{\mathrm{ENS}}, & \mathcal{A}(k) = \{ l : |r_{kl}| > 0.3 \}, \\
\hat{y}_{ik} = (1 - \lambda_k^{*})\, \hat{y}_{ik}^{\mathrm{ENS}} + \lambda_k^{*}\, \hat{y}_{ik}^{\mathrm{MTS}}, & \\
g_{ik} = \mathrm{clip}\big(\mathrm{round}(\phi_k(a_k + b_k \hat{y}_{ik})), 0, 6\big), & \\
g_i^{\mathrm{ac}} = \mathrm{round}\big(6\,\mathrm{OAV}_i / (\mathrm{OAV}_i + 5)\big), & \Delta = 1.073.
\end{cases}
\tag{9}
$$

6.4 模型的求解与检验

求解步骤。化学特征 231 维而样本只有 250 个，所有超参数和堆叠权重都必须在训练折内确定，否则评价会偏乐观。求解步骤如下：

Step1：按风格分层（样本少于 5 的风格合并为一层）做 5 折划分，重复 3 次，共 15 个训练/测试对；

Step2：每个训练折上拟合四个基学习器，岭回归的惩罚系数在折内由广义交叉验证选取；

Step3：在训练折内部再做 5 折，得到集成模型的内层袋外预测，用于训练堆叠层；

Step4：对内层袋外预测再做一次 5 折，得到堆叠层的袋外预测，在 $\{0, 0.1, \dots, 1\}$ 上为每个属性选出 λ_k^{*}；

Step5：在训练折内用内层袋外预测拟合方差校准直线，并由训练标签估计分级锚点与档宽；

Step6：把训练好的模型、校准直线与锚点用于测试折，得到连续分级，三次重复平均后取整。Beer#1 至 Beer#12 的评分直接取它们的袋外结果，回归、校准与锚点都没有用到它们自己的感官标签。

回归精度对比。9 种方法的交叉验证结果见表 3。

表 3　问题一方法对比

模型	平均 R^2	重复间标准差	中位 R^2	平均 ρ	$R^2 > 0$ 属性数
岭回归 Ridge	0.2179	0.0054	0.1452	0.4052	49
偏最小二乘 PLS	0.2083	0.0013	0.1475	0.3931	50
支持向量回归 SVR	0.2008	0.0030	0.1289	0.3994	49
随机森林 RF	0.2412	0.0049	0.1758	0.4278	50
极端随机树 ET	0.2437	0.0057	0.1820	0.4343	50
梯度提升 LGBM	0.2315	0.0049	0.1983	0.4052	44
等权集成 ENS	0.2567	0.0045	0.2024	0.4392	50
集成 + 全量堆叠	0.2564	0.0037	0.1952	0.4292	49
集成 + 自适应堆叠（本文）	**0.2601**	0.0038	0.1975	0.4373	49

由表 3 可以看出三点。第一，树模型整体优于线性模型，ET、RF 的平均 R^2 为 0.244 和 0.241，岭回归、PLS、SVR 只有 0.20 至 0.22，说明化学谱与感官之间存在明显的阈值与交互效应。第二，四类学习器等权集成后 R^2 升至 0.257，比最好的单模型 ET 高 5.3%，说明四者的误差方向不一致，集成后可部分抵消。第三，对全部属性一律堆叠的 MTS 与集成持平（0.256），自适应堆叠 AMTS 的平均 R^2 为 0.260，点估计在 9 种方法中最高，比 LGBM 高 12.4%，重复间标准差为 0.0038。但 AMTS 的中位 R^2（0.1975）与 $R^2 > 0$ 的属性数（49 个）都略低于 ENS（0.2024、50 个），中位 R^2 只有 0.20 左右，说明化学

谱对多数感官属性的解释力有限。参考文献 [1] 用同一批啤酒训练的梯度提升模型，在专家小组数据上的 R^2 为 0.2 左右，本文结果与之相当且略高。

自适应权重的分布与设计预期一致：50 个属性的 λ_k^* 平均为 0.347，麦芽味总体（0.76）、酸度（0.68）、酒体（0.65）、麦芽味焦糖（0.63）等 13 个属性取值超过 0.5，它们大多属于附件 5 中相关性较强的属性族，可以利用相邻属性的预测信息。例外是酯香·花香（0.69），它自身的 R^2 只有 0.047，自身信号很弱，主要依靠果香、总体酯香等相邻属性的预测；仅 1 个属性的权重为 0。

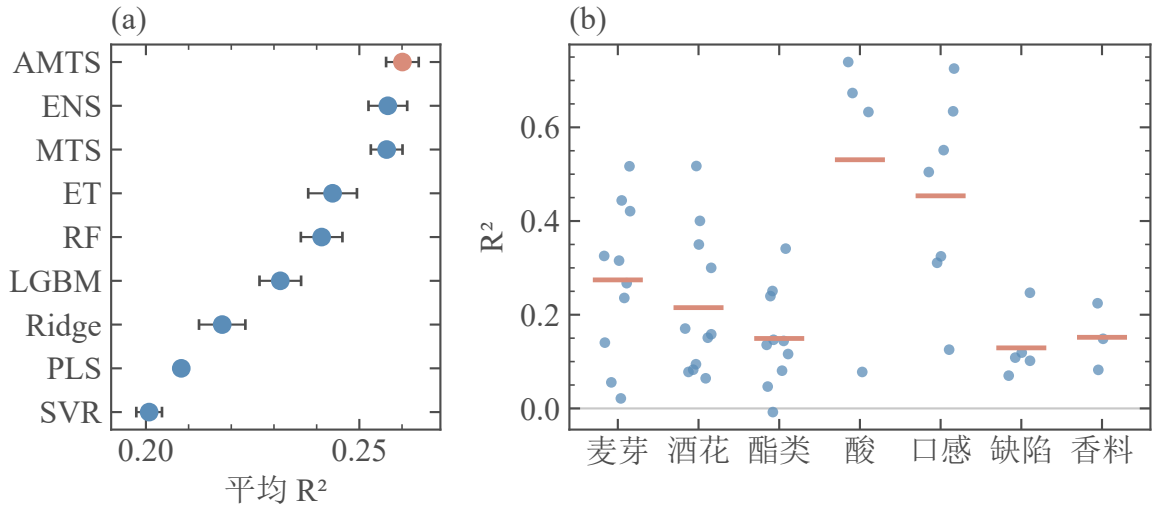


图 5　回归精度：(a) 方法对比；(b) 按类别分组的逐属性 R^2

为判断这些差距是否显著，对 15 个训练/测试对逐折计算平均 R^2 并做配对比较：AMTS 比 ENS 平均高 0.0008，95% bootstrap 置信区间为 $[-0.0037, 0.0047]$，15 折中 12 折占优，Wilcoxon 符号秩检验 $p = 0.19$，差异不显著，二者同属第一梯队；AMTS 比全量堆叠 MTS 高 0.0087（$[0.0059, 0.0118]$，15 折全部占优，$p < 0.001$），比最好的单模型 ET 高 0.0121（$[0.0078, 0.0167]$，14 折占优，$p < 0.001$）。重复交叉验证的各折并不完全独立，这里的 p 值偏乐观，只作参考。也就是说，自适应权重避免了全量堆叠在弱属性上的损失，集成相对单模型的提升是确定的，而 AMTS 相对等权集成的提升还在波动范围之内。

如图 5 所示，图 5(a) 中珊瑚色点为本文的 AMTS，误差棒为三次重复的标准差，AMTS 与 ENS 的差距小于误差棒，与 ET 的差距明显大于误差棒。图 5(b) 把 50 个属性的逐属性 R^2 按感官类别展开，短横线为类内均值：酸类属性最容易预测，酸度、乳酸、乙酸的 R^2 分别为 0.739、0.673、0.633，因为它们直接对应乳酸、乙酸浓度和 pH 这几项化学指标；口感类次之，酒精感 0.725、苦味 0.634、酒体 0.551，分别对应乙醇、异 α-酸和残糖；缺陷与香料类最难，反-2-壬烯醛、陈酒花、花香的 R^2 都在 0.08 以下。这些属性由极少数痕量化合物决定，多数啤酒中察觉不到，样本里几乎没有可学习的变化。

分级评分的检验。把每款啤酒的品评得分按全体数据估计的锚点换算为 0 至 6 分，作为检验评分结果的参照；模型评分则只用各训练折估计的锚点与校准直线，与参照评分逐格比较，结果如表 4 所示。

由表 4 可以看出，模型评分与品评换算评分完全一致的比例为 66.2%，相差不超过一档的比例为 98.9%，平均档差 0.351。全部取中位数的基线也有 64.3% 的完全一致率，因为多数缺陷与香料属性本来就是 0 分，但它的二次加权 Kappa 为 0，没有任何区分能力；本文方法的 Kappa 为 0.303，按常用划分属于一般（0.2 至 0.4）一致，而不是强一致，折内方差再校准使 Kappa 从 0.292 提高到 0.303。表中最后一行为全量校准对照，校

表 4　分级一致性

方案	完全一致率	±1 档一致率	平均绝对档差	二次加权 Kappa
训练折中位数（基线）	64.3%	97.0%	0.395	0.000
直接分级	66.2%	99.0%	0.351	0.292
折内方差再校准后分级（本文）	66.2%	98.9%	0.351	0.303
全量事后校准（对照）	67.2%	98.9%	0.341	0.318

准直线用全部 250 款的标签拟合，测试标签参与了拟合，指标偏乐观约 0.015，只作对照。从分布看，品评换算评分中 0、1、2、3、4 分的比例为 42.3%、27.4%、8.3%、16.6%、5.1%，5 分占 0.4%，模型评分为 33.6%、39.6%、3.4%、20.2%、3.3%，没有 5 分，模型给 1 分偏多、0 分偏少，整体形状接近。

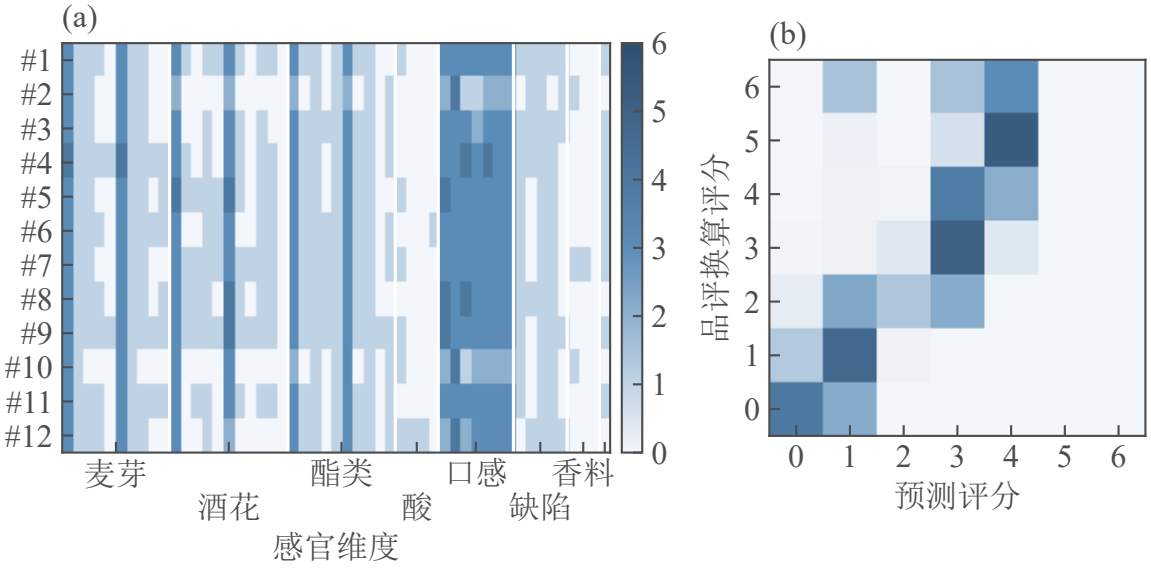


图 6　评分结果：(a) 12 款评分；(b) 评分对照

如图 6 所示，图 6(a) 为 12 款啤酒在 51 个维度上的评分，颜色越深分数越高，横轴按七类属性分组，最右一列为乙醛；图 6(b) 为全部 250 款啤酒、50 个维度上预测评分与品评换算评分的对照，每一行按行归一化，质量集中在对角线及其两侧。评分结果与啤酒风格高度吻合：两款无醇啤酒（#2、#10）的酒精感只有 1 分、甜味 4 分，与其限制发酵的工艺一致；烈性艾尔（#4）的酒精感、酒体、麦芽味都是 4 分；两款 Hoppy（#8、#9）的酒花味与苦味都是 4 分；Lambic（#12）是唯一在乳酸、乙酸上得分的啤酒；Saison（#7）是唯一在芫荽上得 1 分的啤酒，与比利时农舍啤酒常加香料的做法相符。12 款啤酒的评分与它们自身品评换算评分完全一致的比例为 67.0%，相差不超过一档的比例为 98.3%。品评表中谷物、面包、柑橘、香蕉等描述词是存在性勾选框，按模板公式取所属类别总体项的强度，与附件 8 中这些属性的数据类型为 categorical 一致，因此描述词的预测评分不低于 1 时勾选。12 张表由模板公式重新计算后，汇总页 600 个单元格与预测结果全部一致（汇总页 barnyard 列的引用错误已更正，见支撑材料）。完整评分如附录 B 的表 10 所示，填好的 12 张品评表与汇总表见支撑材料“问题一 _ 品评表评分.xlsx”。

缺失条件下的稳健性。在已插补的化学矩阵上再随机遮蔽 5% 至 40% 的单元，分别用均值、KNN 和共表达图邻居回归插补，再用 Ridge 加 ET 的轻量集成评估感官预测精度，每档重复 3 次，结果如图 7 所示。

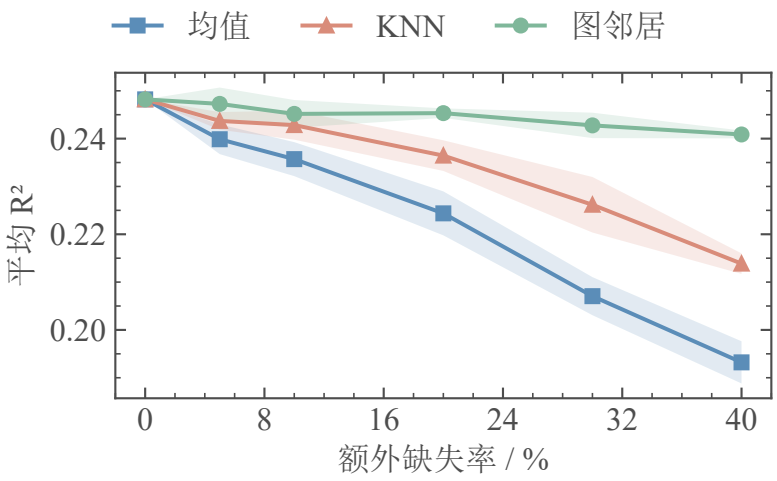


图 7　缺失稳健性

由图 7 可以看出，缺失率升到 40% 时，图邻居回归插补下的平均 R^2 从 0.248 只降到 0.241，降幅 2.9%；KNN 插补降到 0.214，降幅 13.8%；均值插补降到 0.193，降幅 22.2%。共表达图上的邻居携带了被遮蔽化合物的大部分信息，所以即使四成数据缺失，感官预测精度也基本不变。这一实验检验的是轻量集成的回归精度，没有重做分级。

七、　问题二：图去冗余筛选模型

7.1　思路框架

问题二的求解路线如图 8 所示：先由附件 2 构建加权共表达网络并划分模块，再在问题一所用的 231 维特征上从三个视角度量重要性，借共表达图做平滑和去冗余，得到 Top-3。

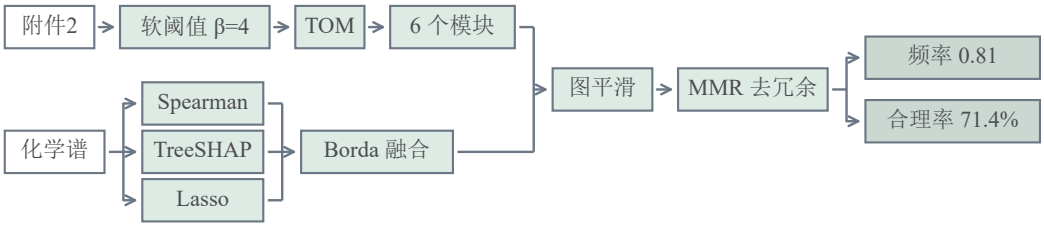


图 8　问题二框架

7.2　机理分析

同一来源的化合物往往同涨同落。酵母代谢同时产生乙醇、高级醇和乙酸酯，酒花投料同时带入一整批萜类，麦芽焙烤同时生成色素、呋喃与吡嗪。这些化合物在相关矩阵中聚成块，若直接按单个化合物的重要性排序，Top-3 容易由同一块中三个高度相关的化合物组成。它们反映的是同一条代谢途径，重新抽样后排在前面的可能是同块中的其他化合物，稳定性较差。因此需要先把相关矩阵转换为网络，识别块的结构，再在不同块之间选取代表性化合物。

这一机制如图 9 所示：圆圈为化合物 x_j，虚线圈出的三块是共表达模块，块内连线越粗表示拓扑重叠 T_{jl} 越大；右侧的酒杯代表感官属性 y_k，箭头为化合物对它的重要性 s_{kj}。左上模块中打叉的化合物单独看重要性较高，但它与已选中的 j_1 拓扑重叠很大，扣除 ωT_{jl} 后落选，最终的 j_1、j_2、j_3 分散在不同的模块中，对应不同的风味形成途径。实际计算中 Top-3 平均覆盖 2.08 个模块，强相关属性（如酒精感）的三个化合物也可能同

在一个模块。

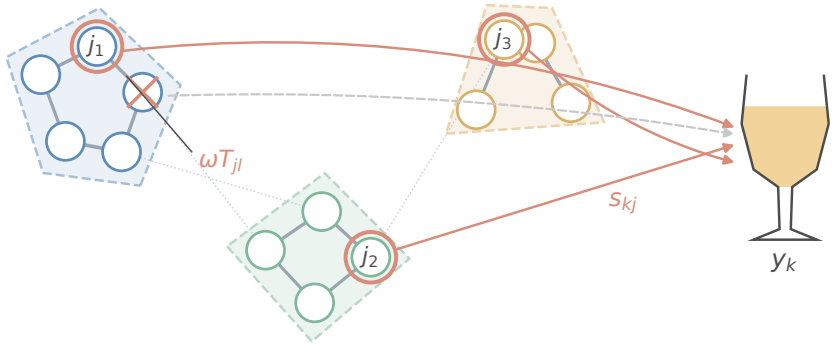


图 9　模块与去冗余

7.3　模型的建立

（1）加权共表达网络。以附件 2 的 Spearman 相关系数构造邻接矩阵 $a_{jl}=|\rho_{jl}|^{\beta}$，幂次 β 用于突出强相关、抑制弱相关。生物网络的连接度通常服从幂律分布 $p(k)\propto k^{-\gamma}$，按加权基因共表达网络分析[6] 的无标度准则，取使 $\log p(k)$ 对 $\log k$ 的线性拟合 $R^2\geq 0.8$ 的最小 β。两个化合物不仅自身相关，而且共享很多邻居时才算紧密，用拓扑重叠度量这种关系：

$$T_{jl}=\frac{\sum_u a_{ju}a_{ul}+a_{jl}}{\min(k_j,k_l)+1-a_{jl}},\qquad k_j=\sum_u a_{ju}. \tag{10}$$

以 T_{jl} 为边权，用 Louvain 算法[9] 最大化模块度来划分模块，不足 5 个化合物的模块归入灰色组。每个模块取成员化合物的第一主成分作为模块特征谱，与 50 个感官属性计算相关。

（2）三个视角的重要性。对属性 k 与化合物 j：

- 单变量视角 $v_{kj}^{(1)}=|\rho_S(x_{\cdot j},y_{\cdot k})|$，即 Spearman 相关；
- 非线性视角 $v_{kj}^{(2)}=\frac{1}{n}\sum_i|\phi_{ij}|$，即梯度提升树的 TreeSHAP 值[7]，三个随机种子取平均；
- 稀疏视角 $v_{kj}^{(3)}$ 为 Lasso 稳定性选择[8] 中化合物被选中的频率，每次随机抽取一半样本，惩罚系数取交叉验证的最优值。

（3）融合、图平滑与去冗余。三种重要性的量纲不同，先转为 Borda 秩分 $r_{kj}^{(m)}=1-(\mathrm{rank}_{kj}^{(m)}-1)/(p-1)$ 再取平均，然后在以 T 为权的共表达图上做一步随机游走平滑，让与强信号化合物共表达的化合物分得一部分证据，抑制单个化合物的测量噪声：

$$s_{kj}=(1-\eta)\,\bar r_{kj}+\eta\sum_l\frac{T_{jl}}{\sum_u T_{ju}}\,\bar r_{kl},\qquad \bar r_{kj}=\frac{1}{3}\sum_{m=1}^{3}r_{kj}^{(m)}. \tag{11}$$

最后按最大边际相关准则逐个选入化合物，后选入的化合物若与已选化合物拓扑重叠大则被扣分：

$$j_t=\arg\max_{j\notin S_{t-1}}\Big[s_{kj}-\omega\max_{l\in S_{t-1}}T_{jl}\Big],\qquad S_t=S_{t-1}\cup\{j_t\},\ t=1,2,3. \tag{12}$$

取 $\eta=0.3$、$\omega=0.6$，第九章检验了这两个参数的灵敏度。aroma_hops_sum 等 5 个由多种化合物加总得到的指标不是单一化合物，不进入候选集。

综上所述，得到**图去冗余筛选模型**：

$$
\begin{cases}
a_{jl} = |\rho_{jl}|^{\beta}, \quad \beta = \min\{\beta : R^2_{\mathrm{SF}}(\beta) \geq 0.8\}, \\
T_{jl} = \dfrac{\sum_u a_{ju} a_{ul} + a_{jl}}{\min(k_j, k_l) + 1 - a_{jl}}, \\
s_{kj} = (1-\eta)\bar{r}_{kj} + \eta \sum_l \tilde{T}_{jl} \bar{r}_{kl}, \quad \bar{r}_{kj} = \frac{1}{3}\sum_m \mathrm{Borda}\big(v^{(m)}_{kj}\big), \\
\mathrm{Top3}_k = \{j_1, j_2, j_3\}, \quad j_t = \arg\max_{j \notin S_{t-1}} \big[s_{kj} - \omega \max_{l \in S_{t-1}} T_{jl} \big].
\end{cases}
\tag{13}
$$

7.4　模型的求解与检验

共表达网络与模块。软阈值 β 从 1 增大到 6 时，无标度拟合 R^2 依次为 −0.19、0.06、0.52、0.85、0.91、0.97，平均连接度从 37.3 降到 0.55。$\beta = 4$ 是首个使 $R^2 \geq 0.8$ 的幂次，此时平均连接度为 1.71，网络既保留了强相关，又避免了弱相关使网络过度连通。在拓扑重叠图上用 Louvain 算法比较 0.6 至 1.5 五个分辨率，分辨率为 1 时模块度最高，为 0.344，得到 6 个模块，只有 2 个化合物归入灰色组。如图 10 所示，图 10(a) 中虚线为 0.8 的准则线，圆圈为选定的 β；图 10(b) 只画出拓扑重叠最强的 1.5% 的边，点的大小表示模块内连接度，颜色表示模块。

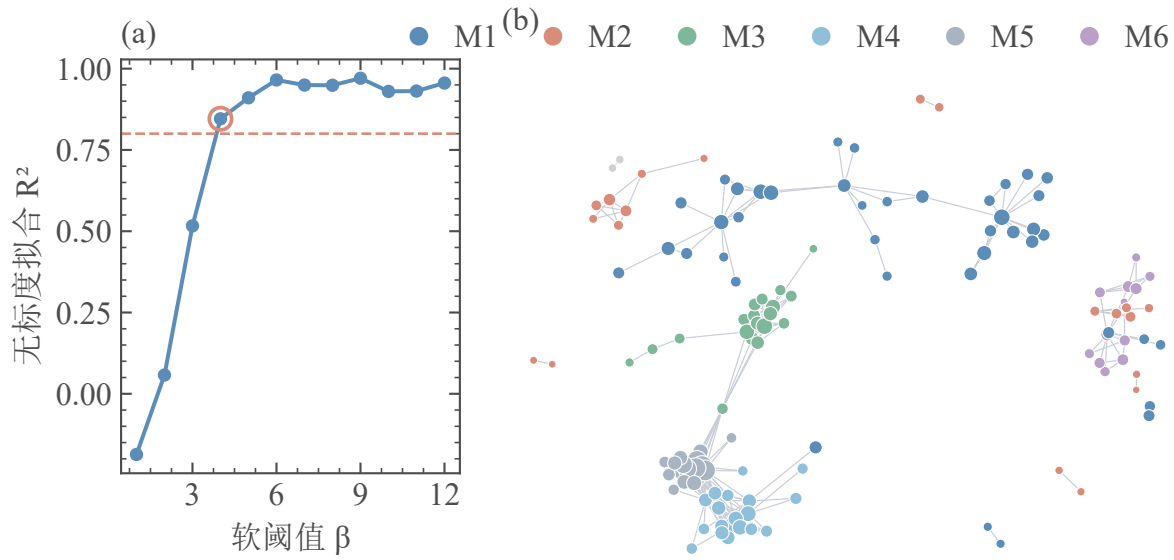


图 10　共表达网络：(a) 软阈值；(b) 模块

六个模块各有清楚的化学来源，模块特征谱与感官属性的相关如图 11 所示。

- M3（31 个，酵母来源 20 个）：枢纽为总酯、乙醇、乙酸异戊酯、乙酸异丁酯和热量值，是酵母发酵代谢模块，与酒精感、酒体、杀口感的相关分别为 0.74、0.60、0.49；
- M4（31 个，酒花来源 21 个）：以 α-木罗醇、表荜澄茄醇、葎草烯环氧化物等倍半萜氧化产物为主，与苦味、酒花味的相关为 0.45、0.42；
- M5（27 个）：辛酸戊酯、辛酸 2-甲基丁酯与 β-法尼烯等酯类与萜类共存，与苦味、酒花味、余味正相关；
- M6（21 个，酒花与香料来源 17 个）：芳樟醇、松油烯-4-醇、β-水芹烯等单萜，与橙皮、芫荽正相关，与麦芽香负相关，对应加香料的比利时风格；
- M1（78 个）：苯乙酸乙酯、邻甲酚、3-呋喃甲醇等野生菌与陈化来源物质，与乳酸、酸度正相关（0.39、0.38），与苦味、酒精感负相关，对应酸啤；
- M2（41 个）：2-甲基-1-丁醇、甲酸异戊酯等高级醇与 E-橙花叔醇，与指甲油酯香、酒精感正相关。

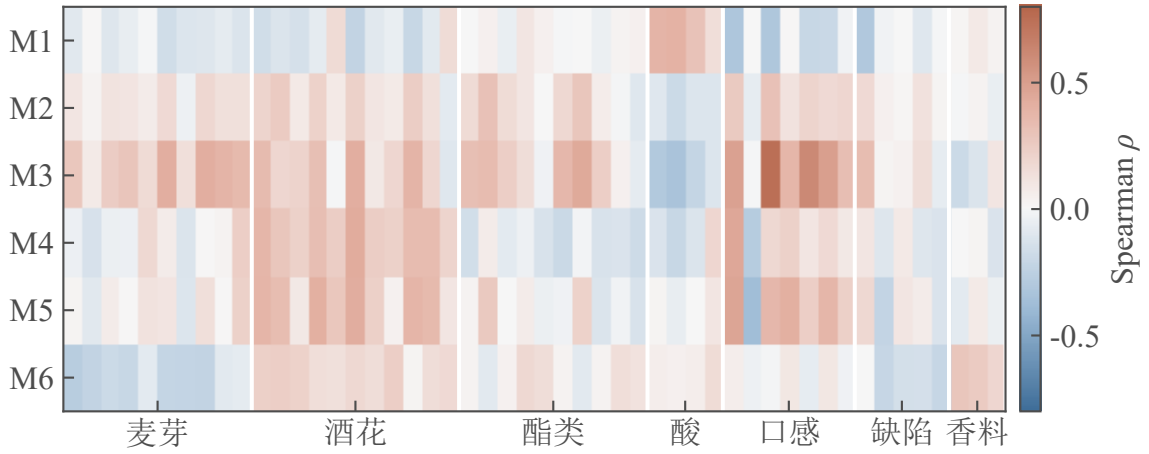


图 11　模块与感官相关

模块的划分与化合物的来源高度对应，而网络构建时只用了附件 2 的相关矩阵，没有用到来源信息，这说明共表达图反映了真实的代谢结构。

Top-3 关键化合物。部分代表性维度的 Top-3 如表 5 所示。Top-1 至 Top-3 按去冗余后的融合得分依次选出，括号内的 bootstrap 入选频率只衡量稳定性，不是排序依据，所以两者不一定单调，例如苦味的 Top-2 异 α-酸频率为 1.00，高于 Top-1。50 个维度的完整结果见附录 C 的表 11（化合物名沿用附件 2 的英文名）。

表 5　代表性 Top-3（按融合得分排序，括号内为 bootstrap 入选频率）

感官维度	Top-1	Top-2	Top-3
苦味	辛酸乙酯（0.92）	异 α-酸（1.00）	乙酸丁酯（0.28）
酒精感	乙醇（0.83）	CO_2（0.68）	辛酸乙酯（0.23）
酒体	乙醇（0.57）	甲硫醇（0.45）	色度（0.40）
甜味	糖（0.98）	癸酸乙酯（0.50）	铁（0.65）
酸度	pH（1.00）	乳酸（0.77）	4-乙基愈创木酚（0.42）
乳酸	乳酸（0.90）	pH（0.97）	乙酸异戊酯（0.40）
4VG	4-乙烯基愈创木酚（1.00）	甲酸橙花酯（0.18）	乳酸异戊酯（0.13）
麦芽味・焦糖	色度（1.00）	乙醛（0.77）	糖（0.23）
酯味・指甲油	乙酸乙酯（0.67）	Z,Z-香叶基芳樟醇（0.28）	甘油（0.42）
酯味・香蕉	乙酸异戊酯（0.32）	苯乙醇（0.40）	糖（0.17）
橙皮	柠檬烯（0.83）	樟脑（0.57）	β-水芹烯（0.10）
芫荽	柠檬烯（0.52）	芳樟醇（0.28）	乙基芳樟醇（0.20）

稳定性检验。对 250 款啤酒做 60 次 bootstrap 重采样，每次完整重跑三种重要性、融合、图平滑与去冗余，统计每个化合物进入 Top-3 的频率，并与四种对照方法比较，结果见表 6。

由表 6 可以看出：在单个化合物层面，本文方法的平均入选频率为 0.474，略低于单用 Spearman 相关（0.513）或 Borda 融合（0.515）。原因在于去冗余：单一相关排序的 Top-3 内平均拓扑重叠为 0.072，经常把同一模块里两三个几乎等价的化合物一起选上，换一次抽样它们仍会一起出现，于是频率偏高；本文方法的 Top-3 内拓扑重叠只有 0.014，三个化合物来自不同的代谢途径。同一模块内的化合物可以相互替代，因此更合理的稳定性应在模块层面衡量：本文方法所选化合物所属的模块在 bootstrap 中被选中的

表 6　筛选方法对比

方法	化合物层面		模块层面		Top-3 内 TOM	合理命中率
	入选频率	Jaccard	入选频率	Jaccard		
Spearman 相关	0.513	0.396	**0.819**	**0.681**	0.0722	67.3%
TreeSHAP	0.489	0.373	0.799	0.619	0.0317	73.5%
Lasso 稳定选择	0.422	0.307	0.704	0.537	0.0147	63.3%
Borda 融合	0.515	0.395	0.791	0.623	0.0306	65.3%
本文 GF-MMR	0.474	0.357	0.808	0.638	0.0144	71.4%

频率升到 0.808，模块集合的 bootstrap Jaccard 为 0.638，点估计在 SHAP、Lasso 稳定选择、Borda 融合这几种多变量方法中最高；与最稳定的单一 Spearman 排序（0.819）只差 0.011，而后者 Top-3 内的冗余是本文方法的 5 倍，平均只覆盖 1.82 个模块，本文方法覆盖 2.08 个。不同方法之间，本文方法与 Borda 融合的 Top-3 Jaccard 为 0.660，与 Lasso 稳定选择为 0.420，与单一 SHAP 为 0.356，说明它综合了三个视角而不偏向其中任何一个。

这些差距是否超出了抽样波动，用成对 bootstrap 检验：五种方法在同一组 50 个感官维度上计算各项指标，对维度做 5000 次有放回重抽样，得到本文方法与各基线差值的 95% 置信区间。Top-3 内拓扑重叠比 Spearman、TreeSHAP、Borda 融合分别低 0.058、0.017、0.016，置信区间都不含 0，去冗余的效果是确定的；模块层面入选频率比 Lasso 稳定选择高 0.104（$[0.059, 0.151]$），而与 Spearman、TreeSHAP、Borda 融合的差值分别为 -0.011、0.009、0.017，置信区间都包含 0，说明在稳定性上本文方法与这三种方法相当，并不显著更高。

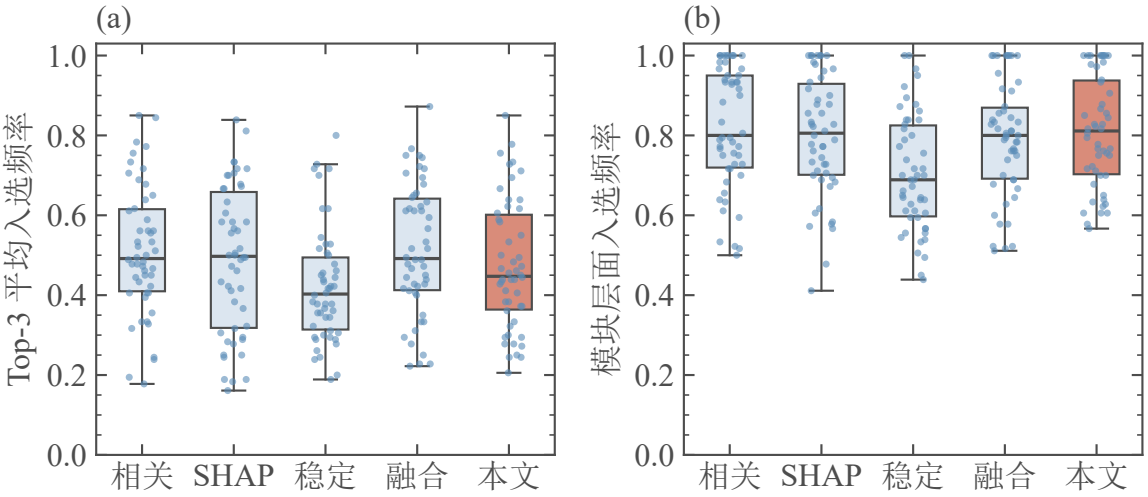


图 12　Top-3 稳定性：(a) 化合物层面；(b) 模块层面

如图 12 所示，每个点代表一个感官维度。图 12(a) 中五种方法的化合物层面频率分布相近，中位数都在 0.4 至 0.5 之间；图 12(b) 换到模块层面后，各方法的频率都升到 0.7 至 0.82，本文方法的中位数与 Spearman 相当、高于其余三种方法，这是去冗余的结果：所选化合物分布在不同的模块中，对应不同的代谢途径，每条途径在重采样中都能稳定出现。

化学合理性检验。对 49 个有明确预期的维度，若 Top-3 中至少有一个化合物的来源类别与维度一致（例如酒花味对应酒花来源），或属于文献公认的标志物（例如苦味对

应异 α-酸，酸度对应乳酸与 pH），则记为合理。本文方法命中 35 个维度，命中率 71.4%，而从候选化合物中随机抽 3 个的期望命中率只有 35.8%。几类关键关系都得到了印证：苦味的 Top-2 为异 α-酸，入选频率为 1.00；4VG 的 Top-1 为 4-乙烯基愈创木酚；甜味的 Top-1 为残糖；酸度与乳酸的 Top-2 都是 pH 与乳酸；指甲油酯味的 Top-1 为乙酸乙酯，香蕉酯味的 Top-1 为乙酸异戊酯[13]；橙皮与芫荽的 Top-1 都是柠檬烯，芫荽还选中了芳樟醇，与芫荽籽精油以芳樟醇为主的事实一致；麦芽焦糖与烟熏的 Top-1 都是色度，反映深色麦芽的美拉德反应产物。

不合理的维度集中在双乙酰、DMS、陈酒花、反-2-壬烯醛、畜棚味等缺陷类属性。这些属性在问题一中的 R^2 都低于 0.12，多数啤酒察觉不到，样本中缺少可学习的变化，任何方法都难以给出可靠的关键化合物。对这些维度，更合理的做法是直接依据单一化合物的感官阈值判断，这也是问题三中缺陷约束采用阈值的原因。

部分维度的 Top-3 在化学上较难直接解释，说明如下。苦味的 Top-1 是辛酸乙酯而不是异 α-酸，是因为辛酸乙酯在高度、重酒花的比利时艾尔中普遍偏高，与苦味的 Spearman 相关达 0.59，而真正的苦味来源异 α-酸（相关 0.68）以 1.00 的入选频率位列第二，结论仍成立；甜味 Top-3 中的铁、酒花味 • 柑橘中的蛋白质与土臭素，更可能反映风格之间的混杂效应，例如深色与水果啤酒铁含量高、苦味低、甜味高，这类化合物应理解为风格的标记而不是风味的成因。

与单一 TreeSHAP 相比，本文方法的合理命中率略低（71.4% 对 73.5%，成对差值 -0.020，置信区间 $[-0.143, 0.102]$），模块层面的稳定性略高（0.808 对 0.799），两项差异都不显著；确定的优势是 Top-3 内的冗余只有它的一半（0.014 对 0.032），覆盖的模块更多（2.08 对 1.94）。TreeSHAP 依赖单个梯度提升模型，更换学习器时结果会发生变化；本文方法在稳定性与合理性上与最好的基线相当，同时显著降低了冗余，在稳定性、合理性与冗余三项指标之间取得了较好的折中，因此作为最终方案。

八、 问题三：收缩生成与约束迁移模型

8.1 思路框架

问题三的求解路线如图 13 所示：共用的隐空间之后分为两支，上支学习风格条件生成模型，并用问题一的模型给生成谱画像；下支在同一个隐空间里做跨风格迁移，再用缺陷约束和问题一的模型复核。

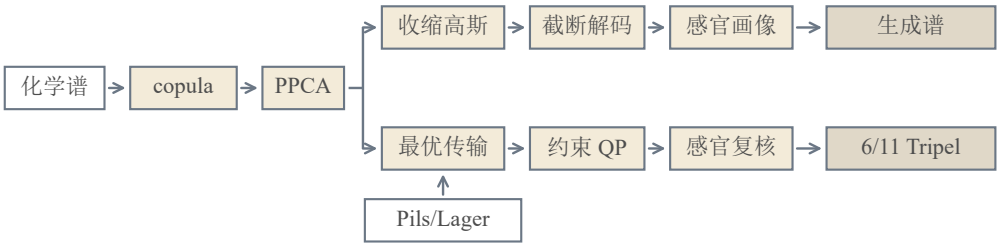


图 13 问题三框架

8.2 机理分析

风格条件生成有三个难点。其一，样本少维度高：22 个风格共 250 款啤酒，Faro 只有 3 款、Brut 只有 4 款，而化学谱有 231 维，直接估计每个风格的协方差矩阵是奇异的。其二，大量化合物是零膨胀的：第五章已经看到 101 列有成片的未检出值，多元高斯会把这团点抹成连续的一片，生成出来的谱在检出下限附近失真。其三，迁移不能只考虑风格相似度：Tripel 是比利时上发酵的高度酒，其酵母常带酚类脱羧活性，4VG 本底偏高，麦芽用量大又带来更多的 DMS 前体，将 Pils/Lager 整体迁移为 Tripel 时，这些缺陷化合物可能随之超过阈值。

针对这三点，模型分三层处理，如图 14 所示。图 14(a) 为隐空间中的风格分布：样本多的风格 c_1 用自身的均值与协方差，样本少的风格 c_2 的原始估计（点线）可靠性较低，按样本量向全体均值 $\mathbf{0}$ 与合并协方差（灰色虚线）收缩。图 14(b) 为约束迁移：源啤酒沿最优传输映射 $\mathcal{T}(\boldsymbol{z})$ 移向 Tripel 分布，若终点落入某个缺陷化合物超标的区域（阴影），就投影回可行域的边界，得到 $\boldsymbol{z}^{*}$。

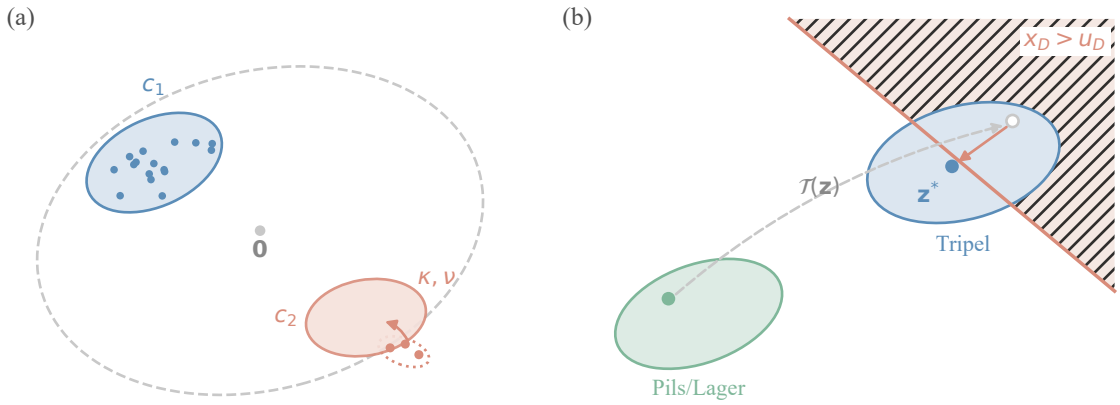


图 14　问题三机理：(a) 收缩生成；(b) 约束迁移

8.3　生成模型的建立

（1）高斯 copula 变换[12]。对每个化合物 j，用训练样本的经验分布函数（并列值取中秩）把 x_{ij} 变为正态得分 $u_{ij}=\Phi^{-1}\big(\hat{F}_j(x_{ij})\big)$。未检出的样本并列在最低值，共享同一个正态得分，生成时再经经验分位数函数 $\hat{F}_j^{-1}(\Phi(\cdot))$ 映回原尺度。正态得分落在未检出比例对应的分位数以下时，映回的值恰为检出下限，因此每个化合物的边缘分布和未检出比例都能得到保留。

（2）概率主成分隐空间。在正态得分上建立概率主成分模型[10]

$$\boldsymbol{u}=\boldsymbol{W}\boldsymbol{z}+\boldsymbol{m}+\boldsymbol{\varepsilon},\qquad \boldsymbol{\varepsilon}\sim N(\mathbf{0},\sigma^{2}\boldsymbol{I}), \tag{14}$$

隐维数 d 由五折交叉验证的留出对数似然确定。

（3）层次收缩的风格分布。风格 c 有 n_c 款啤酒，其隐坐标的样本均值与协方差为 $\bar{\boldsymbol{z}}_c$、$\boldsymbol{S}_c$，全体合并的组内协方差为 $\boldsymbol{S}_w$。按正态逆 Wishart 先验的后验形式，风格分布取

$$\boldsymbol{\mu}_c=\frac{n_c}{n_c+\kappa}\,\bar{\boldsymbol{z}}_c,\qquad \boldsymbol{\Sigma}_c=\frac{n_c}{n_c+\nu}\,\boldsymbol{S}_c+\frac{\nu}{n_c+\nu}\,\boldsymbol{S}_w, \tag{15}$$

κ 控制均值向全体中心收缩，ν 控制协方差向合并协方差收缩，样本越少收缩越强。(κ,ν) 在网格上按留出啤酒的对数似然选取。

综上所述，得到**收缩条件生成模型**（HS-CLG）：

$$\begin{cases}\boldsymbol{z}\mid c\sim N(\boldsymbol{\mu}_c,\boldsymbol{\Sigma}_c),\quad \boldsymbol{\mu}_c=\dfrac{n_c\bar{\boldsymbol{z}}_c}{n_c+\kappa},\quad \boldsymbol{\Sigma}_c=\dfrac{n_c\boldsymbol{S}_c+\nu\boldsymbol{S}_w}{n_c+\nu},\\ \boldsymbol{u}=\boldsymbol{W}\boldsymbol{z}+\boldsymbol{m}+\boldsymbol{\varepsilon},\quad \boldsymbol{\varepsilon}\sim N(\mathbf{0},\sigma^{2}\boldsymbol{I}),\\ x_j=\hat{F}_j^{-1}\big(\Phi(u_j)\big)\ \geq\ \ell_j,\qquad j=1,\dots,231,\\ \hat{\boldsymbol{y}}=\mathcal{M}_1(\boldsymbol{x}),\quad g=\phi(\hat{\boldsymbol{y}}),\end{cases} \tag{16}$$

其中 $\mathcal{M}_1$ 为问题一的感官预测模型，ϕ 为式 (7) 的分级换算，生成谱的感官画像即对大量生成样本求平均后的分级得分。

8.4 约束迁移模型的建立

（1）决策变量。源啤酒（Pils/Lager）的正态得分为 $\boldsymbol{u}_0$，隐坐标为 $\boldsymbol{z}_0$，残差 $\boldsymbol{r}_0 = \boldsymbol{u}_0 - \boldsymbol{W}\boldsymbol{z}_0 - \boldsymbol{m}$ 记录了这款啤酒自身的特点。决策变量是迁移后的隐坐标 $\boldsymbol{z}$，残差保持不变，迁移后的谱为 $\boldsymbol{x}(\boldsymbol{z}) = \hat{F}^{-1}\big(\Phi(\boldsymbol{W}\boldsymbol{z} + \boldsymbol{m} + \boldsymbol{r}_0)\big)$。

（2）目标函数。两个高斯分布之间的最优传输映射有闭式解[11]：

$$\mathcal{T}(\boldsymbol{z}) = \boldsymbol{\mu}_T + \boldsymbol{A}(\boldsymbol{z} - \boldsymbol{\mu}_P), \qquad \boldsymbol{A} = \boldsymbol{\Sigma}_P^{-1/2}\big(\boldsymbol{\Sigma}_P^{1/2}\boldsymbol{\Sigma}_T\boldsymbol{\Sigma}_P^{1/2}\big)^{1/2}\boldsymbol{\Sigma}_P^{-1/2}, \tag{17}$$

下标 P、T 分别表示 Pils/Lager 与 Tripel。迁移强度为 α 时的目标点为 $\boldsymbol{z}_\alpha = \boldsymbol{z}_0 + \alpha\big(\mathcal{T}(\boldsymbol{z}_0) - \boldsymbol{z}_0\big)$。在满足约束的前提下，迁移后的点 $\boldsymbol{z}$ 要尽量接近目标点，距离用 Tripel 分布的马氏距离度量，使结果保持在 Tripel 的典型区域内：

$$\min_{\boldsymbol{z}} \ (\boldsymbol{z} - \boldsymbol{z}_\alpha)^\top \boldsymbol{\Sigma}_T^{-1} (\boldsymbol{z} - \boldsymbol{z}_\alpha). \tag{18}$$

（3）约束条件。赛题注明“缺陷化合物以所列 DMS/Diacetyl/4VG/trans-2-nonenal/Stale hops 等超阈值项为准”，因此本文以超阈值界定缺陷，“不恶化”即迁移后不新增越阈项、已越阈的不再升高；要求浓度一律不升的严格口径另作对照。记缺陷化合物集合 D 为 DMS、双乙酰、4VG、反-2-壬烯醛，阈值 τ_d 取附件 8 的值，分别为 17.25 μg/L、0.09 mg/L、0.2 mg/L；附件 8 没有给出反-2-壬烯醛的阈值，而该化合物的 GC-MS 测量只是半定量的，改用全体样本的 90% 分位数作为阈值。约束分两种情况：源啤酒已经超阈值的缺陷，迁移后浓度不得上升；尚未超阈值的缺陷，迁移后不得超过 $0.8\tau_d$，留出 20% 的安全余量。两种情况合写为

$$x_d(\boldsymbol{z}) \le b_d := \max\big(x_{0d},\ \log_{10} 0.8\tau_d\big), \qquad d \in D. \tag{19}$$

由于 $\hat{F}_d^{-1} \circ \Phi$ 单调，这一约束等价于正态得分上的线性约束 $\boldsymbol{w}_d^\top \boldsymbol{z} + m_d + r_{0d} \le \Phi^{-1}\big(\hat{F}_d(b_d)\big)$。Stale hops（陈酒花）没有对应的单一化合物，与另外四项一起用问题一的模型复核：迁移后预测的缺陷感官评分不得超过 $\max(g_0, 1)$，即不得从察觉不到或刚能察觉变为明显可察觉。复核不通过时把 α 减小 0.05 再求解，α 不低于 0.3。

综上所述，得到**约束最优传输模型**：

$$\begin{aligned} &\min_{\boldsymbol{z}} \ (\boldsymbol{z} - \boldsymbol{z}_\alpha)^\top \boldsymbol{\Sigma}_T^{-1} (\boldsymbol{z} - \boldsymbol{z}_\alpha), \qquad \boldsymbol{z}_\alpha = \boldsymbol{z}_0 + \alpha\big(\boldsymbol{\mu}_T + \boldsymbol{A}(\boldsymbol{z}_0 - \boldsymbol{\mu}_P) - \boldsymbol{z}_0\big) \\ &s.t. \begin{cases} \boldsymbol{w}_d^\top \boldsymbol{z} + m_d + r_{0d} \le \Phi^{-1}\big(\hat{F}_d(b_d)\big), & d \in D, \\ g_k\big(\mathcal{M}_1(\boldsymbol{x}(\boldsymbol{z}))\big) \le \max\big(g_k(\boldsymbol{x}_0), 1\big), & k \in K_D. \end{cases} \end{aligned} \tag{20}$$

其中 D 为四项缺陷化合物，K_D 为 DMS、双乙酰、4VG、反-2-壬烯醛、陈酒花五项缺陷感官属性，$b_d = \max(x_{0d}, \log_{10} 0.8\tau_d)$。决策变量只有 $\boldsymbol{z}$，α 是外层参数。目标为正定二次型，第一组约束为线性，内层是一个凸二次规划，其局部最优解就是全局最优解，用 SLSQP 求数值解，并检查收敛标志与约束违反量；第二组约束含取整运算，不进入内层，而是在外层逐步减小 α 来满足。

8.5 生成模型的求解与检验

参数确定。正态得分上的概率主成分模型在 $d = 5, 10, \ldots, 80$ 中以 $d = 30$ 的五折留出对数似然最大（-240.2），d 再增大时似然下降，说明多出的维度在拟合噪声。在 $\kappa \in \{0.5, 1, 2, 4, 8\}$、$\nu \in \{2, 5, \ldots, 320\}$ 的网格上，留出似然在 $\kappa = 2$、$\nu = 80$ 处最大。$\nu = 80$ 远大于多数风格的样本量，说明各风格的协方差应当主要向合并协方差收缩，而均值只需轻度收缩，即风格之间的差别主要体现在位置上，形状大体相同。

生成器对比。按风格分层做五折划分，在训练折上拟合生成器，每个风格生成 60 个样本，与测试折的真实啤酒比较。评价指标有六类：条件一致率为在真实数据上训练

的风格分类器把生成样本判为目标风格的比例；风格检索率为生成样本与同风格测试啤酒的能量距离在所有风格中最小的比例；C2ST AUC 为随机森林区分真实与生成样本的 AUC，用分层随机五折计算，越接近 0.5 越真实；相关结构误差为生成与真实相关矩阵之差的相对 Frobenius 范数，分别在原始值（Pearson）与秩（Spearman）上计算；边际 KS 距离为 231 个化合物边缘分布 KS 统计量的均值；未检出率误差为各化合物未检出比例之差的均值。结果见表 7。

表 7　生成器对比

生成器	条件一致率	风格检索率	C2ST AUC	相关误差↓		KS 距离↓	未检出率误差↓
				Pearson	Spearman		
无条件 PPCA	0.045±0.007	0.063±0.003	1.000±0.000	0.376±0.016	0.562±0.015	0.238±0.002	0.191±0.002
风格独立高斯	0.994±0.003	0.558±0.060	1.000±0.000	0.767±0.031	0.780±0.016	0.168±0.003	0.173±0.002
同风格插值	1.000±0.000	0.543±0.064	0.990±0.003	0.331±0.019	0.327±0.017	0.136±0.003	0.185±0.003
无 copula 消融	0.553±0.007	0.454±0.093	1.000±0.000	0.393±0.014	0.579±0.024	0.231±0.001	0.195±0.003
本文 HS-CLG	0.523±0.030	0.482±0.101	0.472±0.032	0.588±0.021	0.397±0.015	0.064±0.003	0.029±0.003
真实留出样本（参照）	0.492	–	0.500	–	–	–	–

由表 7 可以看出：无条件 PPCA 的条件一致率只有 0.045，说明不加风格条件就生成不出风格；风格独立高斯与同风格插值的条件一致率高达 0.99 以上，远高于真实测试样本的 0.492，说明它们生成的谱比真实啤酒还要典型，多样性不足，而且 C2ST AUC 接近 1，容易被分类器识别；不加 copula 的 HS-CLG 条件一致率 0.553，与真实样本相当，但同样容易识别（AUC 为 1.000），未检出率误差 0.195、边际 KS 距离 0.231，问题出在零膨胀化合物上。加入 copula 层后，C2ST AUC 降到 0.472，与随机水平 0.5 的偏离只有 0.032，平衡准确率为 0.467，随机森林已基本分不出真实谱与生成谱，而其余生成器的 AUC 都在 0.97 以上；未检出率误差从 0.195 降到 0.029，边际 KS 距离从 0.231 降到 0.064，条件一致率 0.523、风格检索率 0.482 都与真实数据的水平相当。需要说明，若 C2ST 的五折划分不打乱（表中未列的对照），而生成样本按风格顺序排列，测试折中的生成样本全部来自训练折里没有出现的风格，AUC 被系统性地压到 0.370，并不表示生成谱更真实；改用分层随机划分后得到上述结果。copula 保持的是秩相关：按秩计算的 Spearman 相关结构误差从 0.579 降到 0.397，而基于原始值的 Pearson 误差从 0.393 升到 0.588，这是因为大量并列在检出下限的值会扭曲 Pearson 相关，对生成谱的风格判别与感官画像影响不大。

如图 15 所示，图 15(a) 中虚线为真实测试样本的条件一致率 0.492，本文方法的两项指标都落在这条线附近，既不偏低也不虚高。图 15(b) 把全部啤酒投影到前两个主成分上，实心点为真实啤酒，空心圈为生成谱（每个风格画出 300 个中的 50 个），灰点为其他风格：Tripel 位于右侧的高酒精、高酯区域，Kriek 与 Lambic 位于左下方的高酸区域，生成谱覆盖了真实啤酒所在的区域，既没有集中在少数点上，也没有超出该风格的范围。

风格化合物谱与感官画像。对 22 个风格各生成 300 个谱，送入问题一的模型得到感官画像，与该风格真实啤酒的平均品评画像比较。22 个风格的 50 维画像 Spearman 相关平均为 0.82，中位数 0.87，平均档差 0.155。作为参照，把同一风格的真实啤酒送入问题一的模型，所得预测画像与品评画像的相关平均为 0.97，生成画像比它低约 0.15，差距主要集中在 Amber（0.40）、Dubbel（0.60）、Flanders old brown（0.63）、Brown（0.63）等化学上与相邻风格接近的深色风格；Faro 只有 3 款、Brut 只有 4 款，画像相关虽有 0.86、0.78，平均档差却达 0.28、0.32，样本太少，只能作参考。Tripel、Kriek、Lambic 的相关分别为 0.880、0.976、0.927，平均档差为 0.053、0.232、0.102。三个风格生成谱中偏离全体均值最多的化合物如下：

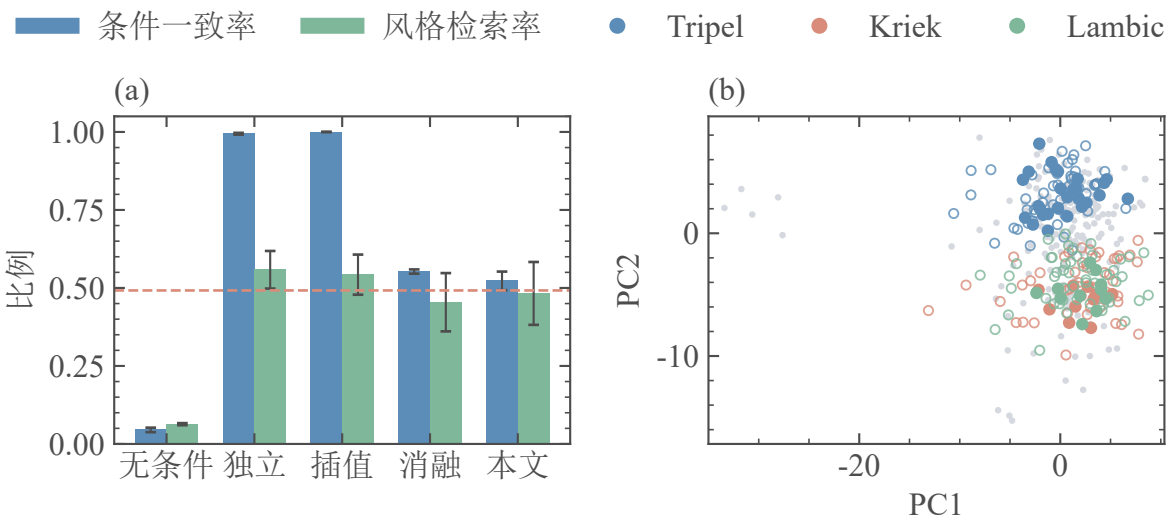


图 15　生成器评价：(a) 一致率；(b) 主成分投影

- Tripel：总酯、乙酸异戊酯、乙酸乙酯、己酸乙酯、乙醇和热量值显著偏高，是典型的高浓度麦汁、高温上发酵的酵母代谢特征[13]；
- Kriek：苯甲酸乙酯、丁香酚、乙基苯酚、乳酸与总酸偏高，苦味偏低。樱桃果核带来苯甲醛类物质，乙基苯酚来自酒香酵母，乳酸来自乳酸菌；
- Lambic：总酸、乳酸、乙酸最高，pH 最低，4-乙基愈创木酚和 4-乙基苯酚显著偏高，是自然发酵中酒香酵母代谢的标志[14]。

生成谱中的这些化合物均值与真实啤酒非常接近，例如 Tripel 生成谱的乙酸异戊酯对数均值为 0.585，真实值为 0.595。Tripel、Kriek、Lambic 各 300 个完整生成谱见支撑材料 q3_generated_*.csv，其余 19 个风格的生成谱可由程序复现。

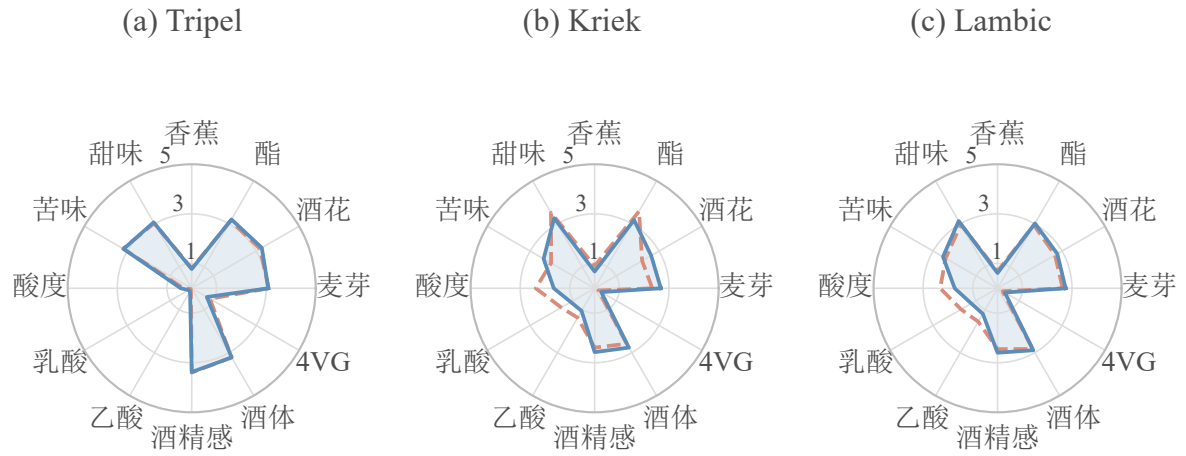


图 16　感官画像：(a) Tripel；(b) Kriek；(c) Lambic

如图 16 所示，实线为生成谱的画像，虚线为真实啤酒的品评画像，坐标为 0 至 6 分。Tripel 的画像以酒精感 3.4、酒体 3.2、酯味 3.2、苦味 3.2 为主，酸类接近 0；Kriek 与 Lambic 的酸度、乳酸、乙酸明显突出，Kriek 的甜味（3.3）高于 Lambic（3.1），与 Kriek 加入樱桃、残糖较多一致。生成画像与品评画像在三个风格上几乎重合，只在 Kriek 与 Lambic 的酸度上偏低约 0.7 档，原因是问题一的模型对极端酸度有向均值收缩的倾向，这一偏差在真实 Kriek 啤酒的预测画像上同样存在。

8.6 迁移模型的求解与检验

求解步骤。对每一款 Pils/Lager：

Step1：计算正态得分、隐坐标 $\boldsymbol{z}_0$ 与残差 $\boldsymbol{r}_0$；

Step2：取 $\alpha = 1$，计算最优传输目标点 $\boldsymbol{z}_\alpha$；

Step3：以四项缺陷化合物为线性约束，求解凸二次规划得到 $\boldsymbol{z}^*$；

Step4：解码得到迁移后的谱，用问题一的模型复核五项缺陷感官评分，不通过则令 $\alpha \leftarrow \alpha - 0.05$ 回到 Step2。

风格判别使用在隐坐标上训练的多项逻辑回归，它在全部样本上的交叉验证准确率为 0.532；真实 Tripel 啤酒被判为 Tripel 的概率中位数为 0.823，到 Tripel 分布中心的马氏距离中位数为 4.69，以这两个值为参照。下文的判别概率与“判为 Tripel”（Tripel 为概率最大的风格）都取这一判别器；生成模型自身的高斯似然给出的 Top-1 风格作为第二口径一并报告。11 款啤酒的迁移结果见表 8。

表 8 迁移结果

啤酒	α	Tripel 判别概率			迁移后排名	Tripel 马氏距离		起作用约束数	恶化项数	
		迁移前	无约束	有约束		迁移前	有约束		无约束	有约束
28	1.00	0.003	0.979	0.929	1	6.77	4.46	1	1	0
81	1.00	0.002	0.989	0.977	1	8.95	5.01	2	2	0
90	1.00	0.000	0.883	0.823	1	8.08	4.39	2	2	0
96	1.00	0.000	0.039	0.017	4	9.57	5.66	2	2	0
119	1.00	0.000	0.529	0.257	2	7.48	4.46	1	1	0
124	1.00	0.000	0.152	0.028	5	9.13	4.83	2	2	0
139	1.00	0.000	0.844	0.614	1	8.99	4.91	2	2	0
185	1.00	0.000	0.272	0.205	2	8.58	3.95	2	2	0
209	1.00	0.000	0.660	0.191	2	8.60	5.19	2	2	0
218	1.00	0.000	0.963	0.910	1	7.28	3.98	2	2	0
230	1.00	0.000	0.786	0.570	1	9.41	5.01	2	2	0

由表 8 可以看出：迁移前 11 款 Pils/Lager 被判为 Tripel 的概率全部接近 0，在 22 个风格中 Tripel 只排在第 5 至 21 位；约束迁移后平均判别概率升到 0.502，6 款被判为 Tripel，这 6 款的判别概率都超过 0.5，另有 3 款排在第 2 位；按高斯似然口径，Top-1 为 Tripel 的有 8 款。到 Tripel 中心的平均马氏距离从 8.44 降到 4.71，与真实 Tripel 啤酒的 4.69 相当，即迁移后的谱已经处在 Tripel 的典型区域。全部 11 款啤酒在 $\alpha = 1$ 时都通过了感官复核，不需要回退；SLSQP 在 11 款上全部正常收敛，线性约束的最大违反量为 0，解码后的截断保护也没有被触发。

缺陷约束的作用如下。表 8、表 9 中的恶化项次按式中的界 b_d 判定：未超阈值的缺陷越过 0.8 倍阈值，或已超阈值的缺陷继续升高，各记一次。不加约束时，11 款啤酒的 DMS 全部恶化，9 款的双乙酰恶化，共 20 项次，其中 13 项是新越过阈值，6 项是已超阈值的继续升高，1 项越过了 0.8 倍阈值的安全余量，这正是机理分析中预期的：Tripel 的麦芽用量大、DMS 本底高。加约束后恶化项次为 0：迁移前已超阈值的 6 项（3 款的 DMS、3 款的双乙酰）保持原浓度不升，其余缺陷都限制在 0.8 倍阈值以内，没有新增越阈项，起作用的约束每款 1 至 2 项。未超阈值的缺陷化合物允许在安全余量内上升，44 个“啤酒–缺陷”组合中有 24 个浓度有所上升，例如双乙酰 OAV 的中位数从 0.46 升到 0.80，但都没有越过阈值。另外，问题一的模型对缺陷属性的预测能力有限（$R^2 < 0.12$），感官复核在 11 款啤酒上都没有触发回退，起主要作用的是化学阈值约束。

为检验迁移方法的选择，把本文方法与均值平移、向 Tripel 中心线性插值（$\alpha = 0.5$）、替换为隐空间中最近的真实 Tripel 啤酒、无约束最优传输，以及要求四项缺陷浓度一律不升的严格版本比较，结果如表 9 所示，其中个体偏移为迁移前后化学谱的平均标准化距离，衡量啤酒自身特点保留了多少。

表 9　迁移方法对比

方法	Tripel 概率	判为 Tripel	似然 Top-1	马氏距离	恶化项次	浓度上升	个体偏移	用时/ms
均值平移	0.604	8/11	8/11	4.89	21	35	0.725	141
线性插值	0.168	2/11	1/11	4.22	16	31	0.610	143
最近邻替换	0.765	10/11	11/11	4.46	17	27	1.094	141
OT 无约束	0.645	8/11	11/11	4.10	20	35	0.728	147
OT+ 阈值约束（本文）	0.502	6/11	8/11	4.71	0	24	0.649	171
OT+ 严格不升	0.463	6/11	6/11	5.12	0	0	0.641	175

由表 9 可以看出：最近邻替换的 Tripel 概率最高（0.765），10 款被判为 Tripel，但个体偏移达 1.094，相当于用另一款 Tripel 替换原啤酒，同时有 17 项次缺陷恶化；均值平移与无约束最优传输各有 8 款被判为 Tripel，但缺陷恶化 20 项次以上；线性插值迁移不充分，只有 2 款被判为 Tripel。只有带约束的两种方法做到了缺陷零恶化，两者都有 6 款被判为 Tripel；本文方法的平均判别概率更高（0.502 对 0.463），到 Tripel 中心的马氏距离更小（4.71 对 5.12），按高斯似然口径 Top-1 也更多（8 款对 6 款），代价是 24 个“啤酒–缺陷”组合的浓度在阈值以内有所上升，严格版本则保证缺陷浓度一律不升。按赛题以超阈值项为准的口径，本文方法满足要求且风格转换更充分；若酿造上要求任何缺陷物质都不增加，可改用严格版本。各方法单款用时都在 0.2 s 以内（PC 同时运行其他任务时测得），计算量不是选择的依据。

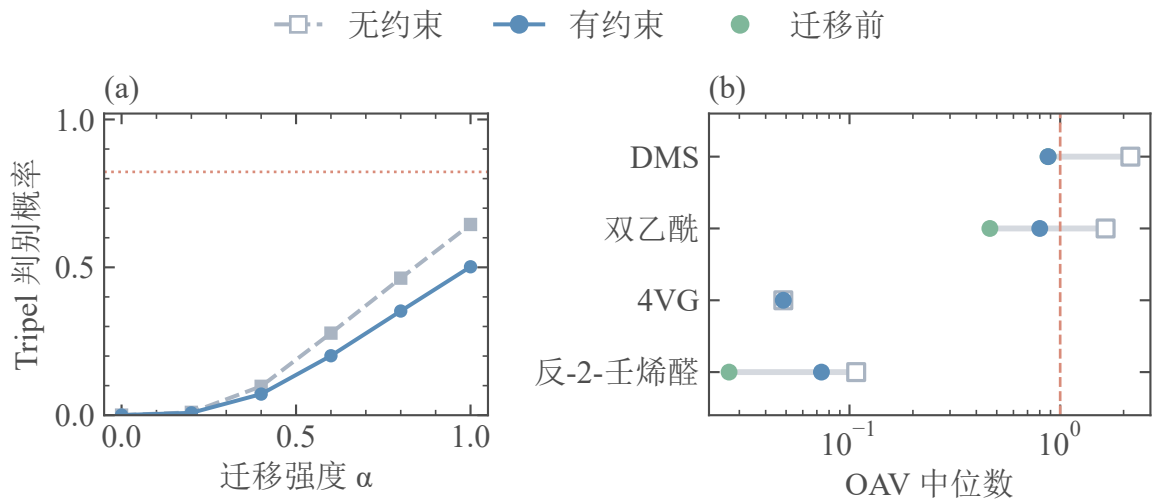


图 17　迁移效果：(a) 迁移强度；(b) 缺陷 OAV

如图 17 所示，图 17(a) 中点线为真实 Tripel 的判别概率中位数 0.823，随 α 增大，有约束与无约束两条曲线都单调上升，两者的差距在 $\alpha > 0.4$ 后逐渐拉开，说明越接近 Tripel，缺陷约束越容易起作用。图 17(b) 中虚线为阈值 OAV= 1，方形为无约束迁移的结果，圆点为有约束迁移的结果：无约束时 DMS 与双乙酰的 OAV 明显右移，有约束时均回到迁移前的水平或阈值的 0.8 倍以内。

迁移后的感官画像也符合 Tripel 的风格。11 款啤酒的平均评分中，酒精感从 2.36 升到 3.09，酯味从 2.09 升到 3.00，香蕉酯香从 0.09 升到 0.91，酒体从 2.64 升到 3.00，接近

模型对真实 Tripel 啤酒的预测均值 3.29、3.11、0.8 和 3.07；DMS 评分由 1.09 降到 1.00，双乙酰、陈酒花两项缺陷评分保持不变，反-2-壬烯醛由 0.91 降到 0.82，4VG 由 0.27 升到 0.91，仍在 1 分（刚能察觉）以内。完整结果见支撑材料 q3_transfer.csv。

九、 模型的灵敏度分析

三个模型各有一两个人为设定或网格选取的关键参数：问题一的公共档宽 Δ，问题二的平滑系数 η 与冗余惩罚 ω，问题三的收缩强度 κ 与 ν。逐个改变这些参数，观察结果的变化，如图 18 所示。

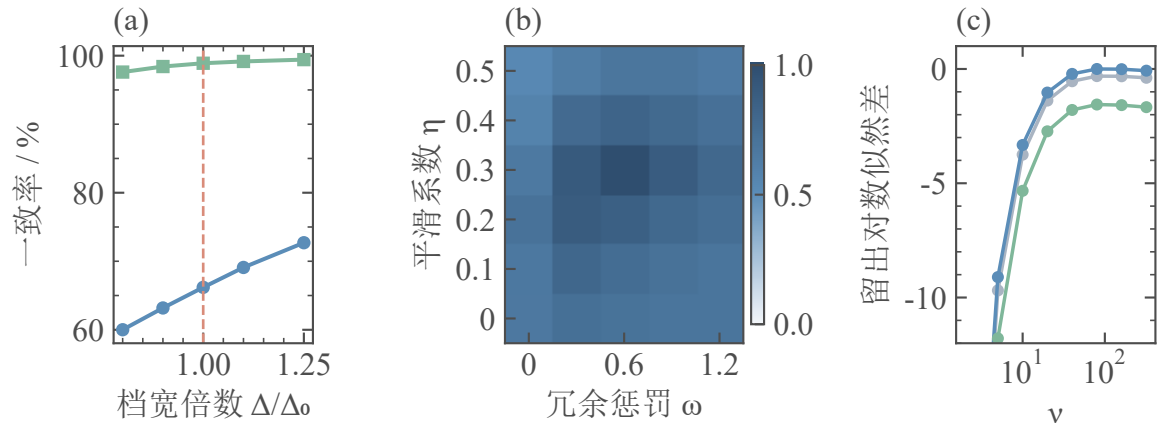


图 18 灵敏度：(a) 档宽；(b) η 与 ω；(c) κ 与 ν

（1）问题一的公共档宽。图 18(a) 中圆点为完全一致率，方点为 ±1 档一致率，虚线为基准值 $\Delta_0 = 1.073$。Δ 在 $0.8\Delta_0$ 至 $1.25\Delta_0$ 之间变化时，完全一致率从 60.0% 单调升到 72.7%，变化 12.7 个百分点，±1 档一致率始终在 97.6% 以上，平均档差从 0.429 降到 0.279。档宽变大时每档更宽，评分整体降低，一致率随之机械上升，所以一致率的升高并不说明大档宽更好，离散分级本身对档宽是敏感的。档宽改变的主要是评分的整体松紧：五个档位下模型评分的平均分为 1.32、1.26、1.20、1.15、1.08，品评换算评分为 1.27、1.21、1.16、1.12、1.06，两者始终相差不到 0.05 档；二次加权 Kappa 在 0.302 至 0.317 之间，说明各款啤酒之间的高低次序基本不随档宽改变。因此，回归分数与品评得分的整体尺度可以对齐，排序对档宽稳健，但具体落在哪一档取决于档宽，Δ 是分级换算的关键参数。各训练折内重新估计的档宽在 0.85 至 1.07 之间（均值 1.03），因为档宽由丁香属性最强的一款啤酒决定，它落入测试折时档宽变小，这也说明公共档宽的估计依赖少数极端样本。

（2）问题二的平滑系数与冗余惩罚。在 $\eta \in \{0, 0.1, \ldots, 0.5\}$、$\omega \in \{0, 0.3, 0.6, 0.9, 1.2\}$ 的 30 种组合下重新筛选 Top-3，计算它与基准 $(\eta, \omega) = (0.3, 0.6)$ 的平均 Jaccard 系数，结果如图 18(b) 所示，颜色越深越接近基准。在 $\eta \in [0.2, 0.4]$、$\omega \in [0.3, 0.9]$ 的范围内，Jaccard 系数不低于 0.77，结果基本不变；$\omega = 0$ 即不去冗余时，Jaccard 系数降到 0.53 至 0.71，Top-3 内的平均拓扑重叠从 0.014 升到 0.050，冗余明显增加；$\eta \geq 0.5$ 时平滑过强，与基准的 Jaccard 系数降到 0.68 以下，个别化合物的独立信号被邻居淹没。因此冗余惩罚是必要的，而在合理区间内结果对两个参数都不敏感。

（3）问题三的收缩强度。图 18(c) 中三条曲线分别为 $\kappa = 0.5$（灰）、$\kappa = 2$（蓝）、$\kappa = 8$（绿），纵轴为留出对数似然与最优值之差。ν 从 2 增大到 40 时似然提高约 30，说明协方差必须向合并协方差强收缩，否则小样本风格的协方差估计会严重过拟合；$\nu \geq 40$ 之后三条曲线都进入平台，ν 在 40 至 320 之间变化时似然的差别不到 0.4。κ 的影响小得多，$\kappa = 0.5$ 与 $\kappa = 2$ 只差 0.3，$\kappa = 8$ 时均值收缩过度，似然下降约 1.6。因此 ν 是关键参数，

必须足够大，而一旦进入平台区，留出似然对 ν 和 κ 都不敏感，所选的 $\kappa = 2$、$\nu = 80$ 处在平台的中央。这里检验的是似然，C2ST、画像相关等生成质量指标随这两个参数的变化没有逐一重算，是后续可以补充的工作。

十、 模型的评价、改进与推广

10.1 模型的优点

（1） 数据处理以校准浓度为参照逐列核实，查清了附件 1 中对数列与原始浓度列混排的结构；缺失值的机制由附件 8 的真值直接检验，删失插补将未检出单元赋值为检出下限，避免了常规插补约 3 个标准差的系统偏差，三问都建立在同一份可靠的数据上。

（2） 分级换算由品评员标准化的机理推出，0 分锚点和公共档宽都来自数据本身，不依赖人为设定的分位数；锚点、档宽与方差校准都在训练折内估计，Beer#1 至 Beer#12 的评分是真正只用化学谱得到的折外结果。

（3） 三问之间可以互相印证：问题一的预测模型用于问题三生成谱的画像和迁移结果的复核；问题二的模块结构解释了问题三中缺陷约束为什么会牵动一批化合物，也解释了问题一中哪些属性预测得准。

（4） 每一问都与多种基线方法做了对比：问题一比较了 9 种回归方法，问题二比较了 5 种筛选方法，问题三比较了 5 种生成器，并且都用重复交叉验证或 bootstrap 给出了波动范围。

10.2 模型的缺点

（1） 感官得分本身噪声较大，化学谱只能解释约四分之一的方差，中位 R^2 只有 0.20 左右，香料、花香、热带水果等由个别痕量化合物决定的属性预测精度偏低，对应的评分主要落在 0 和 1 两档；AMTS 相对等权集成的提升在逐折配对检验中并不显著。

（2） 公共档宽假定每位品评员在所有属性上使用同一尺度，若品评员对不同属性的打分习惯差别较大，换算出的绝对分值会有一档左右的系统偏差；分级的二次加权 Kappa 为 0.303，只属于一般一致，离散档位对档宽也较敏感。

（3） 问题二的图去冗余筛选主要降低了 Top-3 内的冗余，模块层面的稳定性与合理命中率和 TreeSHAP、Borda 融合的差异都在 bootstrap 置信区间之内，合理命中率的点估计还略低于 TreeSHAP。

（4） 问题三的生成模型是线性的高斯 copula 隐变量模型，Amber、Dubbel 等深色风格的画像相关只有 0.40 至 0.63，Faro、Brut 等只有 3 至 4 款的风格平均档差偏大（0.28、0.32）；迁移结果的风格判别依赖在同一批数据上训练的判别器，判别概率只能作为相对参照。

10.3 模型的改进

若能取得品评员的原始打分，可以把分级换算改为有序回归，直接估计每位品评员的阈值，消除公共档宽假设带来的偏差。生成模型目前是线性的隐变量模型，在样本量允许时可以换成条件变分自编码器或条件扩散模型，以刻画化合物之间的非线性共变；迁移约束也可以从单个化合物的阈值扩展为考虑化合物协同效应的感官阈值模型。

10.4 模型的推广

（1） 删失插补适用于所有存在检出下限的仪器分析数据，例如葡萄酒、咖啡、茶叶的挥发性成分分析，以及环境监测中的痕量污染物数据。

（2） 风格条件生成与约束迁移可以用于新品配方设计：给定目标风格，在缺陷物质不超过阈值的前提下，给出化学组成的调整方向，供酿造工艺试验参考。

参考文献

[1] Schreurs M, Piampongsant S, Roncoroni M, et al.，Predicting and improving complex beer flavor through machine learning，Nature Communications，15(1)：2368，2024。

[2] Meilgaard M C，Flavor chemistry of beer: Part II: Flavor and threshold of 239 aroma volatiles，MBAA Technical Quarterly，12(3)：151–168，1975。

[3] Little R J A, Rubin D B，Statistical Analysis with Missing Data（第 3 版），Hoboken：Wiley，2019：13–20。

[4] Beidler L M，A theory of taste stimulation，Journal of General Physiology，38(2)：133–139，1954。

[5] Spyromitros-Xioufis E, Tsoumakas G, Groves W, et al.，Multi-target regression via input space expansion: treating targets as inputs，Machine Learning，104(1)：55–98，2016。

[6] Langfelder P, Horvath S，WGCNA: an R package for weighted correlation network analysis，BMC Bioinformatics，9(1)：559，2008。

[7] Lundberg S M, Erion G, Chen H, et al.，From local explanations to global understanding with explainable AI for trees，Nature Machine Intelligence，2(1)：56–67，2020。

[8] Meinshausen N, Bühlmann P，Stability selection，Journal of the Royal Statistical Society Series B，72(4)：417–473，2010。

[9] Blondel V D, Guillaume J L, Lambiotte R, et al.，Fast unfolding of communities in large networks，Journal of Statistical Mechanics，2008(10)：P10008，2008。

[10] Tipping M E, Bishop C M，Probabilistic principal component analysis，Journal of the Royal Statistical Society Series B，61(3)：611–622，1999。

[11] Peyré G, Cuturi M，Computational optimal transport，Foundations and Trends in Machine Learning，11(5–6)：355–607，2019。

[12] Nelsen R B，An Introduction to Copulas（第 2 版），New York：Springer，2006：51–56。

[13] Verstrepen K J, Derdelinckx G, Dufour J P, et al.，Flavor-active esters: adding fruitiness to beer，Journal of Bioscience and Bioengineering，96(2)：110–118，2003。

[14] Vanbeneden N, Gils F, Delvaux F, et al.，Formation of 4-vinyl and 4-ethyl derivatives from hydroxycinnamic acids: occurrence of volatile phenolic flavour compounds in beer and distribution of Pad1-activity among brewing yeasts，Food Chemistry，107(1)：221–230，2008。

[15] deepseek，2026-10-23 至 2026-10-27。

AI 工具使用声明

本参赛队在 2026 年 10 月 23 日至 27 日期间使用 deepseek，辅助处理数据获取、资料查询、代码整理及格式调整等工作。其中，数据获取涉及文件获取渠道和归档步骤的整理；资料查询围绕啤酒风味、化合物及方法背景展开；代码整理侧重已有脚本的组织、路径和运行说明；格式调整用于规范论文与支撑材料的呈现方式。各项使用内容集中列于支撑材料《AI 工具使用详情说明》。

附录

附录 A　支撑材料文件清单

```
A260XXXX附件/
|-- 代码/
|   |-- common.py            公共路径与数据读取
|   |-- prep.py              第五章 尺度核验、缩尾、删失插补
|   |-- q1.py                问题一 集成与自适应堆叠、分级评分
|   |-- q1model.py           问题一 集成预测器与分级规则（问题三调用）
|   |-- q1_missing.py        问题一 缺失稳健性实验
|   |-- q2.py                问题二 共表达网络、Top-3、bootstrap
|   |-- q3.py                问题三 生成、画像、约束迁移
|   |-- q3_cmp.py            问题三 迁移方法对比
|   |-- q2_ci.py             问题二 成对 bootstrap 置信区间
|   |-- sens.py              灵敏度分析
|   |-- fill_scorecard.py    把评分填入官方品评表
|   `-- 运行说明.txt
|-- 结果/
|   |-- 问题一_品评表评分.xlsx    Beer#1-12 的品评表与汇总页
|   |-- q1_paired.csv          问题一 逐折配对比较
|   |-- q1_grade_metrics.csv   问题一 分级一致性（折内与事后对照）
|   |-- q2_top3.csv            50 个感官维度的 Top-3 及入选频率
|   |-- q2_paired_ci.csv       问题二 与各基线的成对差值及置信区间
|   |-- q3_generated_*.csv     Tripel、Kriek、Lambic 各 300 个生成谱
|   |-- q3_gen_eval.csv        生成器对比的各项指标
|   |-- q3_portraits.csv       22 个风格的感官画像
|   |-- q3_transfer.csv        Pils/Lager 迁移为 Tripel 的结果
|   `-- q3_transfer_cmp.csv    六种迁移方法对比
`-- AI 工具使用详情说明.pdf
```

软件环境：Python 3.13，scikit-learn 1.9.0，LightGBM 4.7.0，NumPy 2.1.3，pandas 2.2.3，SciPy，NetworkX 3.4.2，Matplotlib 3.10.0；论文由 XeLaTeX（MiKTeX）排版。运行命令依次为 `python prep.py`、`python q1.py`、`python q2.py`、`python q3.py`、`python q3_cmp.py`、`python q2_ci.py`、`python q1_missing.py`、`python sens.py`。

附录 B　Beer#1 至 Beer#12 的分级评分

表 10　Beer#1–12 评分（描述词取所属类别强度）

维度	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10	#11	#12
风格	Brut	Low/No	Blond	Str.ale	Blond	Amber	Saison	Hoppy	Hoppy	Low/No	Amber	Lambic
麦芽香 • 总体	3	3	3	4	3	3	3	3	3	3	3	3
麦芽香 • 谷物	3	3	3	4	3	3	3	3	3	3	3	3
麦芽香 • 面包	3	3	3	4	3	3	3	3	3	0	3	3
麦芽香 • 焦糖	3	0	0	4	0	3	0	3	3	0	3	3
麦芽香 • 烟熏	0	0	0	4	0	0	0	0	3	0	0	0

维度	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10	#11	#12
酒花香·总体	3	2	3	3	4	3	3	3	3	3	3	3
酒花香·柑橘	3	0	0	3	4	0	3	3	3	0	0	0
酒花香·热带果	0	0	0	0	4	0	3	3	3	0	3	0
酒花香·草本	3	0	3	3	4	3	3	3	3	0	3	3
酒花香·木质	3	0	0	0	4	3	3	0	3	0	0	0
酯香·总体	3	2	3	3	3	3	3	3	3	2	3	3
酯香·指甲油	3	0	3	3	3	3	3	3	3	0	3	3
酯香·香蕉	3	2	3	3	3	3	3	3	3	2	3	3
酯香·花香	0	0	3	0	0	3	3	3	3	0	0	0
酯香·果香	3	2	3	3	3	3	3	3	3	2	3	3
麦芽味·总体	3	3	3	4	3	3	3	3	3	3	3	3
麦芽味·谷物	3	3	3	4	3	3	3	3	3	3	3	3
麦芽味·面包	3	3	3	4	3	3	3	3	3	0	3	3
麦芽味·焦糖	0	0	0	4	0	3	0	0	3	0	3	0
麦芽味·烟熏	0	0	0	4	3	3	0	3	3	0	3	0
酒花味·总体	3	2	3	3	4	3	3	4	4	3	3	2
酒花味·柑橘	3	0	3	3	4	0	3	4	4	0	3	0
酒花味·热带果	0	0	0	0	0	0	3	0	4	0	0	0
酒花味·草本	3	0	3	3	4	3	3	4	4	0	3	0
酒花味·木质	3	0	0	3	4	3	3	4	4	0	3	0
酯味·总体	3	2	3	3	3	3	3	3	3	3	3	3
酯味·指甲油	3	0	3	3	3	3	3	3	3	0	3	3
酯味·香蕉	3	2	3	3	3	3	3	3	3	3	3	3
酯味·花香	0	0	0	0	0	0	0	0	3	0	0	0
酯味·果香	0	2	3	3	0	3	3	0	3	3	3	3
酒体	3	2	3	4	3	3	3	3	3	2	3	3
酒精感	3	1	3	4	3	3	3	4	3	1	3	2
苦味	3	2	3	3	4	3	3	4	4	2	3	2
甜味	3	4	3	3	3	3	3	3	3	4	3	4
杀口感	3	2	3	3	3	3	3	3	3	2	3	3
酸度	1	0	0	0	1	0	1	0	1	1	0	1
余味	3	1	2	3	3	3	3	3	3	2	3	3
DMS	1	1	1	1	1	1	1	1	1	1	1	1
反-2-壬烯醛	1	1	1	0	0	0	0	0	1	0	0	1
金属味	1	0	1	1	1	1	1	1	1	1	1	1
双乙酰	1	1	1	1	0	0	0	1	0	0	0	1
4VG	1	0	1	1	1	1	1	1	1	1	1	0
陈酒花	0	0	0	0	0	0	0	0	0	0	0	0
橙皮	0	1	0	0	0	0	1	0	0	1	0	0
乳酸	0	0	0	0	0	0	0	0	0	0	0	1
芫荽	0	0	0	0	0	0	1	0	0	0	0	0
乙酸	0	0	0	0	0	0	0	0	0	0	0	1

维度	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10	#11	#12
丁香/茴香	0	0	0	0	0	0	0	0	0	0	0	0
畜棚味	0	0	0	0	0	1	0	0	0	0	0	0
总体评价	3	2	3	3	3	3	3	3	3	2	3	3
乙醛	1	0	1	1	1	1	1	0	1	0	1	0

附录 C　50 个感官维度的 Top-3 关键化合物

表 11　Top-3 全表（按融合得分排序，括号内为入选频率）

感官维度	Top-1	Top-2	Top-3	平均频率
麦芽香・总体	Color (1.00)	Beta-phellandrene (0.38)	Acetaldehyde (0.27)	0.55
麦芽香・谷物	Terpinen-4-ol (0.27)	5-methylfurfural (0.25)	Sugars (0.22)	0.24
麦芽香・面包	Color (0.88)	Glycerol (0.12)	4-ethyl-guaiacol (0.12)	0.37
麦芽香・焦糖	Color (1.00)	2-ethyl-1-hexanol (0.35)	Sugars (0.47)	0.61
麦芽香・烟熏	Color (1.00)	pH (0.85)	Methylpyrazine (0.23)	0.69
酒花香・总体	Ethyl octanoate (0.58)	Cis-dihydro-β-terpineol (0.18)	Dimethyl sulfide (0.38)	0.38
酒花香・柑橘	Citronellol (0.67)	Cis-dihydro-β-terpineol (0.18)	Carbon dioxide (0.37)	0.41
酒花香・热带果	Dimethyl sulfide (0.82)	Methyl geranate (0.15)	Carbon dioxide (0.32)	0.43
酒花香・草本	Ethyl octanoate (0.65)	Iron (0.57)	E-2-nonenal (0.27)	0.49
酒花香・木质	E-ortho-methoxy cinnamaldehyde (0.45)	Ethyl 2-methylbutyrate (0.37)	Z,Z-farnesol (0.15)	0.32
酯香・总体	Acetaldehyde (0.83)	N-hexanol (0.77)	Phenethyl alcohol (0.32)	0.64
酯香・指甲油	Acetaldehyde (0.42)	Phenethyl alcohol (0.37)	2-methyl-1-butanol (0.12)	0.30
酯香・香蕉	N-hexanol (0.77)	Tau-cadinol (0.07)	2-methyltetrahydrofuran-3-one (0.25)	0.36
酯香・花香	Beta-phellandrene (0.27)	E-dihydro-apofarnesal (0.32)	N-hexanol (0.17)	0.25
酯香・果香	Beta-phellandrene (0.77)	Eugenol (0.33)	Epicubenol (0.23)	0.44
麦芽味・总体	Color (1.00)	Acetaldehyde (0.90)	pH (0.65)	0.85
麦芽味・谷物	Terpinen-4-ol (0.22)	E-β-ionone (0.32)	Acetaldehyde (0.28)	0.27
麦芽味・面包	Acetaldehyde (0.48)	Color (0.43)	Carbon dioxide (0.20)	0.37
麦芽味・焦糖	Color (1.00)	Acetaldehyde (0.77)	Sugars (0.23)	0.67
麦芽味・烟熏	Color (1.00)	pH (0.87)	Methanethiol (0.47)	0.78
酒花味・总体	Cis-dihydro-β-terpineol (0.57)	Dimethyl sulfide (0.42)	Iron (0.33)	0.44

感官维度	Top-1	Top-2	Top-3	平均频率
酒花味·柑橘	Protein (0.20)	Geosmin (0.23)	Furfural (0.30)	0.24
酒花味·热带果	Dimethyl sulfide (0.57)	Methyl heptenone (0.45)	Hexyl acetate (0.43)	0.48
酒花味·草本	Ethyl octanoate (0.75)	Iron (0.78)	E-2-nonenal (0.23)	0.59
酒花味·木质	Iso-α-acids (0.43)	Isobutanol (0.73)	Cis-dihydro-β-terpineol (0.43)	0.53
酯味·总体	Acetaldehyde (0.82)	Phenethyl alcohol (0.52)	N-hexanol (0.58)	0.64
酯味·指甲油	Ethyl acetate (0.67)	Z,Z-geranyl linalool (0.28)	Glycerol (0.42)	0.46
酯味·香蕉	Isoamyl acetate (0.32)	Phenethyl alcohol (0.40)	Sugars (0.17)	0.29
酯味·花香	Beta-phellandrene (0.42)	Eucalyptol (0.22)	E-dihydro-apofarnesal (0.20)	0.28
酯味·果香	Acetic acid (0.47)	Beta-phellandrene (0.45)	Sugars (0.38)	0.43
酸度	pH (1.00)	Lactic acid (0.77)	4-ethyl-guaiacol (0.42)	0.73
苦味	Ethyl octanoate (0.92)	Iso-α-acids (1.00)	Butyl acetate (0.28)	0.73
甜味	Sugars (0.98)	Ethyl decanoate (0.50)	Iron (0.65)	0.71
4VG	4-vinylguaiacol (1.00)	Neryl formate (0.18)	Isoamyl lactate (0.13)	0.44
双乙酰	Eugenol (0.88)	Furfural (0.17)	Sugars (0.27)	0.44
DMS	Protein (0.72)	Amyl acetate (0.43)	Unfermentables (0.20)	0.45
金属味	Ethyl 3-methylbutyrate (0.40)	Iron (0.50)	Isopentyl butanoate (0.25)	0.38
陈酒花	E-ortho-methoxy cinnamaldehyde (0.37)	Gamma-eudesmol (0.23)	Linalool isovalerate (0.23)	0.28
反-2-壬烯醛	α-phellandrene (0.20)	Geraniol (0.15)	Glycerol (0.27)	0.21
橙皮	Limonene (0.83)	Camphor (0.57)	Beta-phellandrene (0.10)	0.50
芫荽	Limonene (0.52)	Linalool (0.28)	Ethyl linalool (0.20)	0.33
丁香/茴香	Eucalyptol (0.80)	4-vinylguaiacol (0.27)	Linalool (0.32)	0.46
乳酸	Lactic acid (0.90)	pH (0.97)	Isoamyl acetate (0.40)	0.76
乙酸	pH (0.93)	5-methylfurfural (0.48)	Lactic acid (0.45)	0.62
畜棚味	Gamma-eudesmol (0.32)	E-ortho-methoxy cinnamaldehyde (0.27)	Linalool isovalerate (0.30)	0.29
酒精感	Ethanol (0.83)	Carbon dioxide (0.68)	Ethyl octanoate (0.23)	0.58
余味	Ethyl octanoate (0.43)	Ethyl 3-methylbutyrate (0.73)	Acetaldehyde (0.23)	0.47
酒体	Ethanol (0.57)	Methanethiol (0.45)	Color (0.40)	0.47
杀口感	Ethyl octanoate (0.77)	Carbon dioxide (0.80)	Ethyl butyrate (0.28)	0.62
总体评价	Ethyl acetate (0.52)	Unfermentables (0.57)	2-butyl octanoate (0.15)	0.41

附录 D　源程序

D.1 数据预处理 prep.py

```
"""P0␣data␣preprocessing.

1.␣Identify␣the␣measurement␣scale␣of␣every␣column␣of␣1.csv␣(log10␣vs␣raw)␣by
␣␣␣␣comparing␣with␣the␣calibrated␣concentrations␣in␣8.xlsx.
2.␣Unify␣everything␣to␣log10␣scale␣with␣a␣per-column␣detection␣floor.
3.␣Robust␣outlier␣detection␣(MAD␣on␣detected␣values)␣and␣winsorisation.
4.␣Missing␣values:␣compare␣imputers␣(true␣values␣for␣60␣of␣the␣73␣missing␣cells
␣␣␣␣exist␣in␣8.xlsx␣->␣real␣ground␣truth)␣+␣random-mask␣experiment;␣keep␣the␣best.
"""
from common import *
from sklearn.experimental import enable_iterative_imputer # noqa
from sklearn.impute import IterativeImputer, KNNImputer
from sklearn.linear_model import BayesianRidge, RidgeCV

rng = np.random.default_rng(SEED)
d1 = load_chem()
cal = load_calibrated().loc[d1.index]
style = d1['tasting_category_fine']
X = d1.drop(columns=['beer_id', 'tasting_category_fine']).astype(float)
cols = list(X.columns)

# ------------------------------------------------------------------ 1. scale
scale = {}
for c in cols:
    if c in cal:
        x, y = X[c], cal[c].astype(float)
        m = x.notna() & (y > 0)
        r_raw = abs(np.corrcoef(x[m], y[m])[0, 1])
        r_log = abs(np.corrcoef(x[m], np.log10(y[m]))[0, 1])
        slope = np.polyfit(np.log10(y[m]), x[m], 1)[0]
        scale[c] = 'log10' if (r_log > 0.999 and abs(slope - 1) < 0.01) else 'raw'
    else:
        scale[c] = 'log10'
raw_cols = [c for c in cols if scale[c] == 'raw']
print('raw-scale␣columns:', len(raw_cols))

# ------------------------------------------------------------------ 2. unify
Xl = X.copy()
floor_raw = {}
for c in raw_cols:
    x = X[c]
    med = x.median()
    real = x[x > 1e-5 * med]
    cfl = 0.5 * real.min()
    floor_raw[c] = cfl
    Xl[c] = np.log10(np.maximum(x, cfl)) # NaN stays NaN
# detection floor of every column = its minimum on log scale
floor = Xl.min()
n_floor = (Xl.eq(floor) | (Xl - floor).abs().lt(1e-9)).sum()

# ------------------------------------------------------------------ 3. outliers
K = 5.0
out_cells = []
```

```
Xw = Xl.copy()
for c in cols:
    x = Xl[c]
    det = x[(x - floor[c]) > 1e-9].dropna()
    if len(det) < 20:
        continue
    med = det.median(); mad = 1.4826 * (det - med).abs().median()
    if mad < 1e-9:
        continue
    lo, hi = med - K * mad, med + K * mad
    bad = det[(det < lo) | (det > hi)]
    for b, v in bad.items():
        out_cells.append(dict(col=c, beer=int(b), value_log10=round(v, 3), lo=round(lo,
            3), hi=round(hi, 3),
                              scale=scale[c]))
    Xw.loc[bad.index, c] = det.loc[bad.index].clip(lo, hi)
out_df = pd.DataFrame(out_cells)
out_df.to_csv(o('prep_outliers.csv'), index=False)
print('winsorised cells:', len(out_df), 'in', out_df.col.nunique() if len(out_df) else
    0, 'columns')

# ------------------------------------------------------------------ 4. missing
na_cols = [c for c in cols if Xw[c].isna().any()]
print('NA columns:', {c: int(Xw[c].isna().sum()) for c in na_cols})
# ground truth from calibrated sheet, in the same transformed scale
truth = []
for c in na_cols:
    if c not in cal:
        continue
    for b in Xw.index[Xw[c].isna()]:
        y = float(cal.loc[b, c])
        if np.isnan(y):
            continue
        t = np.log10(max(y, floor_raw.get(c, 0))) if scale[c] == 'raw' else (np.log10(y
            ) if y > 0 else floor[c])
        t = float(np.clip(t, Xw[c].min(), Xw[c].max()))
        truth.append((b, c, t))
print('missing cells with ground truth:', len(truth))

R2 = pd.read_csv(p('2.csv'), index_col=0) # chemical-name labelled correlation matrix
R2.index = cols; R2.columns = cols # verified below: same order as 1.csv
style_d = pd.get_dummies(style).astype(float)


def imp_mean(Z):
    return Z.fillna(Z.mean())


def imp_median(Z):
    return Z.fillna(Z.median())


def imp_knn(Z, k=7):
    mu, sd = Z.mean(), Z.std().replace(0, 1)
    A = KNNImputer(n_neighbors=k, weights='distance').fit_transform((Z - mu) / sd)
    return pd.DataFrame(A, index=Z.index, columns=Z.columns) * sd + mu
```

```
def imp_iter(Z):
    mu, sd = Z.mean(), Z.std().replace(0, 1)
    A = IterativeImputer(estimator=BayesianRidge(), max_iter=10, n_nearest_features=40,
         random_state=SEED,
                         initial_strategy='median').fit_transform((Z - mu) / sd)
    return pd.DataFrame(A, index=Z.index, columns=Z.columns) * sd + mu


def imp_graph(Z, m=15, use_style=True):
    """Co-expression-graph-guided␣regression␣imputation␣(ours):
␣␣␣␣for␣each␣incomplete␣compound,␣regress␣on␣its␣m␣strongest␣neighbours␣in␣the
␣␣␣␣correlation␣graph␣(2.csv)␣plus␣style␣indicators;␣neighbours␣are␣restricted␣to
␣␣␣␣columns␣observed␣for␣the␣target␣rows;␣iterate␣twice."""
    Z0 = Z.fillna(Z.median())
    out = Z.copy()
    for it in range(2):
        for c in Z.columns[Z.isna().any()]:
            miss = Z[c].isna()
            nb = R2[c].drop(c).abs().sort_values(ascending=False).index[:m]
            F = Z0[nb]
            if use_style:
                F = pd.concat([F, style_d], axis=1)
            mu, sd = F.mean(), F.std().replace(0, 1)
            Fs = (F - mu) / sd
            mdl = RidgeCV(alphas=np.logspace(-2, 3, 20)).fit(Fs[~miss], Z.loc[~miss, c])
            out.loc[miss, c] = np.clip(mdl.predict(Fs[miss]), Z[c].min(), Z[c].max())
        Z0 = out.copy()
    return out


def imp_censor(Z):
    """Censoring-aware␣two-mechanism␣imputation␣(ours).
␣␣␣␣Cross-check␣with␣8.xlsx␣shows␣every␣missing␣GC-MS␣cell␣is␣a␣non-detect␣(true␣0),
␣␣␣␣i.e.␣left-censored␣MNAR␣->␣impute␣at␣the␣column␣detection␣floor.␣␣Columns␣without
␣␣␣␣a␣calibrated␣counterpart␣(aroma_hops_sum,␣missing␣mostly␣in␣hop-rich␣beers,␣i.e.
␣␣␣␣right-side␣MNAR)␣->␣graph-guided␣regression,␣which␣extrapolates␣with␣its␣hop
␣␣␣␣neighbours␣instead␣of␣shrinking␣to␣the␣mean."""
    F = imp_graph(Z)
    for c in Z.columns[Z.isna().any()]:
        if c in cal and c in left_censored:
            F.loc[Z[c].isna(), c] = Z[c].min()
    return F


# mechanism diagnosis for the real missing cells (uses the calibrated sheet)
left_censored = set()
for c in na_cols:
    if c in cal:
        tv = cal.loc[Xw.index[Xw[c].isna()], c].astype(float)
        if (tv <= 0).mean() >= 0.8:
            left_censored.add(c)
print('left-censored␣(non-detect)␣columns:', sorted(left_censored))
miss_hop = Xw['aroma_hops_sum'].isna()
diag = {c: (float(Xw.loc[miss_hop, c].mean()), float(Xw.loc[~miss_hop, c].mean()))
```

```
      for c in ['linalool', 'geraniol', 'citronellol', 'bitterness', 'alpha.humulene'
          ]}
print('aroma_hops_sum missing vs observed rows (mean log10):', diag)
import time
methods = {'Censor-aware(ours)': imp_censor, 'mean': imp_mean, 'median': imp_median, '
    KNN(k=7)': imp_knn, 'MICE-BayesRidge': imp_iter,
         'Graph-Ridge(ours)': imp_graph, 'Graph-Ridge w/o style': lambda Z: imp_graph(
            Z, use_style=False)}
sd_all = Xw.std()
res = {}
for name, f in methods.items():
    F = f(Xw)
    err = [(F.loc[b, c] - t) / sd_all[c] for b, c, t in truth]
    res[name] = {'truth_RMSE(sd)': float(np.sqrt(np.mean(np.square(err)))), 'truth_MAE(
        sd)': float(np.mean(np.abs(err)))}
    print(name, res[name])

# random-mask experiment: mask 10% of observed cells in 30 random columns, 10 reps
mask_err = {k: [] for k in methods}
cand = [c for c in cols if Xw[c].notna().all() and Xw[c].std() > 0 and (Xw[c] > floor[
    c] + 1e-9).mean() > 0.5]
for rep in range(10):
    Zm = Xw.copy()
    sel = rng.choice(cand, 30, replace=False)
    masked = []
    for c in sel:
        idx = rng.choice(Xw.index, 25, replace=False)
        for b in idx:
            masked.append((b, c, Xw.loc[b, c]))
        Zm.loc[idx, c] = np.nan
    for name, f in methods.items():
        F = f(Zm)
        e = [(F.loc[b, c] - t) / sd_all[c] for b, c, t in masked]
        mask_err[name].append(np.sqrt(np.mean(np.square(e))))
for name in methods:
    res[name]['mask_RMSE(sd)'] = float(np.mean(mask_err[name]))
    res[name]['mask_RMSE_std'] = float(np.std(mask_err[name]))
imp_tab = pd.DataFrame(res).T
print(imp_tab.round(4))
imp_tab.to_csv(o('prep_imputation_compare.csv'))
best = imp_tab['truth_RMSE(sd)'].idxmin() # real missing cells decide; masks test the
    MAR part
print('best imputer:', best)
Xw.to_csv(o('X_win.csv'))
Xc = methods[best](Xw)
assert Xc.notna().all().all()

# ------------------------------------------------------------- checks / save
S1 = Xc.rank().corr().values
print('mean|spearman(1.csv) - 2.csv| =', float(np.nanmean(np.abs(S1 - R2.values))))
Xc.to_csv(o('X_clean.csv'))
style.rename('style').to_csv(o('style.csv'))
th = load_thresholds().drop_duplicates('field_name_R').set_index('field_name_R')
names = pd.read_csv(p('2.csv'), index_col=0).index
info = pd.DataFrame({'col': cols, 'name': list(names), 'scale_in_1csv': [scale[c] for
    c in cols],
```

```
                'origin': [th['origin_paper'].get(c, np.nan) for c in cols],
                'floor_log10': floor.values.round(4), 'n_at_floor': n_floor.values,
                'n_missing': X.isna().sum().values,
                'n_winsorised': [int((out_df.col == c).sum()) if len(out_df) else 0
                    for c in cols]})
info.to_csv(o('compound_info.csv'), index=False)
save_json({'n_raw_scale': len(raw_cols), 'raw_cols': raw_cols, 'n_winsorised': len(
    out_df),
          'n_missing': int(X.isna().sum().sum()), 'n_truth': len(truth), 'best_imputer'
              : best, 'left_censored': sorted(left_censored), 'hop_sum_diag': diag,
          'imputation': res}, 'prep_summary.json')
print(info.origin.value_counts(dropna=False))
```

D.2 问题一 q1.py

```
"""Q1␣␣chemistry␣->␣50␣trained-panel␣sensory␣attributes␣->␣0-6␣grades␣(3.xlsx).

Base␣learners␣compared␣with␣repeated␣style-stratified␣5-fold␣CV;␣our␣model
"ENS+MTS"␣=␣equal-weight␣ensemble␣of␣complementary␣learners␣followed␣by␣a
multi-target␣stacking␣layer␣that␣borrows␣strength␣from␣correlated␣sensory
attributes␣(5.csv␣correlation␣graph).␣␣Out-of-fold␣predictions␣are␣used␣for
beers␣1-12,␣so␣their␣grades␣are␣genuine␣chemistry-only␣predictions.
"""
from common import *
import time, warnings
from joblib import Parallel, delayed
from sklearn.model_selection import StratifiedKFold, KFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge, RidgeCV
from sklearn.cross_decomposition import PLSRegression
from sklearn.svm import SVR
from sklearn.ensemble import RandomForestRegressor, ExtraTreesRegressor
from sklearn.metrics import r2_score, cohen_kappa_score
from scipy.stats import spearmanr
import lightgbm as lgb

warnings.filterwarnings('ignore')
X, style = load_clean()
Y = load_sensory().loc[X.index].drop(columns=['beer_id', 'tasting_category_fine'])
attrs = list(Y.columns)
# stratification label: merge styles with < 5 beers
cnt = style.value_counts()
strat = style.where(style.map(cnt) >= 5, 'rare')
NREP, NF = 3, 5

# sensory correlation graph from 5.csv (same order as 4.csv attributes)
R5 = load_r5()


def fit_predict(name, Xtr, Ytr, Xte):
    if name == 'Ridge':
        m = RidgeCV(alphas=np.logspace(0, 4, 25)).fit(Xtr, Ytr)
        return m.predict(Xte)
    if name == 'PLS':
        m = PLSRegression(n_components=6, scale=False).fit(Xtr, Ytr)
        return m.predict(Xte)
    if name == 'SVR':
```

```
        return np.column_stack([SVR(C=1.0, epsilon=0.1, gamma='scale').fit(Xtr, Ytr[:,
            k]).predict(Xte)
                                for k in range(Ytr.shape[1])])
    if name == 'RF':
        m = RandomForestRegressor(n_estimators=400, max_features=0.33, min_samples_leaf
            =2, n_jobs=1,
                                  random_state=SEED).fit(Xtr, Ytr)
        return m.predict(Xte)
    if name == 'ET':
        m = ExtraTreesRegressor(n_estimators=400, max_features=0.33, min_samples_leaf
            =2, n_jobs=1,
                                random_state=SEED).fit(Xtr, Ytr)
        return m.predict(Xte)
    if name == 'LGBM':
        out = []
        for k in range(Ytr.shape[1]):
            m = lgb.LGBMRegressor(n_estimators=300, learning_rate=0.03, num_leaves=7,
                min_child_samples=10,
                                  subsample=0.8, subsample_freq=1, colsample_bytree=0.5,
                                      reg_lambda=1.0,
                                  verbose=-1, n_jobs=1, random_state=SEED).fit(Xtr, Ytr[:,
                                      k])
            out.append(m.predict(Xte))
        return np.column_stack(out)
    raise ValueError(name)


BASE = ['Ridge', 'PLS', 'SVR', 'RF', 'ET', 'LGBM']
BTYPE = {'A_malt_all', 'A_hops_all', 'A_esters_all', 'F_malt_all', 'F_hops_all', '
    F_esters_all',
         'bitternes', 'sweetness', 'alcohol', 'aftertaste', 'body', 'co2', 'overall'}


def anchors(Yd):
    """0-grade␣anchor␣(1%␣quantile),␣median␣anchor␣and␣shared␣step,␣estimated␣from␣the␣
        labels␣Yd."""
    q01 = Yd.quantile(0.01)
    step = max((Yd[a].max() - q01[a]) for a in attrs if a not in BTYPE) / 6.0
    return q01.values, Yd.median().values, step


def grade_cont(Z, anc, step_factor=1.0):
    """continuous␣0-6␣grade␣of␣predictions␣Z␣under␣the␣anchors␣anc␣(not␣rounded,␣not␣
        clipped)"""
    q01, med, step = anc
    step = step * step_factor
    bt = np.array([a in BTYPE for a in attrs])
    return np.where(bt, 3.0 + (Z - med) / step, (Z - q01) / step)

ENS = ['Ridge', 'SVR', 'ET', 'LGBM']
LGRID = np.linspace(0, 1, 11) # complementary: linear / kernel / bagging / boosting


def stack_layer(P_in, Y_in, P_out, thr=0.3):
    """Multi-target␣stacking:␣attribute␣k␣is␣re-predicted␣from␣the␣base
␣␣␣␣predictions␣of␣itself␣and␣its␣neighbours␣(|rho|>thr␣in␣5.csv)."""
```

```
    Z = np.zeros_like(P_out)
    for k, a in enumerate(attrs):
        nb = [j for j, b in enumerate(attrs) if j == k or abs(R5.loc[a, b]) > thr]
        m = RidgeCV(alphas=np.logspace(-1, 3, 20)).fit(P_in[:, nb], Y_in[:, k])
        Z[:, k] = m.predict(P_out[:, nb])
    return Z


def job(rep, fold, tr, te):
    sc = StandardScaler().fit(X.values[tr])
    Xtr, Xte = sc.transform(X.values[tr]), sc.transform(X.values[te])
    Ytr = Y.values[tr]
    res = {}
    for name in BASE:
        res[name] = fit_predict(name, Xtr, Ytr, Xte)
    res['ENS'] = np.mean([res[n] for n in ENS], axis=0)
    # inner CV on the training part to build stage-2 training data
    P_in = np.zeros_like(Ytr)
    for itr, ite in KFold(5, shuffle=True, random_state=SEED + rep).split(Xtr):
        P_in[ite] = np.mean([fit_predict(n, Xtr[itr], Ytr[itr], Xtr[ite]) for n in ENS
            ], axis=0)
    res['ENS+MTS'] = stack_layer(P_in, Ytr, res['ENS'])
    # adaptive MTS (ours): per attribute, the stacking weight lambda_k is chosen on the
    # inner out-of-fold predictions, so stacking is only used where it actually helps
    S_in = np.zeros_like(P_in)
    for itr, ite in KFold(5, shuffle=True, random_state=SEED + 7).split(P_in):
        S_in[ite] = stack_layer(P_in[itr], Ytr[itr], P_in[ite])
    lam = np.zeros(len(attrs))
    for k in range(len(attrs)):
        errs = [np.mean((Ytr[:, k] - ((1 - l) * P_in[:, k] + l * S_in[:, k])) ** 2) for
             l in LGRID]
        lam[k] = LGRID[int(np.argmin(errs))]
    res['ENS+AMTS'] = (1 - lam) * res['ENS'] + lam * res['ENS+MTS']
    res['lam'] = lam
    # per-fold test R2 of every model (for the paired comparison)
    res['r2_fold'] = {m: float(np.mean([r2_score(Y.values[te, k], res[m][:, k]) for k
         in range(len(attrs))]))
                      for m in BASE + ['ENS', 'ENS+MTS', 'ENS+AMTS']}
    # grading anchors and variance re-calibration are estimated on the training fold
        only:
    # anchors from the training labels, calibration line from the inner out-of-fold
        AMTS predictions
    A_in = (1 - lam) * P_in + lam * S_in
    res['cal_ab'] = np.array([np.polyfit(A_in[:, k], Ytr[:, k], 1) for k in range(len(
        attrs))])
    res['anchors'] = anchors(pd.DataFrame(Ytr, columns=attrs))
    return rep, te, res


t0 = time.time()
splits = []
for rep in range(NREP):
    skf = StratifiedKFold(NF, shuffle=True, random_state=SEED + rep)
    for f, (tr, te) in enumerate(skf.split(X, strat)):
        splits.append((rep, f, tr, te))
out = Parallel(n_jobs=8, verbose=0)(delayed(job)(*s) for s in splits)
```

```
print('CV␣done␣in␣%.0f␣s' % (time.time() - t0))

MODELS = BASE + ['ENS', 'ENS+MTS', 'ENS+AMTS']
OOF = {m: np.zeros((NREP,) + Y.shape) for m in MODELS}
LAM = []
for rep, te, res in out:
    LAM.append(res['lam'])
    for m in MODELS:
        OOF[m][rep][te] = res[m]
LAM = pd.Series(np.mean(LAM, axis=0), index=attrs)
LAM.to_csv(o('q1_amts_lambda.csv'))
print('mean␣lambda', LAM.mean().round(3))

# ------------------------------------------------------------------ metrics
rows = []
per_attr = {}
for m in MODELS:
    r2 = np.array([[r2_score(Y.values[:, k], OOF[m][r][:, k]) for k in range(len(attrs)
        )] for r in range(NREP)])
    rho = np.array([[spearmanr(Y.values[:, k], OOF[m][r][:, k]).statistic for k in
        range(len(attrs))]
                    for r in range(NREP)])
    per_attr[m] = r2.mean(0)
    rows.append(dict(model=m, R2_mean=r2.mean(), R2_sd_rep=r2.mean(1).std(), rho_mean=
        np.nanmean(rho),
                     n_attr_R2_pos=int((r2.mean(0) > 0).sum()), R2_median=np.median(r2.
                         mean(0))))
cmp = pd.DataFrame(rows).set_index('model')
print(cmp.round(4))
cmp.to_csv(o('q1_model_compare.csv'))
pa = pd.DataFrame(per_attr, index=attrs)
pa.to_csv(o('q1_r2_per_attr.csv'))
print(pa[['LGBM', 'ET', 'ENS', 'ENS+MTS']].round(3).sort_values('ENS+MTS', ascending=
    False).to_string())

# ------------------------------------------------------------------ paired comparison
    over the 15 folds
from scipy.stats import wilcoxon
R2F = pd.DataFrame([dict(rep=rep, **res['r2_fold']) for rep, te, res in out])
R2F.to_csv(o('q1_r2_per_fold.csv'), index=False)
rng = np.random.default_rng(SEED)
pair_rows = []
for ref in ['ENS', 'ENS+MTS', 'ET']:
    d = (R2F['ENS+AMTS'] - R2F[ref]).values
    bs = [rng.choice(d, len(d)).mean() for _ in range(5000)]
    pair_rows.append(dict(ref=ref, mean_diff=d.mean(), ci_lo=np.percentile(bs, 2.5),
        ci_hi=np.percentile(bs, 97.5),
                          n_pos=int((d > 0).sum()), n=len(d), wilcoxon_p=wilcoxon(d).
                              pvalue))
pair = pd.DataFrame(pair_rows).set_index('ref')
print(pair.round(4))
pair.to_csv(o('q1_paired.csv'))

# ------------------------------------------------------------------ 0-6 grading
names = load_sensory_names()
ANC = anchors(Y) # full-label anchors: define the panel-derived reference grades only
```

```
q01, med = pd.Series(ANC[0], index=attrs), pd.Series(ANC[1], index=attrs)
STEP = ANC[2] # z-units per grade point (shared by all attributes)
A_range = STEP * 6.0


def to_grade(Z, step=STEP, mid=3.0):
    Z = pd.DataFrame(Z, columns=attrs)
    G = pd.DataFrame(index=Z.index, columns=attrs, dtype=float)
    for a in attrs:
        g = mid + (Z[a] - med[a]) / step if a in BTYPE else (Z[a] - q01[a]) / step
        G[a] = np.clip(np.rint(g), 0, 6)
    return G.astype(int)


G_true = to_grade(Y.values)
G_true.index = Y.index
best = 'ENS+AMTS'


def oof_grades(kind, step_factor=1.0):
    """Out-of-fold␣0-6␣grades:␣every␣test␣fold␣is␣graded␣with␣the␣anchors␣(and,␣for␣'
        cal',␣the
␣␣␣␣calibration␣line)␣of␣its␣own␣training␣fold;␣the␣three␣repeats␣are␣averaged␣on␣the␣
    continuous
␣␣␣␣grade␣scale␣and␣then␣rounded.␣␣No␣test␣label␣enters␣any␣fitted␣quantity."""
    C = np.zeros((NREP,) + Y.shape)
    for rep, te, res in out:
        P = res[best]
        if kind == 'cal':
            P = res['cal_ab'][:, 0] * P + res['cal_ab'][:, 1]
        elif kind == 'median':
            P = np.tile(res['anchors'][1], (len(te), 1))
        C[rep][te] = grade_cont(P, res['anchors'], step_factor)
    C = C.mean(0)
    return pd.DataFrame(np.clip(np.rint(C), 0, 6).astype(int), index=Y.index, columns=
        attrs), C


def grade_metrics(G, G_ref=None):
    G_ref = G_true if G_ref is None else G_ref
    ex = (G.values == G_ref.values).mean()
    w1 = (np.abs(G.values - G_ref.values) <= 1).mean()
    mae = np.abs(G.values - G_ref.values).mean()
    qwk = np.nanmean([cohen_kappa_score(G_ref[a], G[a], weights='quadratic') if G_ref[a
        ].nunique() > 1 else np.nan
                      for a in attrs])
    return dict(exact=ex, within1=w1, MAE=mae, QWK=qwk)


G_raw, _ = oof_grades('raw')
G_cal, C_cal = oof_grades('cal')
G_med, _ = oof_grades('median')
# post-hoc diagnostic only (calibration line fitted on all labels, as in the first
    version) -- not used for grades
P_best = OOF[best].mean(0)
P_ph = np.column_stack([np.polyval(np.polyfit(P_best[:, k], Y.values[:, k], 1), P_best
```

```
    [:, k]) for k in range(len(attrs))])
gm = pd.DataFrame({'baseline_median': grade_metrics(G_med), 'raw_pred': grade_metrics(
    G_raw),
                'linear_recal': grade_metrics(G_cal), 'posthoc_full_recal':
                    grade_metrics(to_grade(P_ph))}).T
print(gm.round(4))
gm.to_csv(o('q1_grade_metrics.csv'))
# the re-calibrated grades are the final ones by design (fixed before looking at the
    test metrics)
use_cal = True
P_cal = np.zeros((NREP,) + Y.shape)
for rep, te, res in out:
   P_cal[rep][te] = res['cal_ab'][:, 0] * res[best] + res['cal_ab'][:, 1]
P_final = pd.DataFrame(P_cal.mean(0), index=Y.index, columns=attrs)
G_pred = G_cal
slope = {a: float(np.mean([res['cal_ab'][k, 0] for _, _, res in out])) for k, a in
    enumerate(attrs)}

# step sensitivity: grade agreement for alternative step sizes (fold-internal anchors
    scaled by the same factor)
sens = []
for f in [0.8, 0.9, 1.0, 1.1, 1.25]:
    G1, _ = oof_grades('cal', f)
    G0 = to_grade(Y.values, step=STEP * f)
    G0.index = Y.index
    m_ = grade_metrics(G1, G0)
    sens.append(dict(step_factor=f, step=STEP * f, exact=m_['exact'], within1=m_['
        within1'], MAE=m_['MAE'],
                     QWK=m_['QWK'], mean_grade_true=G0.values.mean(), mean_grade_pred=G1.
                         values.mean()))
sens = pd.DataFrame(sens)
print(sens.round(4))
sens.to_csv(o('q1_step_sensitivity.csv'), index=False)

# ------------------------------------------------------------------ acetaldehyde (not
    in 4.csv)
cal = load_calibrated().loc[X.index]
th = load_thresholds().set_index('field_name_R')
T_ac_8xlsx = float(th.loc['acetaldehyde', 'threshold_mean'])
T_ac = 10.0 # Meilgaard literature threshold (mg/L); 8.xlsx value exceeds every beer
d7 = pd.read_csv(p('7.csv')).set_index('beer').loc[X.index]
oav = cal['acetaldehyde'].astype(float) / T_ac
K_HILL = 5.0 # g(OAV=1)=1 grade: Beidler/Hill saturating response
g_ac = np.clip(np.rint(6 * oav / (oav + K_HILL)), 0, 6).astype(int)
rho_ac = spearmanr(cal['acetaldehyde'], d7['aroma_acetaldehyde']).statistic
print('acetaldehyde␣threshold', T_ac, 'OAV␣median', oav.median(), 'spearman␣with␣
    RateBeer␣aroma_acetaldehyde', rho_ac)

# ------------------------------------------------------------------ save
P_final.to_csv(o('q1_oof_pred_z.csv'))
G_pred.assign(acetaldehyde=g_ac).to_csv(o('q1_grades_pred_all.csv'))
G_true.assign(acetaldehyde=g_ac).to_csv(o('q1_grades_panel_all.csv'))
sel = list(range(1, 13))
out12 = G_pred.loc[sel].assign(acetaldehyde=g_ac.loc[sel], style=style.loc[sel])
out12.to_csv(o('q1_grades_beer1_12.csv'))
ref12 = G_true.loc[sel]
```

```
agree12 = dict(exact=float((out12[attrs].values == ref12.values).mean()),
               within1=float((np.abs(out12[attrs].values - ref12.values) <= 1).mean()))
print('beer␣1-12␣agreement␣with␣panel-derived␣grades:', agree12)
print(out12.T.to_string())
save_json({'best_base_by_R2': best, 'STEP': STEP, 'A_range': A_range, '
    use_linear_recal': bool(use_cal),
          'slope': slope, 'acetaldehyde_threshold': T_ac, 'acetaldehyde_threshold_8xlsx
              ': T_ac_8xlsx, 'hill_K': K_HILL, 'rho_acetaldehyde_ratebeer': rho_ac,
          'agree_beer1_12': agree12, 'cv': cmp.to_dict(), 'grade_metrics': gm.to_dict()
              }, 'q1_summary.json')
```

D.3 问题一 q1model.py

```
"""Reusable␣Q1␣sensory␣predictor␣(equal-weight␣ensemble)␣and␣the␣z␣->␣0-6␣grading␣rule
    ."""
from common import *
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV
from sklearn.svm import SVR
from sklearn.ensemble import ExtraTreesRegressor
import lightgbm as lgb

BTYPE = {'A_malt_all', 'A_hops_all', 'A_esters_all', 'F_malt_all', 'F_hops_all', '
    F_esters_all',
        'bitternes', 'sweetness', 'alcohol', 'aftertaste', 'body', 'co2', 'overall'}


class EnsPredictor:
    def fit(self, X, Y):
        self.cols, self.attrs = list(X.columns), list(Y.columns)
        self.sc = StandardScaler().fit(X.values)
        Xs, Yv = self.sc.transform(X.values), Y.values
        self.ridge = RidgeCV(alphas=np.logspace(0, 4, 25)).fit(Xs, Yv)
        self.svr = [SVR(C=1.0, epsilon=0.1, gamma='scale').fit(Xs, Yv[:, k]) for k in
            range(Yv.shape[1])]
        self.et = ExtraTreesRegressor(n_estimators=400, max_features=0.33,
            min_samples_leaf=2, n_jobs=-1,
                                random_state=SEED).fit(Xs, Yv)
        self.lgb = [lgb.LGBMRegressor(n_estimators=300, learning_rate=0.03, num_leaves
            =7, min_child_samples=10,
                                subsample=0.8, subsample_freq=1, colsample_bytree=0.5,
                                     verbose=-1, n_jobs=4,
                                random_state=SEED).fit(Xs, Yv[:, k]) for k in range(Yv
                                    .shape[1])]
        return self

    def predict(self, X):
        Xs = self.sc.transform(np.asarray(X)[:, :len(self.cols)] if not isinstance(X,
            pd.DataFrame)
                           else X[self.cols].values)
        P = [self.ridge.predict(Xs),
             np.column_stack([m.predict(Xs) for m in self.svr]),
             self.et.predict(Xs),
             np.column_stack([m.predict(Xs) for m in self.lgb])]
        return pd.DataFrame(np.mean(P, axis=0), columns=self.attrs)
```

```
class Grader:
    """Shared-step two-anchor grading (same rule as q1.py)."""

    def __init__(self, Y):
        self.attrs = list(Y.columns)
        self.q01, self.med = Y.quantile(0.01), Y.median()
        self.step = max((Y[a].max() - self.q01[a]) for a in self.attrs if a not in
            BTYPE) / 6.0

    def cont(self, Z):
        Z = pd.DataFrame(np.asarray(Z), columns=self.attrs)
        G = pd.DataFrame(index=Z.index, columns=self.attrs, dtype=float)
        for a in self.attrs:
            G[a] = (3 + (Z[a] - self.med[a]) / self.step) if a in BTYPE else (Z[a] -
                self.q01[a]) / self.step
        return G.clip(0, 6)

    def grade(self, Z):
        return np.rint(self.cont(Z)).astype(int)
```

D.4 问题一稳健性实验 q1_missing.py

```
"""Q1 robustness: sensory-prediction accuracy when additional chemistry cells are
    missing.

Cells of the (winsorised) matrix are masked completely at random at rate r, imputed
with mean / KNN / graph-guided ridge, and a light ensemble (Ridge + ExtraTrees) is
scored by style-stratified 5-fold CV (mean R2 over 50 attributes)."""
from common import *
import warnings
from joblib import Parallel, delayed
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.impute import KNNImputer
from sklearn.metrics import r2_score

warnings.filterwarnings('ignore')
Xw = pd.read_csv(o('X_win.csv'), index_col=0)
Xc, style = load_clean()
Y = load_sensory().loc[Xc.index].drop(columns=['beer_id', 'tasting_category_fine'])
cols = list(Xc.columns)
R2 = pd.DataFrame(pd.read_csv(p('2.csv'), index_col=0).values, index=cols, columns=
    cols)
style_d = pd.get_dummies(style).astype(float)
cnt = style.value_counts()
strat = style.where(style.map(cnt) >= 5, 'rare')


def imp_mean(Z):
    return Z.fillna(Z.mean())


def imp_knn(Z):
    mu, sd = Z.mean(), Z.std().replace(0, 1)
    return pd.DataFrame(KNNImputer(n_neighbors=7, weights='distance').fit_transform((Z
```

```
        - mu) / sd),
                      index=Z.index, columns=Z.columns) * sd + mu


def imp_graph(Z, m=15):
    Z0 = Z.fillna(Z.median())
    out = Z.copy()
    for _ in range(2):
        for c in Z.columns[Z.isna().any()]:
            miss = Z[c].isna()
            nb = R2[c].drop(c).abs().sort_values(ascending=False).index[:m]
            F = pd.concat([Z0[nb], style_d], axis=1)
            Fs = (F - F.mean()) / F.std().replace(0, 1)
            mdl = RidgeCV(alphas=np.logspace(-2, 3, 12)).fit(Fs[~miss], Z.loc[~miss, c])
            out.loc[miss, c] = np.clip(mdl.predict(Fs[miss]), Z[c].min(), Z[c].max())
        Z0 = out.copy()
    return out


IMP = {'mean': imp_mean, 'KNN': imp_knn, 'Graph-Ridge': imp_graph}


def cv_r2(Xf):
    P = np.zeros(Y.shape)
    for tr, te in StratifiedKFold(5, shuffle=True, random_state=SEED).split(Xf, strat):
        sc = StandardScaler().fit(Xf.values[tr])
        a, b = sc.transform(Xf.values[tr]), sc.transform(Xf.values[te])
        p1 = RidgeCV(alphas=np.logspace(0, 4, 25)).fit(a, Y.values[tr]).predict(b)
        p2 = ExtraTreesRegressor(300, max_features=0.33, min_samples_leaf=2, n_jobs=1,
                                 random_state=SEED).fit(a, Y.values[tr]).predict(b)
        P[te] = (p1 + p2) / 2
    return np.mean([r2_score(Y.values[:, k], P[:, k]) for k in range(Y.shape[1])])


def job(rate, rep):
    rs = np.random.default_rng(100 * rep + int(rate * 1000))
    Z = Xc.copy()
    if rate > 0:
        M = rs.random(Z.shape) < rate
        Z = Z.mask(M)
    out = []
    for name, f in IMP.items():
        Zi = f(Z) if rate > 0 else Z
        out.append(dict(rate=rate, rep=rep, imputer=name, R2=cv_r2(Zi)))
    return out


res = Parallel(n_jobs=15)(delayed(job)(r, k) for r in [0, 0.05, 0.1, 0.2, 0.3, 0.4]
    for k in range(3))
res = pd.DataFrame([x for r in res for x in r])
tab = res.groupby(['rate', 'imputer']).R2.agg(['mean', 'std']).unstack()
print(tab.round(4))
res.to_csv(o('q1_missing_robust.csv'), index=False)
```

D.5 问题二 q2.py

```
"""Q2␣␣compound␣co-expression␣graph␣+␣Top-3␣key␣compounds␣per␣sensory␣attribute.
```

```
1. Weighted co-expression network from 2.csv: soft threshold beta chosen by the
   scale-free criterion, topological overlap (TOM), average-linkage modules,
   module eigengenes and module-trait correlations.
2. Per attribute, three importance views computed on 1.csv features:
   M1 Spearman |rho|, M2 LightGBM TreeSHAP mean|phi|, M3 Lasso stability selection.
   Ours (GF-MMR): Borda fusion -> random-walk smoothing on TOM -> maximal-marginal-
   relevance selection with TOM redundancy penalty -> Top-3.
3. Stability: bootstrap selection frequency (B resamples) + inter-method Jaccard.
4. Chemical rationality: origin consistency + literature marker hits vs permutation.
"""
from common import *
import time, warnings
from joblib import Parallel, delayed
from scipy.cluster.hierarchy import linkage, fcluster
from scipy.spatial.distance import squareform
from scipy.stats import spearmanr, rankdata
from sklearn.linear_model import Lasso, LassoCV
import lightgbm as lgb

warnings.filterwarnings('ignore')
X, style = load_clean()
Y = load_sensory().loc[X.index].drop(columns=['beer_id', 'tasting_category_fine'])
attrs = list(Y.columns)
info = pd.read_csv(o('compound_info.csv')).set_index('col')
cols = list(X.columns)
p_ = len(cols)
R2 = pd.read_csv(p('2.csv'), index_col=0).values
R2 = pd.DataFrame(R2, index=cols, columns=cols)
AGG = ['esters_sum', 'aroma_hops_sum', 'acids_sum', 'sulfur_sum', 'kcalperc'] #
    derived sums, not compounds
cand = np.array([c not in AGG for c in cols])
B = 60

# ================================================================ 1. network
absR = R2.abs().values.copy()
np.fill_diagonal(absR, 0)
sf = []
for beta in range(1, 13):
    k = (absR ** beta).sum(1)
    h, e = np.histogram(k, bins=10)
    mid = 0.5 * (e[1:] + e[:-1])
    m = h > 0
    x_, y_ = np.log10(mid[m]), np.log10(h[m] / h.sum())
    b = np.polyfit(x_, y_, 1)
    r2fit = 1 - np.sum((y_ - np.polyval(b, x_)) ** 2) / np.sum((y_ - y_.mean()) ** 2)
    sf.append(dict(beta=beta, signed_R2=-np.sign(b[0]) * r2fit, slope=b[0], mean_k=k.
        mean(), median_k=np.median(k)))
sf = pd.DataFrame(sf)
ok = sf[sf.signed_R2 >= 0.80]
BETA = int(ok.beta.iloc[0]) if len(ok) else int(sf.loc[sf.signed_R2.idxmax(), 'beta'])
print(sf.round(3)); print('beta =', BETA)
A = absR ** BETA
kdeg = A.sum(1)
L = A @ A
TOM = (L + A) / (np.minimum.outer(kdeg, kdeg) + 1 - A)
```

```
np.fill_diagonal(TOM, 1.0)
TOM = pd.DataFrame(TOM, index=cols, columns=cols)
import networkx as nx
Gt = nx.Graph()
Tv = TOM.values
iu = np.triu_indices(p_, 1)
for i, j in zip(*iu):
    if Tv[i, j] > 0:
        Gt.add_edge(cols[i], cols[j], weight=float(Tv[i, j]))
best_s, best_res, comms = -1, None, None
for res_ in [0.6, 0.8, 1.0, 1.2, 1.5]:
    cm = nx.community.louvain_communities(Gt, weight='weight', resolution=res_, seed=
        SEED)
    q = nx.community.modularity(Gt, cm, weight='weight')
    nbig = sum(len(c) >= 5 for c in cm)
    print('louvain resolution', res_, 'modularity %.3f' % q, 'n_comm', len(cm), 'n>=5',
         nbig, 'max', max(len(c) for c in cm))
    if q > best_s:
        best_s, best_res, comms = q, res_, cm
best_k = len(comms)
comms = sorted(comms, key=len, reverse=True)
mod = pd.Series('grey', index=cols)
k_ = 0
for c in comms:
    if len(c) >= 5:
        k_ += 1
        mod[list(c)] = f'M{k_}'
Z = None
print('modules: resolution', best_res, 'modularity %.3f' % best_s, mod.value_counts().
    to_dict())
Xs = (X - X.mean()) / X.std()
ME = {}
for m in sorted(set(mod) - {'grey'}, key=lambda s: int(s[1:])):
    sub = Xs[mod.index[mod == m]].values
    u, s_, vt = np.linalg.svd(sub - sub.mean(0), full_matrices=False)
    pc = u[:, 0] * s_[0]
    if np.corrcoef(pc, sub.mean(1))[0, 1] < 0:
        pc = -pc
    ME[m] = pc
ME = pd.DataFrame(ME, index=X.index)
MT = pd.DataFrame({a: [spearmanr(ME[m], Y[a]).statistic for m in ME] for a in attrs},
    index=ME.columns)
kin = pd.Series({c: TOM.loc[c, mod.index[mod == mod[c]]].sum() - 1 for c in cols})
modinfo = pd.DataFrame({'module': mod, 'origin': info.origin, 'name': info.name, '
    k_total': kdeg, 'k_in': kin})
hubs = modinfo[modinfo.module != 'grey'].sort_values('k_in', ascending=False).groupby(
    'module').head(3)
print(hubs.sort_values('module')[['module', 'name', 'origin', 'k_in']])
comp = modinfo[modinfo.module != 'grey'].groupby('module').origin.agg(lambda s: s.
    value_counts().to_dict())
print(comp)
sf.to_csv(o('q2_softthreshold.csv'), index=False)
modinfo.to_csv(o('q2_modules.csv'))
MT.to_csv(o('q2_module_trait.csv'))
TOM.to_csv(o('q2_tom.csv'))
```

```
# ============================================================== 2. importance
Xv = Xs.values
Yv = Y.values
lasso_alpha = {}
NJ = 14
_al = Parallel(n_jobs=NJ)(delayed(lambda k: LassoCV(cv=5, alphas=40, random_state=SEED
    , max_iter=5000)
                              .fit(Xv, Yv[:, k]).alpha_)(k) for k in range(len(attrs)))
lasso_alpha = dict(zip(attrs, _al))


def imp_spearman(Xa, ya):
    rx = np.apply_along_axis(rankdata, 0, Xa)
    ry = rankdata(ya)
    rx = (rx - rx.mean(0)) / (rx.std(0) + 1e-12)
    ry = (ry - ry.mean()) / ry.std()
    return np.abs((rx * ry[:, None]).mean(0))


def imp_shap(Xa, ya, seeds=(0,)):
    v = np.zeros(Xa.shape[1])
    for s in seeds:
        m = lgb.LGBMRegressor(n_estimators=300, learning_rate=0.03, num_leaves=7,
            min_child_samples=10,
                              subsample=0.8, subsample_freq=1, colsample_bytree=0.5,
                                  verbose=-1, n_jobs=1,
                              random_state=SEED + s).fit(Xa, ya)
        v += np.abs(m.predict(Xa, pred_contrib=True)[:, :-1]).mean(0)
    return v / len(seeds)


def imp_stab(Xa, ya, alpha, nsub=20, rs=None):
    rs = rs or np.random.default_rng(SEED)
    f = np.zeros(Xa.shape[1])
    n = len(ya)
    for _ in range(nsub):
        idx = rs.choice(n, n // 2, replace=False)
        c = Lasso(alpha=alpha, max_iter=5000).fit(Xa[idx], ya[idx]).coef_
        f += (np.abs(c) > 1e-8)
    return f / nsub


def borda(v):
    r = rankdata(-v, method='average') # 1 = best
    return 1 - (r - 1) / (len(v) - 1)


Wt = TOM.values.copy()
np.fill_diagonal(Wt, 0)
Wn = Wt / Wt.sum(1, keepdims=True)
ALPHA, LAM = 0.3, 0.6


def gf_mmr(scores, k=3):
    """scores: list of importance vectors -> Top-k indices (GF-MMR)."""
    s = np.mean([borda(v) for v in scores], axis=0)
```

```
    s = (1 - ALPHA) * s + ALPHA * (Wn @ s) # random-walk smoothing on co-expression
        graph
    s = np.where(cand, s, -np.inf)
    sel = [int(np.argmax(s))]
    while len(sel) < k:
        red = TOM.values[:, sel].max(1)
        g = s - LAM * red
        g[sel] = -np.inf
        sel.append(int(np.argmax(g)))
    return sel, s


def topk(v, k=3):
    v = np.where(cand, v, -np.inf)
    return list(np.argsort(-v)[:k])


def attr_job(k, a, Xa, Ya, seeds, nsub, seed):
    rs = np.random.default_rng(seed)
    m1 = imp_spearman(Xa, Ya[:, k])
    m2 = imp_shap(Xa, Ya[:, k], seeds)
    m3 = imp_stab(Xa, Ya[:, k], lasso_alpha[a], nsub, rs) + 1e-3 * m1 # tie-break by |
        rho|
    g, s = gf_mmr([m1, m2, m3])
    fused_plain = topk(np.mean([borda(m1), borda(m2), borda(m3)], axis=0))
    return a, dict(M1=topk(m1), M2=topk(m2), M3=topk(m3), FUSE=fused_plain, GF=g, m1=m1
        , m2=m2, m3=m3, s=s)


def run_all(Xa, Ya, seeds=(0,), nsub=20, rs=None, n_jobs=1):
    seed0 = int(rs.integers(1 << 30)) if rs is not None else SEED
    if n_jobs == 1:
        res = [attr_job(k, a, Xa, Ya, seeds, nsub, seed0 + k) for k, a in enumerate(
            attrs)]
    else:
        res = Parallel(n_jobs=n_jobs)(delayed(attr_job)(k, a, Xa, Ya, seeds, nsub,
            seed0 + k)
                                      for k, a in enumerate(attrs))
    return dict(res)


t0 = time.time()
full = run_all(Xv, Yv, seeds=(0, 1, 2), nsub=50, n_jobs=NJ)
print('full-data␣importance␣%.0f␣s' % (time.time() - t0))


def boot(b):
    rs = np.random.default_rng(1000 + b)
    idx = rs.choice(len(Yv), len(Yv), replace=True)
    Xb = Xv[idx]
    Xb = (Xb - Xb.mean(0)) / (Xb.std(0) + 1e-12)
    r = run_all(Xb, Yv[idx], seeds=(b,), nsub=20, rs=rs)
    return {a: {m: r[a][m] for m in ['M1', 'M2', 'M3', 'FUSE', 'GF']} for a in attrs}


t0 = time.time()
```

```
BOOT = Parallel(n_jobs=NJ)(delayed(boot)(b) for b in range(B))
print('bootstrap␣%.0f␣s' % (time.time() - t0))

# ================================================================ 3. stability
METH = ['M1', 'M2', 'M3', 'FUSE', 'GF']
freq = {m: np.zeros((len(attrs), p_)) for m in METH}
for r in BOOT:
    for k, a in enumerate(attrs):
        for m in METH:
            freq[m][k, r[a][m]] += 1
for m in METH:
    freq[m] /= B
stab_rows = []
for k, a in enumerate(attrs):
    row = {'attr': a}
    for m in METH:
        row[m] = freq[m][k, full[a][m]].mean() # mean bootstrap frequency of the full-
            data Top-3
    stab_rows.append(row)
stab = pd.DataFrame(stab_rows).set_index('attr')


def jacc(a, b):
    a, b = set(a), set(b)
    return len(a & b) / len(a | b)


# bootstrap Jaccard of each method's Top-3 to its own full-data Top-3
bj = {m: np.mean([[jacc(r[a][m], full[a][m]) for a in attrs] for r in BOOT]) for m in
    METH}
JM = pd.DataFrame([[np.mean([jacc(full[a][m1], full[a][m2]) for a in attrs]) for m2 in
     METH] for m1 in METH],
                  index=METH, columns=METH)
# redundancy: mean pairwise TOM inside each Top-3
redund = {m: np.mean([np.mean([TOM.values[i, j] for i in full[a][m] for j in full[a][m
    ] if i < j]) for a in attrs])
          for m in METH}
# module-level stability: co-expressed compounds are interchangeable, so compare the
    sets of
# modules (grey compounds count as their own singleton module) chosen in each resample
def modset(sel):
    return {mod.iloc[i] if mod.iloc[i] != 'grey' else cols[i] for i in sel}


mbj = {m: np.mean([[jacc(modset(r[a][m]), modset(full[a][m])) for a in attrs] for r in
     BOOT]) for m in METH}
mfreq = {m: np.mean([np.mean([np.mean([mod.iloc[i] in modset(r[a][m]) if mod.iloc[i]
    != 'grey' else i in r[a][m]
                                   for r in BOOT]) for i in full[a][m]]) for a in attrs
                                       ]) for m in METH}
import pickle
pickle.dump({'BOOT': BOOT, 'full_top': {a: {m: full[a][m] for m in METH} for a in
    attrs}}, open(o('q2_boot.pkl'), 'wb'))
summary_stab = pd.DataFrame({'mean_top3_boot_freq': stab.mean(), 'boot_Jaccard': pd.
    Series(bj),
                             'module_freq': pd.Series(mfreq), 'module_boot_Jaccard': pd.
```

```
                            Series(mbj),
                        'mean_TOM_within_top3': pd.Series(redund),
                        'n_modules_in_top3': pd.Series({m: np.mean([len(set(mod.iloc[
                            full[a][m]])) for a in attrs])
                                                    for m in METH})})
print(summary_stab.round(3)); print(JM.round(3))
stab.to_csv(o('q2_stability_per_attr.csv'))
summary_stab.to_csv(o('q2_stability_summary.csv'))
JM.to_csv(o('q2_method_jaccard.csv'))

# ================================================================ 4. rationality
names = load_sensory_names().set_index('R_name')
LIT = {
    'bitternes': ['bitterness'], 'alcohol': ['ethanol..v.v.', 'glycerol', '
        isoamyl_alcohol', 'isobutanol', 'X1.propanol'],
    'body': ['ethanol..v.v.', 'glycerol', 'unfermentables', 'protein.g.L.', 'betaglucan
        .mg.L.', 'GFS'],
    'acidity': ['lactic_acid.mg.L.', 'acetic_acid.g.L.', 'pH'], 'lactic': ['lactic_acid
        .mg.L.', 'pH'],
    'acetic': ['acetic_acid.g.L.', 'ethyl_acetate', 'pH'],
    'barnyard': ['X4.ethyl.phenol', 'X4.ethyl.guaiacol', 'lactic_acid.mg.L.', '
        acetic_acid.g.L.', 'ethyl.lactate'],
    'X4vg': ['X4VG', 'styrene'], 'clove': ['X4VG', 'eugenol', 'styrene'], 'dms': ['DMS'
        ],
    'diacetyl': ['X2.3.butanedione', 'X2.3.pentanedione'], 't2n': ['X.E..2.nonenal', 'X
        .E.E..2.4.decadienal'],
    'metallic': ['iron'], 'sweetness': ['GFS', 'unfermentables', 'glycerol'], 'co2': ['
        CO2.PSI.'],
    'orange': ['limonene', 'linalool', 'citronellol', 'geraniol', 'myrcene', 'valencene
        '],
    'coriander': ['linalool', 'geraniol', 'camphor', 'X1.alpha.terpineol'],
    'stale_hops': ['X3.methylbutanal', 'isovaleric', 'lactic_acid.mg.L.'],
    'aftertaste': ['bitterness', 'ethanol..v.v.'], 'overall': [],
}
CATMAP = {'Malt': {'Malt'}, 'Hops': {'Hops'}, 'Yeast': {'Yeast'}, 'Wild␣flora': {'Wild
    ␣flora'}}
MALT_EXTRA = {'color.EBC.'}
rat_rows = []
rs = np.random.default_rng(SEED)
for k, a in enumerate(attrs):
    cat = names.loc[a, 'Category'] if a in names.index else np.nan
    exp_cols = set(LIT.get(a, []))
    if a[:2] in ('A_', 'F_') and cat in CATMAP:
        exp_origin = CATMAP[cat]
        exp_cols |= MALT_EXTRA if cat == 'Malt' else set()
    else:
        exp_origin = set()
    if not exp_origin and not exp_cols:
        continue
    ok_mask = np.array([(info.origin.get(c) in exp_origin) or (c in exp_cols) for c in
        cols]) & cand
    top = full[a]['GF']
    hit = bool(ok_mask[top].any())
    frac = ok_mask[top].mean()
    # permutation baseline: random Top-3 from candidates
    perm = np.mean([ok_mask[rs.choice(np.where(cand)[0], 3, replace=False)].any() for _
```

```
        in range(2000)])
    rat_rows.append(dict(attr=a, category=cat, hit_any=hit, frac_consistent=frac,
        random_hit_prob=perm,
                         **{f'hit_{m}': bool(ok_mask[full[a][m]].any()) for m in METH}))
rat = pd.DataFrame(rat_rows).set_index('attr')
print(rat.round(3))
print('GF hit rate %.3f vs random expectation %.3f' % (rat.hit_any.mean(), rat.
    random_hit_prob.mean()))
print('hit rate by method', {m: rat[f'hit_{m}'].mean().round(3) for m in METH})
rat.to_csv(o('q2_rationality.csv'))

# ============================================================== output table
rows = []
for k, a in enumerate(attrs):
    r = {'attr': a, 'label': sensory_label_map().get(a, a).split('\n')[0].replace('
        _x000D_', '').strip()}
    for j, i in enumerate(full[a]['GF']):
        c = cols[i]
        r[f'top{j+1}'] = info.name[c]
        r[f'top{j+1}_col'] = c
        r[f'top{j+1}_origin'] = info.origin.get(c)
        r[f'top{j+1}_module'] = mod[c]
        r[f'top{j+1}_rho'] = spearmanr(X[c], Y[a]).statistic
        r[f'top{j+1}_freq'] = freq['GF'][k, i]
    rows.append(r)
T3 = pd.DataFrame(rows).set_index('attr')
T3.to_csv(o('q2_top3.csv'), encoding='utf-8-sig')
print(T3[['label', 'top1', 'top2', 'top3', 'top1_freq', 'top2_freq', 'top3_freq']].
    to_string())
# keep full-data importance for figures
imp = {a: pd.DataFrame({'spearman': full[a]['m1'], 'shap': full[a]['m2'], 'stab': full
    [a]['m3'], 'fused': full[a]['s']},
                       index=cols) for a in attrs}
pd.concat(imp, names=['attr', 'col']).to_csv(o('q2_importance_long.csv'))
np.save(o('q2_freq_GF.npy'), freq['GF'])
save_json({'beta': BETA, 'n_modules': int(mod[mod != 'grey'].nunique()), 'modularity':
     best_s, 'louvain_resolution': best_res,
          'module_sizes': mod.value_counts().to_dict(), 'B': B, 'alpha_rw': ALPHA, '
              lambda_mmr': LAM,
          'stability': summary_stab.to_dict(), 'jaccard': JM.to_dict(),
          'rational_hit_GF': float(rat.hit_any.mean()), 'rational_random': float(rat.
              random_hit_prob.mean()),
          'rational_by_method': {m: float(rat[f'hit_{m}'].mean()) for m in METH}}, '
              q2_summary.json')
```

D.6 问题三 q3.py

```
"""Q3  style-conditional generation of compound profiles + cross-style transfer.

Ours: HS-CLG (hierarchically-shrunk conditional latent Gaussian):
  probabilistic PCA latent space (dimension by CV likelihood); per-style latent
  mean/covariance shrunk toward the global / pooled-within estimates with
  Normal-inverse-Wishart style weights (kappa, nu chosen by held-out likelihood);
  decoding with PPCA noise and detection-limit censoring (zero inflation).
Baselines: unconditional PPCA, independent per-style Gaussian, same-style
    interpolation.
Transfer Pils/Lager -> Tripel: Gaussian optimal-transport map in the latent space,
```

```
source␣residual␣kept␣(beer␣identity),␣then␣a␣QP␣projection␣that␣keeps␣every␣defect
compound␣from␣getting␣worse␣(threshold-based),␣checked␣again␣with␣the␣Q1␣sensory␣model
    .
"""
from common import *
import time, warnings
from scipy.linalg import sqrtm
from scipy.optimize import minimize
from scipy.stats import spearmanr, norm, rankdata, ks_2samp
from sklearn.decomposition import PCA
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import roc_auc_score
from q1model import EnsPredictor, Grader

warnings.filterwarnings('ignore')
rng = np.random.default_rng(SEED)
X, style = load_clean()
Y = load_sensory().loc[X.index].drop(columns=['beer_id', 'tasting_category_fine'])
cols, attrs = list(X.columns), list(Y.columns)
styles = sorted(style.unique())
XMIN, XMAX = X.min(), X.max()


# ================================================================ generators
class HSCLG:
    def __init__(self, d=None, kappa=2.0, nu=10.0):
        self.d, self.kappa, self.nu = d, kappa, nu

    # ---- input-space transform (standardisation here, normal scores in the copula
        subclass)
    def _prep(self, X):
        self.mu, self.sd = X.mean(), X.std().replace(0, 1)

    def to_in(self, X):
        X = np.asarray(X, dtype=float)
        return (X - self.mu.values) / self.sd.values

    def inv(self, Xs):
        Xo = Xs * self.sd.values + self.mu.values
        return np.clip(Xo, XMIN.values, XMAX.values + 0.1 * self.sd.values) # detection
            -limit censoring

    def bound_in(self, j, u):
        return (u - self.mu.values[j]) / self.sd.values[j]

    def fit(self, X, s):
        self._prep(X)
        Xs = self.to_in(X.values)
        self.pca = PCA(self.d, random_state=SEED).fit(Xs)
        Z = self.pca.transform(Xs)
        self.Z, self.s = Z, np.asarray(s)
        self.sig2 = self.pca.noise_variance_
        within = np.zeros((Z.shape[1], Z.shape[1]))
```

```
        nw = 0
        self.stats = {}
        for st in np.unique(self.s):
            Zs = Z[self.s == st]
            m = Zs.mean(0)
            S = np.cov(Zs.T, bias=True) if len(Zs) > 1 else np.zeros_like(within)
            self.stats[st] = (len(Zs), m, S)
            within += S * len(Zs); nw += len(Zs)
        self.Sw = within / nw
        self.Sall = np.cov(Z.T)
        return self

    def latent(self, st):
        n, m, S = self.stats[st]
        mu = n * m / (n + self.kappa)
        lam = n / (n + self.nu)
        Sig = lam * S + (1 - lam) * self.Sw
        return mu, Sig

    def decode(self, Z, resid=None):
        Xs = Z @ self.pca.components_ + self.pca.mean_
        Xs = Xs + (resid if resid is not None else rng.normal(0, np.sqrt(self.sig2), Xs
            .shape))
        return self.inv(Xs)

    def sample(self, st, n):
        mu, Sig = self.latent(st)
        Z = rng.multivariate_normal(mu, Sig, n, method='eigh')
        return self.decode(Z)

    def loglik(self, Xte, ste):
        Z = self.pca.transform(self.to_in(Xte.values))
        ll = []
        for z, st in zip(Z, ste):
            if st not in self.stats:
                continue
            mu, Sig = self.latent(st)
            Sig = Sig + 1e-6 * np.eye(len(mu))
            sgn, logdet = np.linalg.slogdet(Sig)
            r = z - mu
            ll.append(-0.5 * (logdet + r @ np.linalg.solve(Sig, r) + len(mu) * np.log(2
                * np.pi)))
        return np.mean(ll)


class HSCLGCopula(HSCLG):
    """HS-CLG␣on␣Gaussian-copula␣normal␣scores:␣every␣compound␣is␣mapped␣through␣its
␣␣␣␣empirical␣CDF␣(mid-ranks,␣so␣the␣non-detect␣block␣keeps␣its␣mass)␣to␣N(0,1);␣
    generated
␣␣␣␣scores␣are␣mapped␣back␣through␣the␣empirical␣quantile␣function,␣which␣reproduces␣
    the
␣␣␣␣marginal␣shape␣and␣the␣detection-floor␣proportion␣of␣every␣compound␣exactly."""

    def _prep(self, X):
        self.V = np.sort(X.values, axis=0) # n x p sorted training values
        self.n = len(X)
```

```
        self.grid = (np.arange(self.n) + 0.5) / self.n
        self.mu = pd.Series(0.0, index=X.columns); self.sd = pd.Series(1.0, index=X.
            columns)

    def _cdf(self, j, x):
        v = self.V[:, j]
        F = (np.searchsorted(v, x, 'left') + np.searchsorted(v, x, 'right')) / 2 / self
            .n
        return np.clip(F, 0.5 / self.n, 1 - 0.5 / self.n)

    def to_in(self, X):
        X = np.atleast_2d(np.asarray(X, dtype=float))
        return np.column_stack([norm.ppf(self._cdf(j, X[:, j])) for j in range(X.shape
            [1])])

    def inv(self, Xs):
        Q = norm.cdf(Xs)
        return np.column_stack([np.interp(Q[:, j], self.grid, self.V[:, j]) for j in
            range(Q.shape[1])])

    def bound_in(self, j, u):
        # exact inverse of inv(): the same quantile grid is used in both directions
        q = float(np.interp(u, self.V[:, j], self.grid))
        return float(norm.ppf(np.clip(q, 0.5 / self.n, 1 - 0.5 / self.n)))


class Uncond(HSCLG):
    def latent(self, st):
        return np.zeros(self.Z.shape[1]), self.Sall


class IndepGauss:
    def fit(self, X, s):
        self.X, self.s = X, np.asarray(s)
        self.psd = X.groupby(self.s).std().mean()
        return self

    def sample(self, st, n):
        Xs = self.X[self.s == st]
        m = Xs.mean().values
        sd = Xs.std().fillna(0).values if len(Xs) >= 3 else self.psd.values
        sd = np.where(sd > 0, sd, self.psd.values)
        return np.clip(rng.normal(m, sd, (n, len(m))), XMIN.values, XMAX.values)


class Interp:
    def fit(self, X, s):
        self.X, self.s, self.sd = X, np.asarray(s), X.std().values
        return self

    def sample(self, st, n):
        Xs = self.X[self.s == st].values
        i, j = rng.integers(0, len(Xs), n), rng.integers(0, len(Xs), n)
        lam = rng.uniform(0, 1, (n, 1))
        G = lam * Xs[i] + (1 - lam) * Xs[j] + rng.normal(0, 0.1 * self.sd, (n, Xs.shape
            [1]))
```

```
        return np.clip(G, XMIN.values, XMAX.values)


# ================================================================ 1. hyper-parameters
cnt = style.value_counts()
strat = style.where(style.map(cnt) >= 5, 'rare')
skf = StratifiedKFold(5, shuffle=True, random_state=SEED)
folds = list(skf.split(X, strat))
Xs_all = ((X - X.mean()) / X.std()).values
dgrid = [5, 10, 15, 20, 30, 40, 50, 60, 80]
_cop = HSCLGCopula(); _cop._prep(X)
Ns_all = _cop.to_in(X.values)
dll = {d: np.mean([PCA(d).fit(Ns_all[tr]).score(Ns_all[te]) for tr, te in folds]) for
    d in dgrid}
D_BEST = max(dll, key=dll.get)
print('PPCA␣CV␣loglik', {k: round(v, 2) for k, v in dll.items()}, '->␣d␣=', D_BEST)
hp = []
for kappa in [0.5, 1, 2, 4, 8]:
    for nu in [2, 5, 10, 20, 40, 80, 160, 320]:
        ll = np.mean([HSCLGCopula(D_BEST, kappa, nu).fit(X.iloc[tr], style.iloc[tr]).
            loglik(X.iloc[te], style.iloc[te].values)
                  for tr, te in folds])
        hp.append(dict(kappa=kappa, nu=nu, heldout_loglik=ll))
hp = pd.DataFrame(hp)
KAPPA, NU = hp.loc[hp.heldout_loglik.idxmax(), ['kappa', 'nu']]
print(hp.sort_values('heldout_loglik', ascending=False).head(5)); print('kappa', KAPPA
    , 'nu', NU)
hp.to_csv(o('q3_hyperparams.csv'), index=False)
pd.Series(dll).to_csv(o('q3_ppca_dim.csv'))

# ================================================================ 2. evaluation
ev_pca = PCA(10, random_state=SEED).fit(Xs_all)
gsc = StandardScaler().fit(X.values)


def edist(A, B):
    A, B = ev_pca.transform(gsc.transform(A)), ev_pca.transform(gsc.transform(B))
    ab = np.linalg.norm(A[:, None] - B[None], axis=2).mean()
    aa = np.linalg.norm(A[:, None] - A[None], axis=2).mean()
    bb = np.linalg.norm(B[:, None] - B[None], axis=2).mean()
    return 2 * ab - aa - bb


floor_cols = np.where((X == XMIN).sum() >= 10)[0]
GENS = {'Uncond-PPCA': lambda: Uncond(D_BEST), 'Indep-Gauss': IndepGauss, 'Interp':
    Interp,
        'HS-CLG': lambda: HSCLG(D_BEST, KAPPA, NU), 'HS-CLG+Copula(ours)': lambda:
            HSCLGCopula(D_BEST, KAPPA, NU)}
NG = 60
ev_rows = []
for fi, (tr, te) in enumerate(folds):
    Xtr, Xte, str_, ste = X.iloc[tr], X.iloc[te], style.iloc[tr], style.iloc[te]
    clf = LogisticRegression(C=0.05, max_iter=3000).fit(gsc.transform(Xtr.values), str_
        )
    acc_real = (clf.predict(gsc.transform(Xte.values)) == ste.values).mean()
    for gname, G in GENS.items():
```

```
    g = G().fit(Xtr, str_)
    gen, glab = [], []
    for st in styles:
        if (str_ == st).sum() == 0:
            continue
        gen.append(g.sample(st, NG)); glab += [st] * NG
    gen = np.vstack(gen); glab = np.array(glab)
    # conditional consistency
    acc = (clf.predict(gsc.transform(gen)) == glab).mean()
    # style retrieval by energy distance to held-out real beers
    tst = [st for st in styles if (ste == st).sum() >= 2]
    hit = 0
    for st in tst:
        gs = gen[glab == st]
        dists = {s2: edist(gs, Xte.values[ste.values == s2]) for s2 in tst}
        hit += (min(dists, key=dists.get) == st)
    ret = hit / len(tst)
    ed_same = np.mean([edist(gen[glab == st], Xte.values[ste.values == st]) for st
        in tst])
    # C2ST: real(train) vs generated, matched style mix
    idx = np.concatenate([np.where(glab == st)[0][:(str_ == st).sum()] for st in
        styles if (str_ == st).sum()])
    Fk = np.vstack([Xtr.values, gen[idx]]); lab = np.r_[np.zeros(len(Xtr)), np.ones
        (len(idx))]
    # shuffled stratified folds: the generated block is ordered by style,
        unshuffled folds would hold
    # out whole styles of the generated class only (kept as a diagnostic column)
    c2cv = StratifiedKFold(5, shuffle=True, random_state=SEED)
    auc = cross_val_score(RandomForestClassifier(300, random_state=SEED, n_jobs=8),
         Fk, lab, cv=c2cv,
                          scoring='roc_auc').mean()
    bacc = cross_val_score(RandomForestClassifier(300, random_state=SEED, n_jobs=8)
        , Fk, lab, cv=c2cv,
                           scoring='balanced_accuracy').mean()
    auc_ns = cross_val_score(RandomForestClassifier(300, random_state=SEED, n_jobs
        =8), Fk, lab, cv=5,
                             scoring='roc_auc').mean()
    # correlation structure (Pearson on values, Spearman on ranks), marginals &
        detection-rate fidelity
    Cg, Cr = np.corrcoef(gen[idx].T), np.corrcoef(Xtr.values.T)
    Cg, Cr = np.nan_to_num(Cg), np.nan_to_num(Cr)
    cerr = np.linalg.norm(Cg - Cr) / np.linalg.norm(Cr)
    Sg = np.nan_to_num(np.corrcoef(rankdata(gen[idx], axis=0).T))
    Sr = np.nan_to_num(np.corrcoef(rankdata(Xtr.values, axis=0).T))
    cerr_sp = np.linalg.norm(Sg - Sr) / np.linalg.norm(Sr)
    ks = np.mean([ks_2samp(gen[idx][:, j], Xtr.values[:, j]).statistic for j in
        range(gen.shape[1])])
    det_g = (gen[idx][:, floor_cols] <= XMIN.values[floor_cols] + 1e-9).mean(0)
    det_r = (Xtr.values[:, floor_cols] <= XMIN.values[floor_cols] + 1e-9).mean(0)
    ev_rows.append(dict(fold=fi, gen=gname, cond_acc=acc, real_acc=acc_real,
        retrieval=ret, ED_same=ed_same,
                        C2ST_AUC=auc, C2ST_dev=abs(auc - 0.5), C2ST_bacc=bacc,
                            C2ST_AUC_unshuffled=auc_ns,
                        corr_err=cerr, corr_err_spearman=cerr_sp, KS_mean=ks,
                        floor_rate_err=np.abs(det_g - det_r).mean()))
print('fold', fi, 'done')
```

```
ev = pd.DataFrame(ev_rows)
evs = ev.groupby('gen').agg(['mean', 'std']).drop(columns='fold')
print(evs.round(3))
ev.to_csv(o('q3_gen_eval_folds.csv'), index=False)
evs.to_csv(o('q3_gen_eval.csv'))

# ============================================================== 3. final model + sensory portraits
gen_model = HSCLGCopula(D_BEST, KAPPA, NU).fit(X, style)
pred = EnsPredictor().fit(X, Y)
grader = Grader(Y)
clf_all = LogisticRegression(C=0.05, max_iter=3000).fit(gsc.transform(X.values), style)
port_rows, gen_store = [], {}
for st in styles:
    G = gen_model.sample(st, 300)
    gen_store[st] = G
    zg = pred.predict(pd.DataFrame(G, columns=cols)).mean()
    zr_pred = pred.predict(X[style == st]).mean()
    zr_panel = Y[style == st].mean()
    gg, gp = grader.cont([zg.values]).iloc[0], grader.cont([zr_panel.values]).iloc[0]
    port_rows.append(dict(style=st, n_real=int((style == st).sum()),
                          rho_gen_vs_panel=spearmanr(zg, zr_panel).statistic,
                          rho_realpred_vs_panel=spearmanr(zr_pred, zr_panel).statistic,
                          grade_MAE_gen_vs_panel=np.abs(gg - gp).mean(),
                          cond_acc=(clf_all.predict(gsc.transform(G)) == st).mean()))
    for a in attrs:
        port_rows[-1]['g_' + a] = gg[a]
        port_rows[-1]['p_' + a] = gp[a]
port = pd.DataFrame(port_rows).set_index('style')
print(port[['n_real', 'rho_gen_vs_panel', 'rho_realpred_vs_panel', 'grade_MAE_gen_vs_panel', 'cond_acc']].round(3))
port.to_csv(o('q3_portraits.csv'))
for st in ['Tripel', 'Kriek', 'Lambic']:
    G = pd.DataFrame(gen_store[st], columns=cols) # all 300 generated profiles
    G.insert(0, 'style', st)
    G.to_csv(o(f'q3_generated_{st}.csv'), index=False)
# compound-level fingerprint of the three target styles: mean generated vs real (top deviating from global)
fp = []
for st in ['Tripel', 'Kriek', 'Lambic']:
    gm_, rm_ = gen_store[st].mean(0), X[style == st].mean().values
    zdev = (gm_ - X.mean().values) / X.std().values
    for i in np.argsort(-np.abs(zdev))[:12]:
        fp.append(dict(style=st, col=cols[i], gen_mean=gm_[i], real_mean=rm_[i],
            z_vs_all=zdev[i]))
pd.DataFrame(fp).to_csv(o('q3_fingerprint_top.csv'), index=False)

# ============================================================== 4. transfer Pils/Lager -> Tripel
SRC, TGT = 'Pils/Lager', 'Tripel'
cal = load_calibrated()
P90_t2n = float(np.log10(cal['X.E..2.nonenal'].astype(float).quantile(0.9)))
DEF = { # col: (log10 threshold, sensory attr)
    'DMS': (np.log10(17.25), 'dms'), # ug/L, 8.xlsx
    'X2.3.butanedione': (np.log10(0.09), 'diacetyl'), # mg/L, 8.xlsx median (literature
```

```
        0.1)
    'X4VG': (np.log10(0.2), 'X4vg'), # mg/L, 8.xlsx
    'X.E..2.nonenal': (P90_t2n, 't2n'), # no absolute threshold -> data P90
}
SENS_DEF = ['dms', 'diacetyl', 'X4vg', 't2n', 'stale_hops']
d_idx = [cols.index(c) for c in DEF]
Wc = gen_model.pca.components_ # d x p
mu_s, sd_s = gen_model.mu.values, gen_model.sd.values
muT, SigT = gen_model.latent(TGT)
muP, SigP = gen_model.latent(SRC)
SigT_inv = np.linalg.inv(SigT + 1e-6 * np.eye(len(muT)))
rP = np.real(sqrtm(SigP)); rPi = np.linalg.inv(rP)
A_ot = rPi @ np.real(sqrtm(rP @ SigT @ rP)) @ rPi
ptr = list(clf_all.classes_).index(TGT)


Zall = gen_model.pca.transform(gen_model.to_in(X.values))
clf_z = LogisticRegression(C=0.5, max_iter=5000).fit(Zall, style)
pzi = list(clf_z.classes_).index(TGT)
print('latent␣style␣classifier␣CV␣acc', cross_val_score(LogisticRegression(C=0.5,
    max_iter=5000), Zall, style, cv=skf.split(X, strat)).mean())
LAT = {st_: gen_model.latent(st_) for st_ in styles}
LATinv = {st_: np.linalg.inv(S_ + 1e-6 * np.eye(len(m_))) for st_, (m_, S_) in LAT.
    items()}


def pz(z):
    return float(clf_z.predict_proba(z[None])[0, pzi])


def rankTz(z):
    """rank␣of␣Tripel␣under␣the␣latent␣logistic␣discriminator␣(1␣=␣predicted␣style)"""
    pr = clf_z.predict_proba(z[None])[0]
    return int((pr > pr[pzi]).sum() + 1)


def rankT(z):
    d = {st_: (z - LAT[st_][0]) @ LATinv[st_] @ (z - LAT[st_][0]) + np.linalg.slogdet(
        LAT[st_][1])[1] for st_ in styles}
    return int(sorted(d, key=d.get).index(TGT) + 1)


def mahal_T(z):
    r = z - muT
    return float(np.sqrt(r @ SigT_inv @ r))


DIAG = {}


def transfer(x, alpha=1.0, constrain=True, slack=0.0):
    xs = gen_model.to_in(x[None])[0]
    z = gen_model.pca.transform(xs[None])[0]
    resid = xs - (z @ Wc + gen_model.pca.mean_)
    zt = z + alpha * ((muT + A_ot @ (z - muP)) - z)
    if not constrain:
```

```
        return gen_model.decode(zt[None], resid[None])[0], zt, 0
    # upper bounds on log10 concentration of every defect compound
    ub = np.array([gen_model.bound_in(i, max(x[i], DEF[c][0] + np.log10(0.8))) - slack
        for c, i in zip(DEF, d_idx)])
    a_mat = Wc[:, d_idx].T # input-space x_d = a_mat @ z + b_d (monotone in x_d)
    b_vec = gen_model.pca.mean_[d_idx] + resid[d_idx]
    cons = [{'type': 'ineq', 'fun': lambda zz: ub - (a_mat @ zz + b_vec), 'jac': lambda
         zz: -a_mat}]
    res = minimize(lambda zz: (zz - zt) @ SigT_inv @ (zz - zt), zt, jac=lambda zz: 2 *
        SigT_inv @ (zz - zt),
                   constraints=cons, method='SLSQP', options={'maxiter': 500, 'ftol': 1e
                       -10})
    zc = res.x
    active = int(np.sum(np.abs(ub - (a_mat @ zc + b_vec)) < 1e-4))
    xc = gen_model.decode(zc[None], resid[None])[0]
    clip = 0.0
    for c, i in zip(DEF, d_idx): # final safeguard against interpolation round-off
        u = max(x[i], DEF[c][0] + np.log10(0.8))
        clip = max(clip, xc[i] - u)
        xc[i] = min(xc[i], u)
    # QP diagnostics: convergence, max constraint violation in the latent problem,
        round-off clip (log10)
    DIAG.update(success=bool(res.success), viol=float(max(0.0, (a_mat @ zc + b_vec - ub
        ).max())), clip=float(clip))
    return xc, zc, active


src_ids = X.index[style == SRC]
tr_rows, curve = [], []
for b in src_ids:
    x = X.loc[b].values
    g0 = grader.grade(pred.predict(pd.DataFrame([x], columns=cols))).iloc[0]
    z0 = gen_model.pca.transform(gen_model.to_in(x[None]))[0]
    # unconstrained OT
    xu, zu, _ = transfer(x, 1.0, constrain=False)
    gu = grader.grade(pred.predict(pd.DataFrame([xu], columns=cols))).iloc[0]
    # constrained, with sensory re-check: back off alpha until no predicted defect
        grade worsens
    alpha = 1.0
    while True:
        xc, zc, act = transfer(x, alpha, constrain=True)
        gc = grader.grade(pred.predict(pd.DataFrame([xc], columns=cols))).iloc[0]
        worse = [a for a in SENS_DEF if gc[a] > max(g0[a], 1)]
        if not worse or alpha <= 0.3:
            break
        alpha = round(alpha - 0.05, 2)
    row = dict(beer=int(b), alpha=alpha, active_constraints=act,
               pT_before=clf_all.predict_proba(gsc.transform(x[None]))[0, ptr],
               pT_uncon=clf_all.predict_proba(gsc.transform(xu[None]))[0, ptr],
               pT_after=clf_all.predict_proba(gsc.transform(xc[None]))[0, ptr],
               dT_before=mahal_T(z0), dT_uncon=mahal_T(zu), dT_after=mahal_T(zc),
               moved_sd=float(np.linalg.norm((xc - x) / X.std().values) / np.sqrt(len(x)
                   )),
               sens_def_worse_uncon=int(sum(gu[a] > max(g0[a], 1) for a in SENS_DEF)),
               sens_def_worse_after=int(sum(gc[a] > max(g0[a], 1) for a in SENS_DEF)),
               pTz_before=pz(z0), pTz_uncon=pz(zu), pTz_after=pz(zc), rankT_before=rankT
```

```
                (z0), rankT_after=rankT(zc),
            rankTz_before=rankTz(z0), rankTz_uncon=rankTz(zu), rankTz_after=rankTz(zc
                ), rankT_uncon=rankT(zu),
            qp_success=DIAG['success'], qp_viol=DIAG['viol'], qp_clip=DIAG['clip'])
    for c, i in zip(DEF, d_idx):
        thr = DEF[c][0]
        row[f'{c}_before'], row[f'{c}_uncon'], row[f'{c}_after'] = x[i], xu[i], xc[i]
        row[f'{c}_OAV_before'] = 10 ** (x[i] - thr)
        row[f'{c}_OAV_uncon'] = 10 ** (xu[i] - thr)
        row[f'{c}_OAV_after'] = 10 ** (xc[i] - thr)
        row[f'{c}_worse_uncon'] = bool(xu[i] > max(x[i], thr + np.log10(0.8)) + 1e-3)
        row[f'{c}_worse_after'] = bool(xc[i] > max(x[i], thr + np.log10(0.8)) + 1e-3)
    for a in ['alcohol', 'body', 'F_esters_all', 'A_esters_all', 'F_esters_isoaa', '
        sweetness', 'bitternes',
             'F_malt_all', 'overall'] + SENS_DEF:
        row[f'g0_{a}'], row[f'gu_{a}'], row[f'g1_{a}'] = int(g0[a]), int(gu[a]), int(gc
            [a])
    tr_rows.append(row)
    for al in [0, 0.2, 0.4, 0.6, 0.8, 1.0]:
        xa, za, _ = transfer(x, al, constrain=True)
        xb, zb, _ = transfer(x, al, constrain=False)
        curve.append(dict(beer=int(b), alpha=al,
                          pT_con=clf_all.predict_proba(gsc.transform(xa[None]))[0, ptr],
                          pT_uncon=clf_all.predict_proba(gsc.transform(xb[None]))[0, ptr],
                          dT_con=mahal_T(za), dT_uncon=mahal_T(zb), pTz_con=pz(za),
                              pTz_uncon=pz(zb),
                          n_def_worse_con=sum(xa[i] > max(x[i], DEF[c][0] + np.log10(0.8))
                               + 1e-3 for c, i in zip(DEF, d_idx)),
                          n_def_worse_uncon=sum(xb[i] > max(x[i], DEF[c][0] + np.log10
                              (0.8)) + 1e-3 for c, i in zip(DEF, d_idx))))
TR = pd.DataFrame(tr_rows).set_index('beer')
CV = pd.DataFrame(curve)
TR.to_csv(o('q3_transfer.csv'))
CV.to_csv(o('q3_transfer_curve.csv'), index=False)
print(TR[['alpha', 'active_constraints', 'pTz_before', 'pTz_uncon', 'pTz_after', '
    rankTz_after', 'rankT_before', 'rankT_after',
         'qp_success', 'qp_viol', 'qp_clip', 'dT_before', 'dT_uncon', 'dT_after',
         'sens_def_worse_uncon', 'sens_def_worse_after']].round(3))
print('chem defects worsened (uncon):', {c: int(TR[f'{c}_worse_uncon'].sum()) for c in
     DEF})
print('chem defects worsened (after):', {c: int(TR[f'{c}_worse_after'].sum()) for c in
     DEF})
print(CV.groupby('alpha').mean().drop(columns='beer').round(3))
dT_ref = [mahal_T(z_) for z_ in Zall[(style == TGT).values]]
print('real Tripel Mahalanobis median', np.median(dT_ref), 'real Tripel latent pT
    median', np.median([pz(z_) for z_ in Zall[(style == TGT).values]]))
# Tripel reference grades for the same attributes
ref = grader.grade(pred.predict(X[style == TGT])).mean()
save_json({'d': D_BEST, 'kappa': KAPPA, 'nu': NU, 'ppca_ll': dll,
          'gen_eval': ev.drop(columns='fold').groupby('gen').mean().to_dict(),
          'P90_t2n_log10': P90_t2n, 'transfer_mean': TR.mean(numeric_only=True).to_dict
              (),
          'tripel_pred_grade_mean': ref.to_dict(), 'dT_ref_median': float(np.median(
              dT_ref)),
          'pTz_real_tripel_median': float(np.median([pz(z_) for z_ in Zall[(style ==
              TGT).values]]))}, 'q3_summary.json')
```

D.7 问题三迁移方法对比 q3_cmp.py

```
"""Q3 transfer: comparison of transfer methods (Pils/Lager -> Tripel).
Reuses the generator classes of q3.py and the tuned (d, kappa, nu) in q3_summary.json.
Methods: mean shift, linear interpolation to the Tripel centre, nearest-Tripel
    replacement,
Gaussian OT (no constraint), Gaussian OT + threshold constraint (ours), Gaussian OT +
    strict non-increase."""
import time
src = open(__file__.replace('q3_cmp.py', 'q3.py'), encoding='utf-8').read()
exec(src[:src.index('cnt = style.value_counts()')])
from sklearn.linear_model import LogisticRegression
from scipy.linalg import sqrtm

S = json.load(open(o('q3_summary.json'), encoding='utf-8'))
D_BEST, KAPPA, NU = int(S['d']), float(S['kappa']), float(S['nu'])
gen_model = HSCLGCopula(D_BEST, KAPPA, NU).fit(X, style)
pred = EnsPredictor().fit(X, Y)
grader = Grader(Y)
SRC, TGT = 'Pils/Lager', 'Tripel'
cal = load_calibrated()
P90_t2n = float(np.log10(cal['X.E..2.nonenal'].astype(float).quantile(0.9)))
DEF = {'DMS': (np.log10(17.25), 'dms'), 'X2.3.butanedione': (np.log10(0.09), 'diacetyl
    '),
      'X4VG': (np.log10(0.2), 'X4vg'), 'X.E..2.nonenal': (P90_t2n, 't2n')}
SENS_DEF = ['dms', 'diacetyl', 'X4vg', 't2n', 'stale_hops']
d_idx = [cols.index(c) for c in DEF]
Wc = gen_model.pca.components_
muT, SigT = gen_model.latent(TGT)
muP, SigP = gen_model.latent(SRC)
SigT_inv = np.linalg.inv(SigT + 1e-6 * np.eye(len(muT)))
rP = np.real(sqrtm(SigP)); rPi = np.linalg.inv(rP)
A_ot = rPi @ np.real(sqrtm(rP @ SigT @ rP)) @ rPi
Zall = gen_model.pca.transform(gen_model.to_in(X.values))
clf_z = LogisticRegression(C=0.5, max_iter=5000).fit(Zall, style)
pzi = list(clf_z.classes_).index(TGT)
LAT = {s_: gen_model.latent(s_) for s_ in styles}
LATinv = {s_: np.linalg.inv(Sg + 1e-6 * np.eye(len(m))) for s_, (m, Sg) in LAT.items()
    }
Ztrip = Zall[(style == TGT).values]
Xtrip = X[style == TGT].values


def pz(z):
    return float(clf_z.predict_proba(z[None])[0, pzi])


def rankTz(z):
    pr = clf_z.predict_proba(z[None])[0]
    return int((pr > pr[pzi]).sum() + 1)


def rankT(z):
    d = {s_: (z - LAT[s_][0]) @ LATinv[s_] @ (z - LAT[s_][0]) + np.linalg.slogdet(LAT[
        s_][1])[1] for s_ in styles}
    return int(sorted(d, key=d.get).index(TGT) + 1)
```

```
def mahal_T(z):
    r = z - muT
    return float(np.sqrt(r @ SigT_inv @ r))


def enc(x):
    xs = gen_model.to_in(x[None])[0]
    z = gen_model.pca.transform(xs[None])[0]
    return xs, z, xs - (z @ Wc + gen_model.pca.mean_)


def ot_target(z, alpha=1.0):
    return z + alpha * ((muT + A_ot @ (z - muP)) - z)


def project(x, zt, resid, ub_log):
    ub = np.array([gen_model.bound_in(i, u) for i, u in zip(d_idx, ub_log)])
    a_mat = Wc[:, d_idx].T
    b_vec = gen_model.pca.mean_[d_idx] + resid[d_idx]
    cons = [{'type': 'ineq', 'fun': lambda zz: ub - (a_mat @ zz + b_vec), 'jac': lambda
         zz: -a_mat}]
    res = minimize(lambda zz: (zz - zt) @ SigT_inv @ (zz - zt), zt, jac=lambda zz: 2 *
        SigT_inv @ (zz - zt),
                   constraints=cons, method='SLSQP', options={'maxiter': 500, 'ftol': 1e
                       -10})
    zc = res.x
    xc = gen_model.decode(zc[None], resid[None])[0]
    for i, u in zip(d_idx, ub_log):
        xc[i] = min(xc[i], u)
    return xc, zc


def method(name, x):
    xs, z, resid = enc(x)
    if name == '均值平移':
        zn = z + (muT - muP)
        return gen_model.decode(zn[None], resid[None])[0], zn
    if name == '线性插值':
        zn = 0.5 * z + 0.5 * muT
        return gen_model.decode(zn[None], resid[None])[0], zn
    if name == '最近邻替换':
        k = int(np.argmin([(z - zt) @ SigT_inv @ (z - zt) for zt in Ztrip]))
        return Xtrip[k].copy(), Ztrip[k]
    zt = ot_target(z)
    if name == 'OT无约束':
        return gen_model.decode(zt[None], resid[None])[0], zt
    if name == 'OT+阈值约束(本文)':
        ub = [max(x[i], DEF[c][0] + np.log10(0.8)) for c, i in zip(DEF, d_idx)]
        return project(x, zt, resid, ub)
    if name == 'OT+严格不升':
        ub = [x[i] for i in d_idx]
        return project(x, zt, resid, ub)
    raise ValueError(name)
```

```
METHODS = ['均值平移', '线性插值', '最近邻替换', 'OT无约束', 'OT+阈值约束(本文)', 'OT+严
    格不升']
rows = []
for b in X.index[style == SRC]:
    x = X.loc[b].values
    g0 = grader.grade(pred.predict(pd.DataFrame([x], columns=cols))).iloc[0]
    for mname in METHODS:
        t0 = time.perf_counter()
        xn, zn = method(mname, x)
        dt = time.perf_counter() - t0
        gn = grader.grade(pred.predict(pd.DataFrame([xn], columns=cols))).iloc[0]
        thr_worse = sum(xn[i] > max(x[i], DEF[c][0] + np.log10(0.8)) + 1e-3 for c, i in
            zip(DEF, d_idx))
        any_up = sum(xn[i] > x[i] + 1e-3 for i in d_idx)
        rows.append(dict(beer=int(b), method=mname, pTz=pz(zn), rank1=int(rankTz(zn) ==
            1), rank1_gauss=int(rankT(zn) == 1),
                         p_gt_half=int(pz(zn) > 0.5), dT=mahal_T(zn),
                         thr_worse=int(thr_worse), conc_up=int(any_up),
                         sens_worse=int(sum(gn[a] > max(g0[a], 1) for a in SENS_DEF)),
                         moved_sd=float(np.linalg.norm((xn - x) / X.std().values) / np.
                             sqrt(len(x))),
                         alcohol=int(gn['alcohol']), esters=int(gn['F_esters_all']), ms=dt
                              * 1000))
R = pd.DataFrame(rows)
T = R.groupby('method')[['pTz', 'rank1', 'rank1_gauss', 'p_gt_half', 'dT', 'thr_worse'
    , 'conc_up', 'sens_worse', 'moved_sd', 'alcohol', 'esters', 'ms']] \
    .agg({'pTz': 'mean', 'rank1': 'sum', 'rank1_gauss': 'sum', 'p_gt_half': 'sum', 'dT'
        : 'mean', 'thr_worse': 'sum', 'conc_up': 'sum', 'sens_worse': 'sum',
          'moved_sd': 'mean', 'alcohol': 'mean', 'esters': 'mean', 'ms': 'mean'}).loc[
              METHODS]
print(T.round(3).to_string())
R.to_csv(o('q3_transfer_cmp_beer.csv'), index=False)
T.to_csv(o('q3_transfer_cmp.csv'))
# OAV means before / after for the honest defect report
OAV = {}
for mname in ['OT无约束', 'OT+阈值约束(本文)', 'OT+严格不升']:
    acc = {c: [] for c in DEF}
    acc0 = {c: [] for c in DEF}
    for b in X.index[style == SRC]:
        x = X.loc[b].values
        xn, _ = method(mname, x)
        for c, i in zip(DEF, d_idx):
            acc[c].append(10 ** (xn[i] - DEF[c][0])); acc0[c].append(10 ** (x[i] - DEF[c
                ][0]))
    OAV[mname] = {c: float(np.median(v)) for c, v in acc.items()}
    OAV['迁移前'] = {c: float(np.median(v)) for c, v in acc0.items()}
print(pd.DataFrame(OAV).round(3))
pd.DataFrame(OAV).to_csv(o('q3_transfer_oav.csv'))
```

D.8 问题二成对置信区间 q2_ci.py

```
"""Q2␣paired␣comparison:␣per-attribute␣stability␣/␣redundancy␣/␣rationality␣of␣GF-MMR␣
    minus␣each␣baseline,
95%␣bootstrap␣CI␣over␣the␣50␣sensory␣attributes␣(paired,␣the␣same␣attributes␣for␣every
    ␣method).
Reads␣the␣saved␣bootstrap␣selections␣of␣q2.py,␣no␣refitting."""
from common import *
```

```
import pickle

D = pickle.load(open(o('q2_boot.pkl'), 'rb'))
BOOT, full = D['BOOT'], D['full_top']
attrs = list(full)
modinfo = pd.read_csv(o('q2_modules.csv'), index_col=0)
cols = list(modinfo.index)
mod = modinfo['module']
TOM = pd.read_csv(o('q2_tom.csv'), index_col=0).loc[cols, cols].values
rat = pd.read_csv(o('q2_rationality.csv'), index_col=0)
METH = ['M1', 'M2', 'M3', 'FUSE', 'GF']


def jacc(a, b):
    a, b = set(a), set(b)
    return len(a & b) / len(a | b)


def modset(sel):
    return {mod.iloc[i] if mod.iloc[i] != 'grey' else cols[i] for i in sel}


per = {}
for m in METH:
    per[m] = pd.DataFrame({
        'module_freq': [np.mean([np.mean([mod.iloc[i] in modset(r[a][m]) if mod.iloc[i]
             != 'grey' else i in r[a][m]
                                      for r in BOOT]) for i in full[a][m]]) for a in
                                          attrs],
        'module_jaccard': [np.mean([jacc(modset(r[a][m]), modset(full[a][m])) for r in
            BOOT]) for a in attrs],
        'TOM_top3': [np.mean([TOM[i, j] for i in full[a][m] for j in full[a][m] if i <
            j]) for a in attrs],
        'hit': [float(rat.loc[a, f'hit_{m}']) if a in rat.index else np.nan for a in
            attrs]}, index=attrs)
rng = np.random.default_rng(SEED)
rows = []
for m in ['M1', 'M2', 'M3', 'FUSE']:
    for k in ['module_freq', 'module_jaccard', 'TOM_top3', 'hit']:
        d = (per['GF'][k] - per[m][k]).dropna().values
        bs = [rng.choice(d, len(d)).mean() for _ in range(5000)]
        rows.append(dict(ref=m, metric=k, n=len(d), diff=d.mean(), ci_lo=np.percentile(
            bs, 2.5),
                         ci_hi=np.percentile(bs, 97.5)))
R = pd.DataFrame(rows)
print(R.round(4).to_string())
R.to_csv(o('q2_paired_ci.csv'), index=False)
print({m: per[m].mean().round(4).to_dict() for m in METH})
```

D.9 灵敏度分析 sens.py

```
"""Sensitivity␣analysis.
Q2:␣GF-MMR␣parameters␣alpha␣(graph␣smoothing)␣and␣lambda␣(redundancy␣penalty),␣using␣
    the
␣␣␣␣saved␣full-data␣importances:␣Top-3␣agreement␣with␣the␣baseline,␣redundancy,␣
    rationality.
Q1:␣grade␣step␣factor␣(already␣in␣q1_step_sensitivity.csv)␣is␣only␣collected␣here.
```

```
Q3: held-out log-likelihood surface over (kappa, nu) (from q3_hyperparams.csv) is
    collected here."""
from common import *
from scipy.stats import rankdata

X, style = load_clean()
cols = list(X.columns)
imp = pd.read_csv(o('q2_importance_long.csv'), index_col=[0, 1])
TOM = pd.read_csv(o('q2_tom.csv'), index_col=0).loc[cols, cols].values
rat = pd.read_csv(o('q2_rationality.csv'), index_col=0)
info = pd.read_csv(o('compound_info.csv')).set_index('col')
AGG = ['esters_sum', 'aroma_hops_sum', 'acids_sum', 'sulfur_sum', 'kcalperc']
cand = np.array([c not in AGG for c in cols])
W = TOM.copy(); np.fill_diagonal(W, 0); Wn = W / W.sum(1, keepdims=True)
attrs = list(dict.fromkeys(imp.index.get_level_values(0)))


def borda(v):
    r = rankdata(-v, method='average')
    return 1 - (r - 1) / (len(v) - 1)


def select(a, alpha, lam):
    d = imp.loc[a].loc[cols]
    s = np.mean([borda(d['spearman'].values), borda(d['shap'].values), borda(d['stab'].
        values)], axis=0)
    s = (1 - alpha) * s + alpha * (Wn @ s)
    s = np.where(cand, s, -np.inf)
    sel = [int(np.argmax(s))]
    while len(sel) < 3:
        g = s - lam * TOM[:, sel].max(1)
        g[sel] = -np.inf
        sel.append(int(np.argmax(g)))
    return sel


base = {a: select(a, 0.3, 0.6) for a in attrs}
rows = []
for alpha in [0.0, 0.1, 0.2, 0.3, 0.4, 0.5]:
    for lam in [0.0, 0.3, 0.6, 0.9, 1.2]:
        jac, red = [], []
        for a in attrs:
            s = select(a, alpha, lam)
            jac.append(len(set(s) & set(base[a])) / len(set(s) | set(base[a])))
            red.append(np.mean([TOM[i, j] for i in s for j in s if i < j]))
        rows.append(dict(alpha=alpha, lam=lam, jaccard_vs_base=np.mean(jac), mean_TOM=
            np.mean(red)))
S2 = pd.DataFrame(rows)
print(S2.pivot(index='alpha', columns='lam', values='jaccard_vs_base').round(3))
print(S2.pivot(index='alpha', columns='lam', values='mean_TOM').round(3))
S2.to_csv(o('sens_q2.csv'), index=False)
```

D.10 公共模块 common.py

```
"""Shared paths and loaders for MathorCup 2026 A (beer flavour)."""
import os, sys, json
import numpy as np, pandas as pd
```

```
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA = os.path.join(ROOT, 'official', '附件')
OUT = os.path.join(ROOT, 'out')
os.makedirs(OUT, exist_ok=True)
SEED = 2026
try:
    sys.stdout.reconfigure(encoding='utf-8')
except Exception:
    pass
pd.set_option('display.width', 250)
pd.set_option('display.max_rows', 400)
pd.set_option('display.max_columns', 60)

META = ['beer', 'beer_id', 'tasting_category_fine']


def p(*a):
    return os.path.join(DATA, *a)


def o(*a):
    return os.path.join(OUT, *a)


def load_chem():
    d = pd.read_csv(p('1.csv'))
    d = d.set_index('beer').sort_index()
    return d


def load_sensory():
    d = pd.read_csv(p('4.csv')).set_index('beer').sort_index()
    return d


def load_calibrated():
    c = pd.read_excel(p('8.xlsx'), 'Calibrated␣chemical␣dataset').set_index('beer').
        sort_index()
    return c


def load_thresholds():
    return pd.read_excel(p('8.xlsx'), 'Sensory␣thresholds')


def load_sensory_names():
    return pd.read_excel(p('8.xlsx'), 'Sensory␣names')


def save_json(obj, name):
    with open(o(name), 'w', encoding='utf-8') as f:
        json.dump(obj, f, ensure_ascii=False, indent=1, default=float)


def load_clean():
```

```
    """Preprocessed␣chemistry␣matrix␣(log10␣scale,␣winsorised,␣imputed)␣+␣style."""
    X = pd.read_csv(o('X_clean.csv'), index_col=0)
    style = pd.read_csv(o('style.csv'), index_col=0)['style']
    return X, style.loc[X.index]


def sensory_label_map():
    """4.csv␣attribute␣name␣->␣label␣used␣in␣5.csv␣/␣6.csv␣(via␣8.xlsx␣Sensory␣names)."
        ""
    sn = load_sensory_names()
    sn = sn[sn.dataset == 'Trained␣panel']
    m = dict(zip(sn.R_name, sn.CleanName_PanelFeedback.astype(str).str.strip()))
    return m


def _norm(s):
    return '␣'.join(str(s).replace('\r', '␣').replace('\n', '␣').split()).lower()


def load_r5():
    """5.csv␣re-indexed␣by␣4.csv␣attribute␣names␣(explicit␣label␣matching,␣not␣order)."
        ""
    R = pd.read_csv(p('5.csv'), index_col=0)
    attrs = [c for c in load_sensory().columns if c not in META]
    lab = {_norm(v): k for k, v in sensory_label_map().items()}
    idx = [lab.get(_norm(x)) for x in R.index]
    miss = [x for x, i in zip(R.index, idx) if i is None]
    # fall back to prefix matching for labels that differ slightly
    for j, x in enumerate(R.index):
        if idx[j] is None:
            cands = [k for kk, k in lab.items() if _norm(x).startswith(kk[:12])]
            idx[j] = cands[0] if len(cands) == 1 else None
    assert None not in idx, ('unmatched␣5.csv␣labels', [x for x, i in zip(R.index, idx)
         if i is None])
    R.index = idx; R.columns = idx
    return R.loc[attrs, attrs]
```